

帕金森病的特征识别和声音诊断模型

摘要：帕金森病常伴随发声稳定性、声带振动和谱结构等方面的改变，语音声学特征为辅助筛查和探索性分型提供了低侵入、可重复采集的数据基础。本文围绕两个含重复录音结构的帕金森病语音数据集，建立受试者级二分类辅助诊断模型、判别性声学特征综合评价模型以及 PD 内部潜在语音表型无监督分型模型。针对同一受试者多次录音可能导致的样本单位错误，本文将受试者作为核心分析单位；在监督诊断模型中采用按受试者分组的交叉验证，并将标准化、模型训练和相关特征处理限制在训练折内部完成。

对问题一和问题三，本文构建并比较 Logistic Regression、SVM-RBF 和 Random Forest 三类二分类辅助诊断模型。Dataset 1 主分析采用受试者级指标，Random Forest 的准确率和 AUC 分别为 0.8290 和 0.8511，但其特异度为 0.4385，显示模型更偏向识别 PD 类；SVM-RBF 的 balanced accuracy 为 0.7118，综合表现较稳健。Dataset 2 按受试者 ID 分组交叉验证，SVM-RBF 达到 accuracy 0.8375、balanced accuracy 0.8375、AUC 0.8906、specificity 0.8750，作为该数据集的主推荐模型。

对问题二，本文在 Dataset 1 受试者级数据上整合 Mann-Whitney U 检验、Welch t-test、FDR-BH 校正、Cohen's *d*、Cliff's delta、L1 Logistic 稳定选择、ExtraTrees 重要性和 held-out permutation importance，得到候选语音生物标志物综合排序。Top20 特征主要包括 Delta / Delta-delta 动态谱波动特征与 TQWT 多尺度特征；Top50 中 TQWT 特征 31 个、Delta / Delta-delta 特征 16 个、Shimmer、MFCC 和 Nonlinear 特征各 1 个。上述结果仅表示判别性声学特征或候选语音生物标志物，不构成临床因果解释。

对问题四和问题五，由于数据集中没有真实运动型/非运动型标签，也没有静止性震颤、运动迟缓、肌肉僵直、疼痛、痴呆、睡眠障碍六类临床症状标签，本文仅在 Dataset 1 的 188 名 PD 受试者内部进行无监督潜在语音表型分析。二类分型主方案为 top20_biomarkers / pca_5 / KMeans，得到两簇人数 62 和 126，silhouette 为 0.3674，stability ARI mean 为 0.9854。六类分型主方案为 top20_biomarkers / pca_5 / AgglomerativeClustering，六簇人数为 13、55、34、28、25、33，silhouette 为 0.1823，stability ARI mean 为 0.4582。六类方案的聚类质量弱于二类方案，因此只作为探索性六类潜在语音表型，需要真实临床症状标签进一步验证。

本文在建模流程中保持受试者分组、训练折内标准化、主声学模型排除非声学变量和无监督标签不回流输入等约束，以降低样本单位错误和信息泄漏风险。综上，语音声学特征对帕金森病辅助筛查、判别性特征识别和探索性潜在语音表型分析具有一定价值，但本文模型不应解释为临床确诊工具，仍需更大样本和真实临床症状标签验证。

关键词：帕金森病；语音特征；受试者级交叉验证；候选语音生物标志物；潜在语音表型；无监督聚类

一 问题重述

帕金森病是一类常见神经退行性疾病，临床表现既包括静止性震颤、运动迟缓、肌强直等运动症状，也常伴随睡眠、认知、情绪和自主神经等非运动症状^[1]。除上述临床表现外，患者还可能出现发声稳定性下降、音强控制异常、音调波动和语音清晰度下降等语音障碍。已有研究表明，发声障碍和非侵入语音测试可用于帕金森病监测和辅助评估^[2, 3]。语音采集成本较低，适合用于辅助筛查和持续随访，因此本题要求基于给定声学数据建立帕金森病特征识别与声音诊断模型。

题目包含五个相互关联的问题。第一问要求利用 Dataset 1 建立健康人与帕金森病患者的二分类辅助诊断模型；第二问要求识别与帕金森病判别相关的关键语音特征；第三问要求利用 Dataset 2 建立声音特征智能诊断模型；第四问希望区分运动型与非运动型帕金森病；第五问希望进一步区分六类临床症状。由于附件数据没有提供第四问所需的真实运动型/非运动型标签，也没有第五问所需的六类临床症状标签，本文不构建真实临床症状监督诊断模型，而是在 PD 受试者内部进行无监督探索，得到基于语音特征的二类和六类潜在表型。

本文工作的边界是基于既有数据和已确定的计算结果形成完整论文表达，不引入新的实验、不改变已有模型输出。所有模型结论均限于语音声学数据的统计关联和预测表现，不能解释为临床因果结论或临床确诊结论。

二 问题分析与模型假设

两个数据集均包含同一受试者的多次录音。若直接把录音记录作为完全独立样本进行行级随机划分，同一受试者的不同录音可能同时进入训练集与测试集，从而高估模型性能。因此，监督建模必须以受试者为分组单位，Dataset 1 的主分析也应采用 subject-level 口径报告结果。

五个问题对应的方法路线如下。问题一使用 Dataset 1 的声学特征建立 PD/Healthy 二分类辅助诊断模型，并以受试者级交叉验证结果作为主结论。问题二基于 Dataset 1 受试者级数据，从统计差异、效应量、稳定选择和模型重要性等多个角度综合识别判别性声学特征。问题三单独使用 Dataset 2 建立诊断模型，并按 subject ID 分组交叉验证。问题四和问题五均缺少真实临床症状标签，因此仅在 Dataset 1 的 PD 受试者内部进行无监督聚类，分别形成二类和六类潜在语音表型。

为保证建模问题可计算并与数据结构一致，作如下假设：第一，同一受试者的多次录音是同一统计个体的重复观测，模型评估时不能跨训练集和测试集泄漏；第二，给定标签仅表示 Healthy/PD 二分类状态，不含运动型/非运动型或六类症状真标签；第三，声学特征的标准化参数、模型参数和特征选择参数只能由训练折估计；第四，聚类标签只表示声学空间中的潜在结构，不预设临床亚型含义。

设第 i 名受试者的声学特征向量为 $\mathbf{x}_i \in \mathbb{R}^p$ ，二分类标签为 $y_i \in \{0, 1\}$ ，其中 $y_i = 1$ 表示 PD。监督诊断模型学习映射

$$f : \mathbf{x}_i \mapsto \hat{p}_i = P(y_i = 1 \mid \mathbf{x}_i), \quad (1)$$

并由阈值得到类别预测 $\hat{y}_i$ 。主要评价指标定义为

$$\text{Sensitivity} = \frac{TP}{TP + FN}, \quad \text{Specificity} = \frac{TN}{TN + FP}, \quad (2)$$

$$\text{Balanced Accuracy} = \frac{\text{Sensitivity} + \text{Specificity}}{2}, \quad F_1 = \frac{2TP}{2TP + FP + FN}. \quad (3)$$

特征识别模型输出候选语音生物标志物排序。无监督分型模型在 PD 受试者集合上学习聚类映射 $g(\mathbf{x}_i) = c_i$ ，并使用 silhouette、Davies-Bouldin、簇大小比例和重采样稳定性 ARI 评价声学空间划分质量。由于聚类标签来自声学特征内部结构，后续聚类标签复现模型只能用于复现聚类规则，不能解释为真实临床症状诊断模型。

三 数据预处理与质量控制

3.1 数据结构审计

Dataset 1 原始数据规模为 756 行、755 列，包含 252 名受试者，每名受试者理论上有 3 次录音。数据包含 752 个纯声学特征，元数据列为 `id`、`gender` 和 `class`。该数据集来源于 Sakar 等构建的多类型录音帕金森语音数据集^[4]。审计发现 1 条完全重复记录，去重后建模准备文件为 755 行；受试者级聚合后为 252 行，其中 Healthy 64 名、PD 188 名。Dataset 1 主分析使用 subject-level 口径，避免把重复录音当作完全独立样本。

Dataset 2 原始数据规模为 240 行、48 列，包含 80 名受试者，每人 3 次录音，共 44 个声学特征。元数据列为 `ID`、`Recording`、`Status` 和 `Gender`。该数据集及其重复录音结构在 Naranjo 等关于录音重复对帕金森病检测影响的研究中已有说明^[5]；后续相关工作也讨论了重复录音条件下的变量选择与分类建模^[6]。受试者级类别分布为 Healthy 40 名、PD 40 名，类别均衡。

表 1 两个数据集的基本结构

数据集	原始规模	受试者数	录音结构	声学特征数	受试者级类别分布
Dataset 1	756 行、755 列	252	每人约 3 次录音	752	Healthy 64, PD 188
Dataset 2	240 行、48 列	80	每人 3 次录音	44	Healthy 40, PD 40

3.2 性别变量统一与主模型边界

两个数据集原始性别编码方向不同。为便于描述统计，预处理时统一构造 `sex_male`：Dataset 1 中 `sex_male = gender`，Dataset 2 中 `sex_male = 1 - Gender`。该变量仅用于描述统计和补充敏感性分析，主声学模型不使用 `sex_male`，从而避免把非声学人口学变量混入主要声学诊断结论。

3.3 交叉验证与标准化原则

监督诊断模型使用 `StratifiedGroupKFold`，并以受试者 ID 作为分组变量。标准化、模型训练和特征选择等涉及数据分布学习的步骤均在交叉验证训练折内部完成。Dataset 2 的准备文件只完成元数据清理和性别编码，声学特征没有在交叉验证前做全局 z-score 标准化。

四 模型一：帕金森病二分类辅助诊断模型

4.1 模型输入与方法

模型一输入为两个数据集的 acoustic-only 特征。Dataset 1 使用去重后的建模准备文件，并在报告主结果时接受试者聚合；Dataset 2 使用准备后的 240 条录音，并在交叉验证中按受试者 ID 分组。语音特征结合机器学习用于帕金森病识别已有较多研究基础，Little 等和 Tsanas 等分别从发声障碍测量与非侵入语音测试角度验证了语音特征在帕金森病远程监测中的价值^[2, 3]；Naranjo 等进一步讨论了重复录音条件下的分类问题^[5, 6]。

本文比较的模型包括 Logistic Regression、SVM-RBF 和 Random Forest。其中 SVM 适合处理非线性间隔分类问题^[7]，Random Forest 通过多棵决策树集成提高稳定性^[8]。模型实现采用 scikit-learn 机器学习库^[9]。对 Logistic Regression 与 SVM-RBF，标准化嵌入 sklearn Pipeline，并且只在训练折内拟合。

评价指标包括 accuracy、balanced accuracy、AUC、sensitivity、specificity 和 F1。其中 sensitivity 表示 PD 类召回率，specificity 表示 Healthy 类召回率。由于 Dataset 1 的 PD 受试者多于 Healthy 受试者，balanced accuracy 与 specificity 对判断模型是否过度偏向 PD 类尤为重要。

4.2 受试者级主结果

表 2 Dataset 1 subject-level acoustic-only 主结果

模型	Accuracy	Balanced Acc.	AUC	Sensitivity	Specificity	F1
Logistic Regression	0.7893	0.6989	0.8353	0.8825	0.5154	0.8616
Random Forest	0.8290	0.7004	0.8511	0.9623	0.4385	0.8936
SVM-RBF	0.8171	0.7118	0.8234	0.9249	0.4987	0.8825

Dataset 1 中 Random Forest 的 accuracy 和 AUC 较高，分别为 0.8290 和 0.8511，但 specificity 仅为 0.4385，说明该模型在类别不均衡背景下更倾向于判为 PD。SVM-RBF 的 balanced accuracy 为 0.7118，在敏感度和特异度之间相对均衡；Logistic Regression 具有较好的可解释性，但综合指标略低。

表 3 Dataset 2 subject-level acoustic-only 主结果

模型	Accuracy	Balanced Acc.	AUC	Sensitivity	Specificity	F1
Logistic Regression	0.7875	0.7875	0.8781	0.8250	0.7500	0.7905
Random Forest	0.7875	0.7875	0.8656	0.7250	0.8500	0.7748
SVM-RBF	0.8375	0.8375	0.8906	0.8000	0.8750	0.8300

Dataset 2 中 SVM-RBF 的 accuracy、balanced accuracy 和 AUC 均最高，specificity 也达到 0.8750，因此在该数据集上作为主推荐诊断模型。Random Forest 的 specificity 较高但 sensitivity 较低，提示其对 Healthy 类更保守；Logistic Regression 的 sensitivity 较高但整体 accuracy 低于 SVM-RBF。

五 模型二：关键语音生物标志物识别

5.1 输入、方法与输出

模型二使用 Dataset 1 subject-level 数据，共 252 名受试者，其中 Healthy 64 名、PD 188 名。输入特征为 752 个声学特征，排除 id、gender、class 和 sex_male。候选特征识别方法包括 Mann-Whitney U 检验、Welch t-test、FDR-BH 多重检验校正^[10]、Cohen's *d*、Cliff's delta、L1 Logistic 稳定选择、ExtraTrees 重要性和 held-out permutation importance。L1 Logistic 的稀疏选择思想与 LASSO 正则化一脉相承^[11]。最终排序综合单变量差异、效应量、稳定性和模型重要性，而不是依赖单一树模型重要性。

帕金森病语音研究中常用的 jitter、shimmer、MFCC、谐噪比和非线性复杂度等声学特征，已被用于发声障碍测量和疾病进展监测^[2, 3]。因此，本文将统计差异和模型解释结果统一表述为候选语音生物标志物或判别性声学特征，而不作临床因果外推。

5.2 Top20 候选语音生物标志物

表 4 Dataset 1 Top20 候选语音生物标志物

序号	特征	类别	Mean Rank
1	std_delta_delta_log_energy	Delta / Delta-delta	5.6667
2	std_6th_delta_delta	Delta / Delta-delta	15.5000
3	std_8th_delta	Delta / Delta-delta	20.6667
4	std_9th_delta_delta	Delta / Delta-delta	70.5833
5	std_7th_delta_delta	Delta / Delta-delta	79.4167
6	tqwt_TKE0_std_dec_12	TQWT	80.8333
7	tqwt_minValue_dec_12	TQWT	85.0000
8	std_9th_delta	Delta / Delta-delta	85.6667
9	mean_MFCC_2nd_coef	MFCC	86.1667
10	tqwt_TKE0_std_dec_11	TQWT	95.0833
11	std_10th_delta	Delta / Delta-delta	96.0833
12	tqwt_maxValue_dec_11	TQWT	97.4167
13	std_7th_delta	Delta / Delta-delta	97.6667
14	tqwt_kurtosisValue_dec_36	TQWT	101.5000
15	std_6th_delta	Delta / Delta-delta	101.9167
16	tqwt_kurtosisValue_dec_27	TQWT	103.7500
17	tqwt_energy_dec_12	TQWT	112.5833
18	DFA	Nonlinear	118.4167
19	tqwt_entropy_shannon_dec_17	TQWT	128.0833
20	tqwt_stdValue_dec_7	TQWT	131.6667

5.3 结果解释

Top20 中 Delta / Delta-delta 动态谱波动特征和 TQWT 多尺度特征占主要部分。Top50 类别分布为：TQWT 31 个，Delta / Delta-delta 16 个，Shimmer 1 个，MFCC 1 个，Nonlinear 1 个。该分布提示 PD/Healthy 的声学差异主要体现在动态谱波动和多尺度时频结构上。

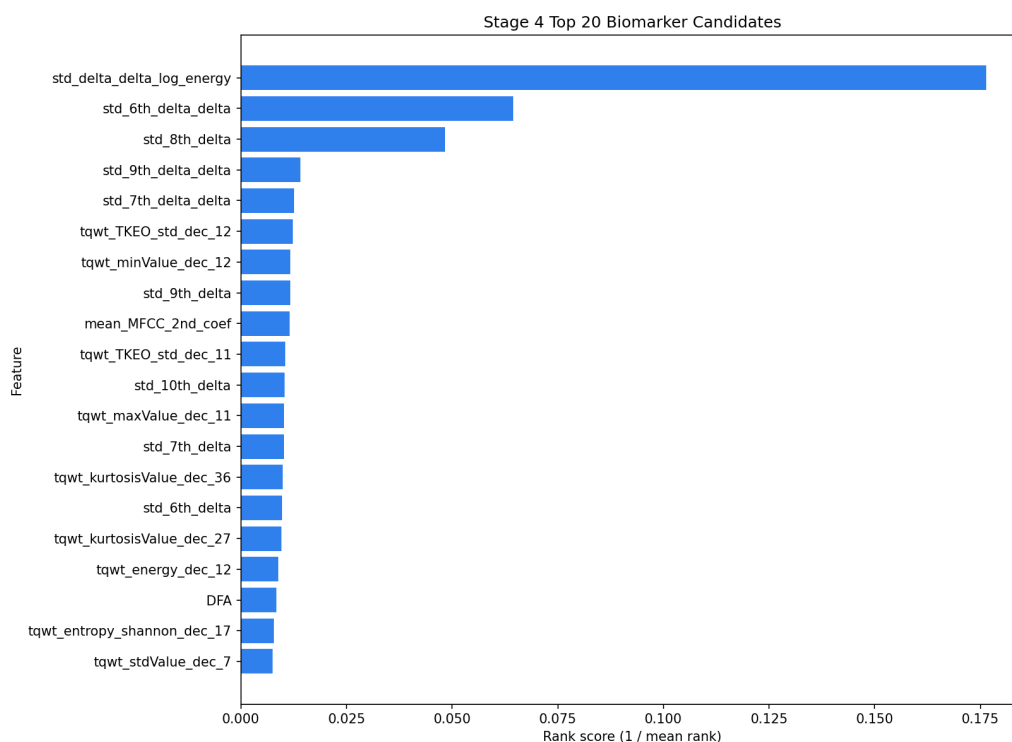


图 1 Top20 候选语音生物标志物综合排序

图中展示综合排名得分最高的 20 个候选声学特征。排名靠前的特征并非由单一模型重要性决定，而是综合了单变量统计差异、效应量、稳定选择、树模型重要性和置换重要性等证据。排名前列的特征集中于 Delta / Delta-delta 和 TQWT，说明 PD/Healthy 的主要判别信息与动态谱变化和多尺度时频结构有关。该图支持候选特征排序的稳定解释，但不能说明这些声学特征是疾病发生发展的临床原因。

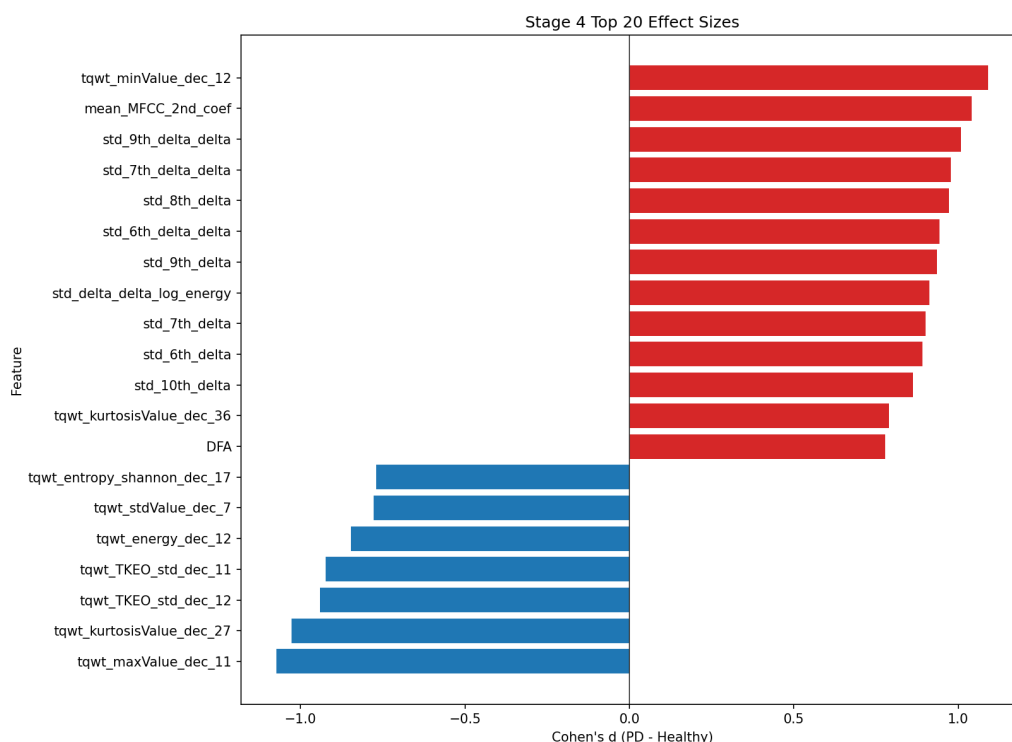


图 2 Top20 候选特征效应量

该图展示 Top20 特征的效应量方向和大小。正负方向表示 PD 组和 Healthy 组在该特征上的均值差异方向，绝对值越大表示组间差异越明显。效应量图与综合排序图共同说明，部分动态谱波动特征具有较明显的组间差异，可为二分类模型提供判别信息。需要强调的是，效应量只能说明统计关联和判别能力，不能推出该特征是帕金森病的临床致病机制。

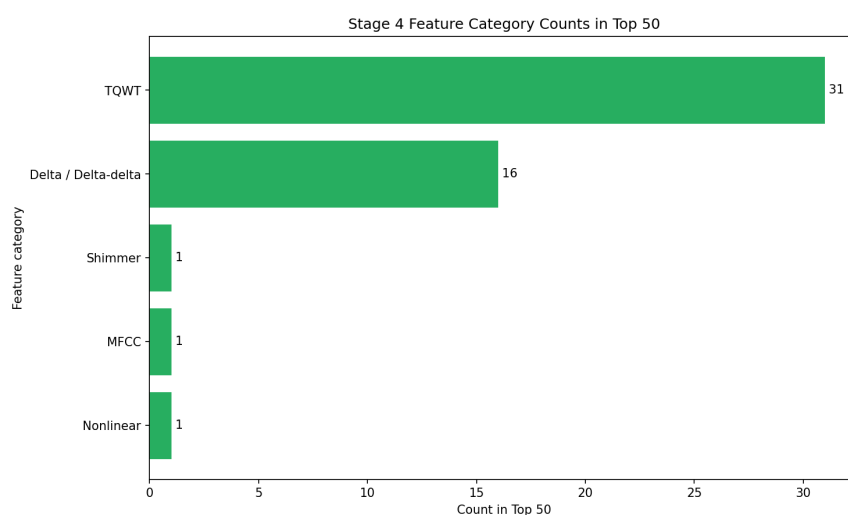


图 3 Top50 候选特征类别分布

该图显示 Top50 候选特征的类别构成，其中 TQWT 和 Delta / Delta-delta 占主导。这一结果与 Top20 排名一致，说明候选语音生物标志物不是零散分布在所有特征类别中，而是集中在多尺度声学波动和动态谱变化相关特征中。该集中性增强了模型解释的一致性，但仍不能替代独立数据集和临床机制验证。

上述特征只能称为候选语音生物标志物或判别性声学特征。由于本文没有临床机制实验和外部因果验证，不能作因果机制外推。

六 模型三：Dataset 2 声音特征智能诊断

Dataset 2 包含 80 名受试者、240 条录音，每名受试者 3 次录音，受试者级 Healthy 与 PD 各 40 名，类别均衡。该数据集的声学特征数为 44，与 Dataset 1 的高维 TQWT 特征空间不同，因此本文将其作为独立数据集单独建模和解释，而不与 Dataset 1 直接合并训练。

建模流程与模型一保持一致：输入为 acoustic-only 特征，模型包括 Logistic Regression、SVM-RBF 和 Random Forest，交叉验证采用 StratifiedGroupKFold 并按 ID 分组。标准化嵌入 Pipeline，仅在训练折内拟合。输出为受试者级 PD/Healthy 辅助诊断结果。

从表 2 可见，SVM-RBF 在 Dataset 2 上取得 accuracy 0.8375、balanced accuracy 0.8375、AUC 0.8906、sensitivity 0.8000、specificity 0.8750 和 F1 0.8300，综合表现最好。Random Forest 的 specificity 较高但 sensitivity 较低，提示其对 Healthy 类更保守；Logistic Regression 的 sensitivity 较高但整体 accuracy 低于 SVM-RBF。

Dataset 2 样本量较小，但类别均衡、每名受试者录音次数一致，适合作为独立验证风格的参考分析。由于两个数据集特征空间不完全一致，本文不将 Dataset 2 结果解释为 Dataset 1 模型的严格外部验证，而作为同一研究问题下的独立声学诊断建模证据。

七 模型四：二类潜在语音表型辅助分型

7.1 建模边界

题目第四问涉及运动型和非运动型帕金森病，但给定数据没有真实运动型/非运动型临床标签。因此，本文不训练真实临床亚型监督分类器，而是在 Dataset 1 的 188 名 PD 受试者内部进行 $k=2$ 无监督聚类，得到基于语音特征的二类潜在表型。该结果可作为探索性辅助分型线索，不能等同于真实临床亚型诊断。

7.2 输入、方法与结果

候选输入特征集包括前述综合排序得到的 Top20、Top50 和 all acoustic 特征。主方案选择 top20_biomarkers / pca_5 / KMeans。所有候选方案先执行标准化，再按方案进行 PCA 和聚类。聚类输入不包含 id、gender、class、sex_male、record_count 或任何标签列。聚类质量通过 silhouette、Calinski-Harabasz、Davies-Bouldin 和重采样 ARI 等指标综合评价^[12, 13, 14, 15]。

表 5 二类潜在语音表型主方案指标

项目	数值
主方案	top20_biomarkers / pca_5 / KMeans
PD 受试者数	188
Cluster 0 人数	62
Cluster 1 人数	126
Smaller cluster ratio	0.3298
Silhouette	0.3674
Stability ARI mean	0.9854

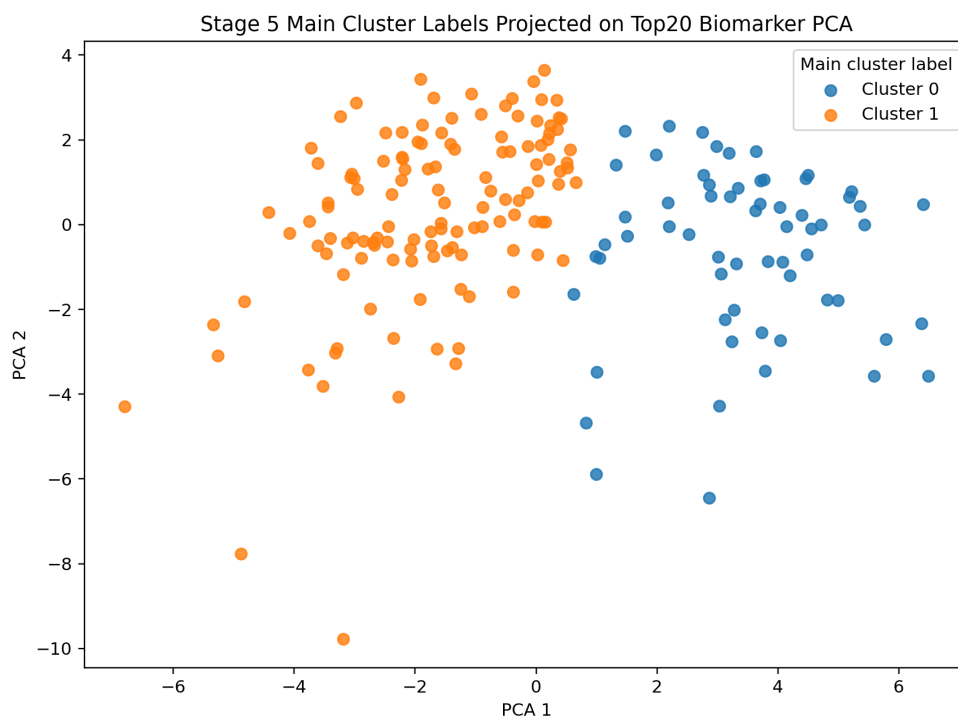


图 4 Dataset 1 PD 受试者二类潜在语音表型 PCA 可视化

该图将主推荐方案得到的二类聚类标签投影到 Top20 特征的前两个 PCA 方向上。图中两类样本在二维空间中呈现较明显的左右分离趋势，说明 Top20 候选特征能够支持 PD 受试者内部二类潜在语音表型划分。但 PCA 二维图只是可视化投影，不能替代 silhouette、簇规模和稳定性 ARI 等定量指标，也不能直接说明真实临床亚型存在。

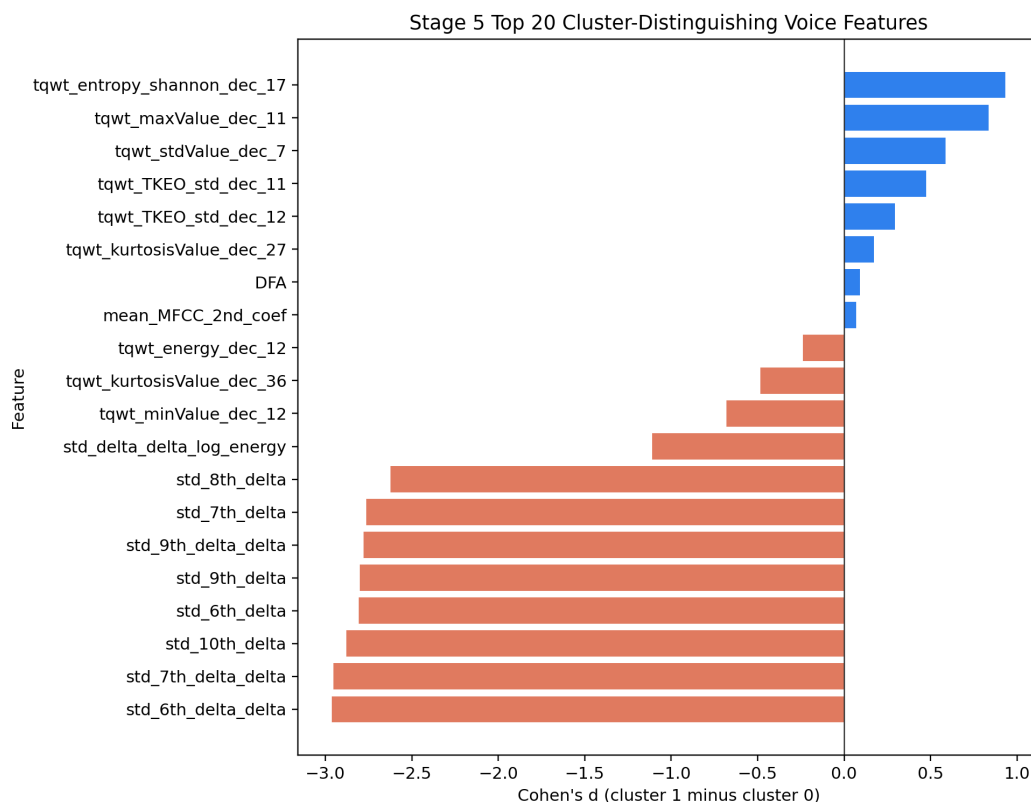


图 5 二类潜在语音表型的 Top20 特征差异

该图以 Cohen's d 展示二类潜在表型在 Top20 特征上的差异方向。横轴为 cluster 1 minus cluster 0，因此负值表示 cluster 0 更高，正值表示 cluster 1 更高。图中 cluster 0 在多项 Delta / Delta-delta 动态谱波动特征上更高，cluster 1 在若干 TQWT 特征上更高，提示两簇主要反映动态谱波动和多尺度声学结构的差异。该差异只能解释为语音表型差异，不能直接解释为真实运动型/非运动型临床差异。

7.3 表型解释

主方案得到 cluster 0 共 62 人、cluster 1 共 126 人。cluster 0 在多项 Delta / Delta-delta 动态谱波动特征上更高，例如 `std_6th_delta_delta`、`std_7th_delta_delta`、`std_10th_delta`、`std_6th_delta` 等。cluster 1 在部分 TQWT 多尺度特征上更高，例如 `tqwt_entropy_shannon_dec_17`、`tqwt_maxValue_dec_11`、`tqwt_stdValue_dec_7`、`tqwt_TKEO_std_dec_11` 等。

后续聚类标签复现模型学习的是上述无监督聚类标签，用于将探索性分型规则模型化。该分类器不是根据真实运动型/非运动型标签训练，不能解释为真实临床亚型诊断性能。

八 模型五：六类潜在语音表型辅助分型

8.1 建模边界

题目第五问涉及六类帕金森病症状，但数据集中没有静止性震颤、运动迟缓、肌肉僵直、疼痛、痴呆、睡眠障碍六类真实临床症状标签。因此，本文只在 Dataset 1 的 188 名 PD 受试者内部固定 $k = 6$ 进行无监督聚类，输出基于语音特征的六类潜在表型。六个 cluster 编号没有直接临床症状含义，仍需真实临床症状标签进一步验证。

8.2 输入、方法与结果

六类表型建模对 Top20、Top50 和 all acoustic 特征集进行候选方案比较，并做 $k = 2$ 到 $k = 8$ 的参考扫描。主方案为 top20_biomarkers / pca_5 / AgglomerativeClustering。所有表示均先进行标准化，PCA 和聚类仅使用声学特征；二类聚类标签没有进入六类聚类输入。聚类评价同样结合 silhouette、Calinski-Harabasz、Davies-Bouldin 和 ARI 等指标^[12, 13, 14, 15]。

表 6 六类潜在语音表型主方案指标

项目	数值
主方案	top20_biomarkers / pca_5 / AgglomerativeClustering
PD 受试者数	188
六簇人数	13、55、34、28、25、33
最小簇比例	0.0691
Silhouette	0.1823
Stability ARI mean	0.4582
二类与六类表型 NMI	0.4089
Collapsed majority ARI	0.8128

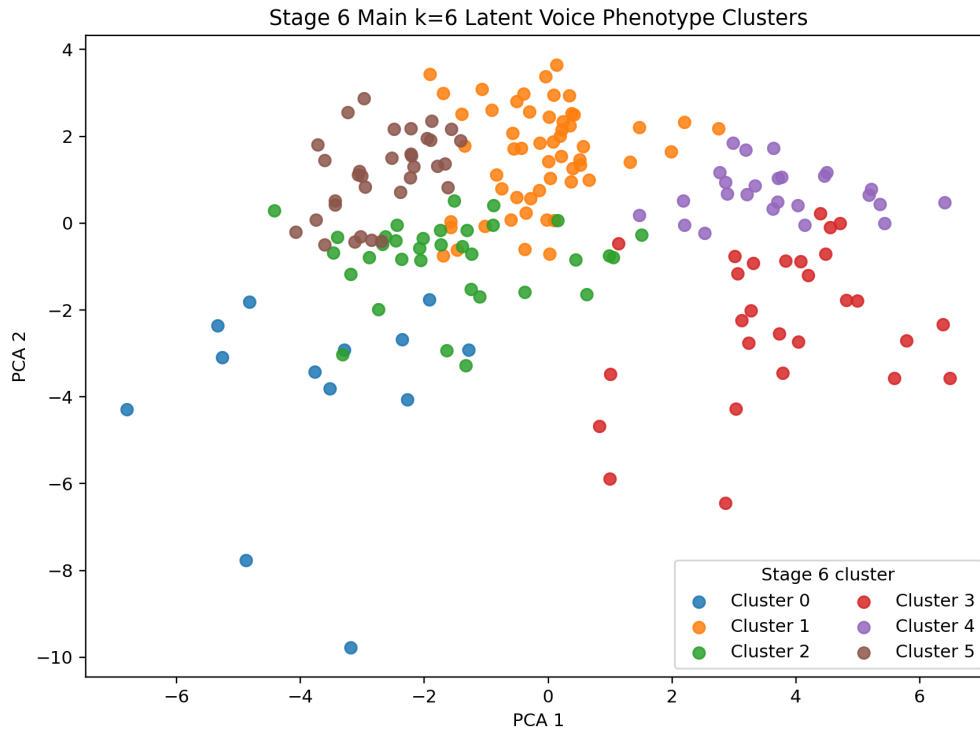


图 6 Dataset 1 PD 受试者六类潜在语音表型 PCA 可视化

该图将六类潜在语音表型投影到 PCA 二维平面。部分类别在平面上具有较清晰的区域分布，但也存在一定重叠，说明 $k = 6$ 划分比 $k = 2$ 更细，同时稳定性和分离度更弱。因此该图支持“探索性六类语音表型”的表述，而不能支持“六类临床症状完全可分”的结论。

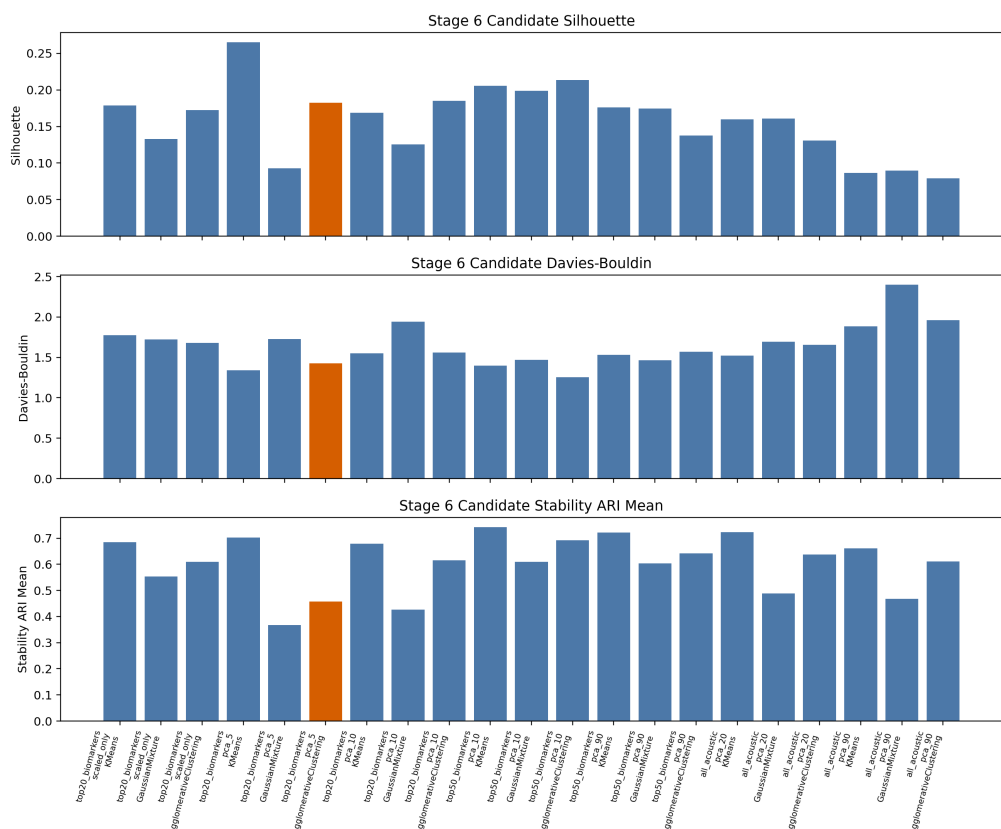


图 7 六类潜在语音表型候选方案指标比较

该候选指标图用于说明主推荐方案不是单一指标最优的机械选择，而是在簇规模约束、稳定性、解释性和内部聚类指标之间折中选择。部分候选方案可能在单个指标上表现较好，但若出现过小簇、稳定性不足或依赖高维噪声空间，则不宜作为主解释方案。该图支持多指标综合推荐思路，不能说明六类表型具有外部临床标签验证。

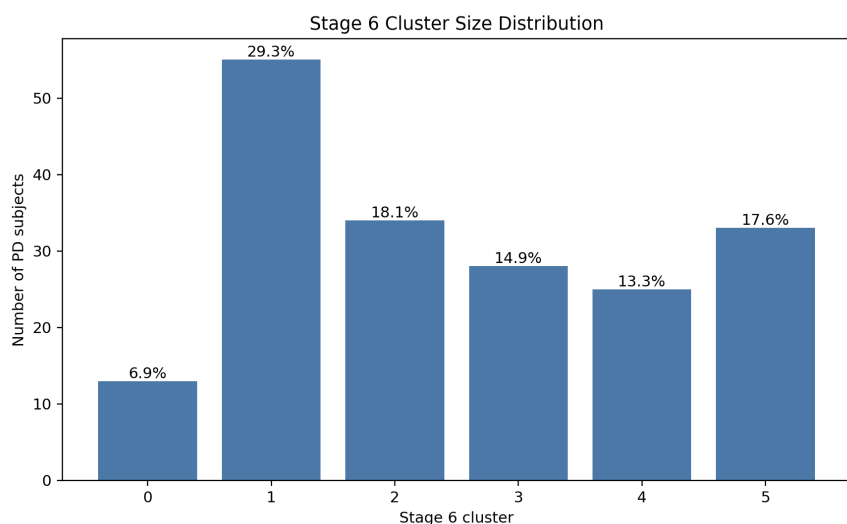


图 8 六类潜在语音表型簇规模分布

该图展示六个簇的人数分布。主方案最小簇为 13 人，占 6.9%，没有出现 1 人或 2 人小簇，说明该方案在固定 $k = 6$ 的前提下避免了极端碎片化划分。簇规模分布只能说明无监督划分在样本数量上的可解释性，不能说明各簇对应真实临床症状类别。

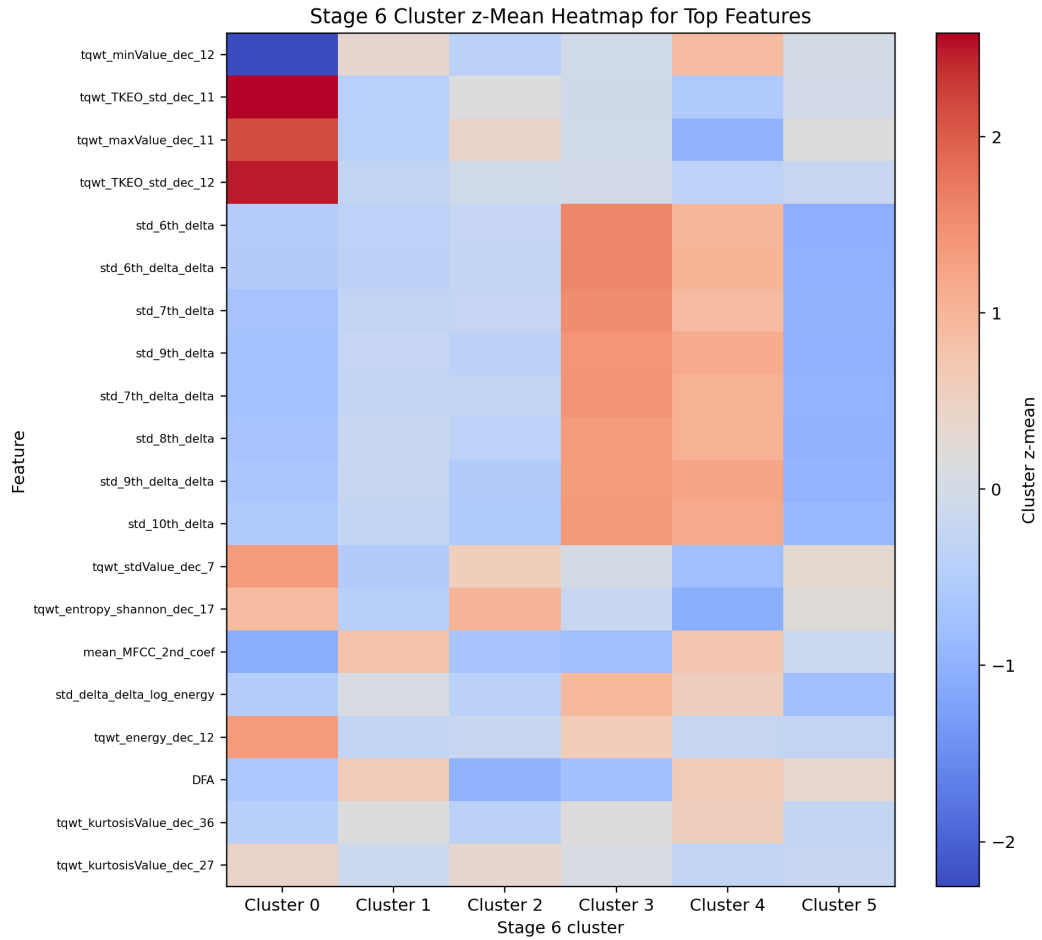


图 9 六类潜在语音表型特征热图

该热图展示六个簇在主要区分特征上的标准化均值差异，颜色深浅反映各簇在对应特征上的相对高低。Cluster 0 在部分 TQWT 特征上明显较高，Cluster 3 和 Cluster 4 在多项 Delta / Delta-delta 特征上较高，Cluster 5 在动态谱波动特征上相对较低。该图提供了六类潜在语音表型的声学画像基础，但不能说明这些画像对应具体临床症状。

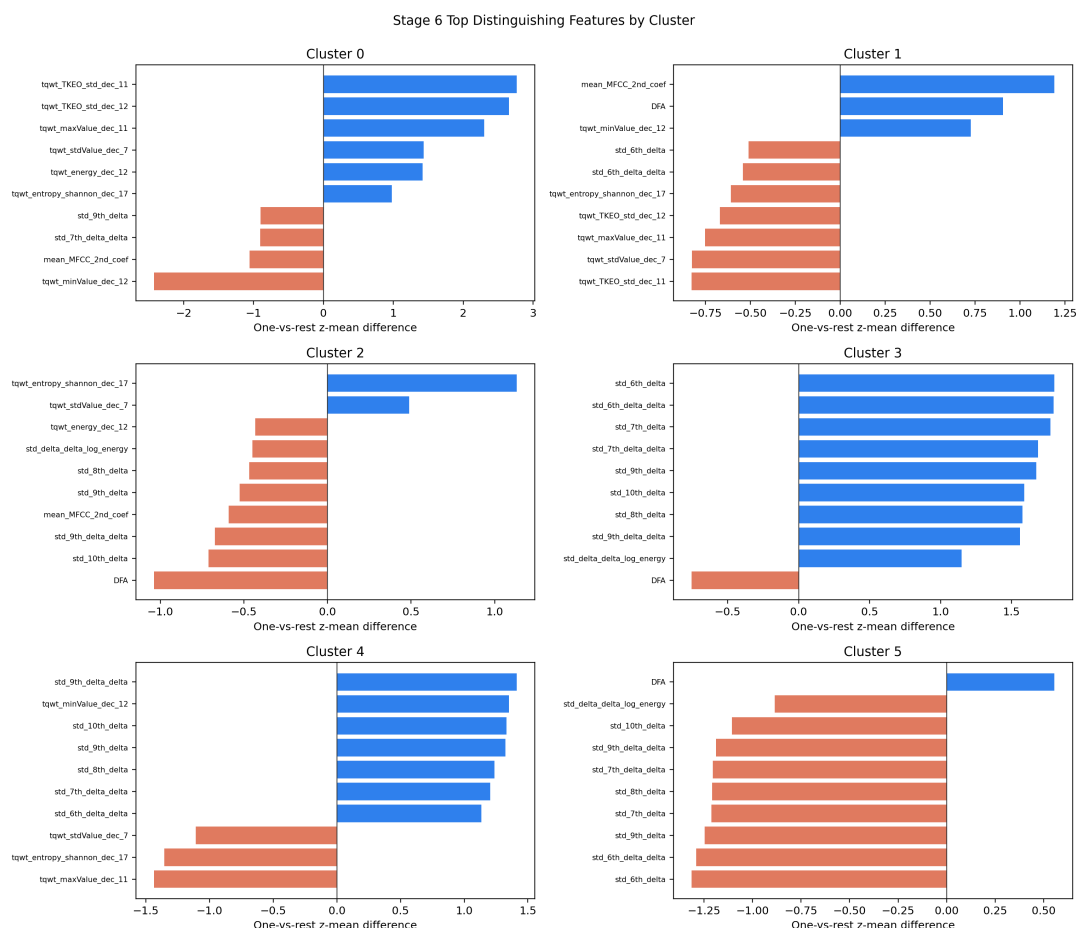


图 10 六类潜在语音表型各簇 Top 区分特征

该图进一步给出不同簇的主要区分特征，有助于观察各类潜在语音表型的具体声学画像。不同簇的 Top 特征集中在 TQWT 与 Delta / Delta-delta 相关特征，说明六类划分主要反映多尺度时频结构和动态谱波动模式的差异。该图只能解释声学空间内的聚类画像，不能作为临床症状命名依据。

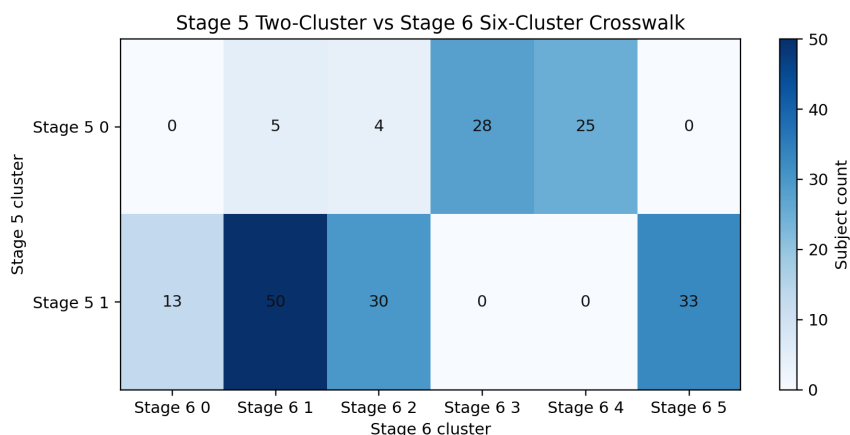


图 11 二类潜在表型与六类潜在表型交叉关系

该图展示二类表型与六类表型之间的交叉关系。图中六类表型的部分簇主要落在二类表型 cluster 0，另一些簇主要落在二类表型 cluster 1，说明六类潜在表型在一定程

度上可以看作二类潜在表型的进一步细分。结合 NMI 0.4089 和 collapsed majority ARI 0.8128, 可说明两级语音表型之间存在中等程度对应关系。但这只是声学聚类标签之间的关系, 不能解释为运动型/非运动型到六类临床症状的真实层级关系。

8.3 结果解释

六类潜在语音表型主要由 Delta / Delta-delta 与 TQWT 特征区分。部分簇表现为动态谱波动特征较高, 部分簇则表现为 TQWT 多尺度能量、熵或统计量差异。与二类方案相比, 六类方案的 silhouette 下降到 0.1823, stability ARI mean 下降到 0.4582, 说明固定 $k = 6$ 后聚类结构更细但稳定性和分离度较弱。

六类潜在表型与二类潜在表型存在一定嵌套关系, NMI 为 0.4089, 按六簇多数映射回二簇后的 collapsed majority ARI 为 0.8128。该交叉关系只能说明六类潜在表型与二类潜在表型在声学空间中存在一定对应, 不能解释为真实临床分层诊断。

六类聚类标签复现模型预测的是六类聚类标签, 不是真实静止性震颤、运动迟缓、肌肉僵直、疼痛、痴呆或睡眠障碍标签。因此, 其指标只表示复现无监督聚类划分的能力, 不表示六类临床症状诊断能力。

九 模型优缺点分析

9.1 优点

第一，本文严格识别了两个数据集的重复录音结构，监督学习中按受试者分组交叉验证，并以 subject-level 指标作为主结果，降低了样本单位错误带来的性能高估风险。第二，主声学模型排除了 `sex_male`，避免将人口学变量混入声学诊断结论。第三，候选语音生物标志物识别同时结合统计检验、效应量、稳定选择和模型重要性，使特征解释不依赖单一方法。第四，对缺少真实标签的第四问和第五问，本文采用无监督潜在表型分析，并明确其探索性边界。第五，全流程一致性检查对分组、泄漏和指标一致性进行了系统控制。

9.2 局限性

本文也存在明显局限。Dataset 1 受试者数有限且类别不均衡，导致部分模型特异度偏低；Dataset 2 虽类别均衡，但样本量较小。两个数据集的特征空间不完全一致，不能简单合并或直接外推。第四问和第五问缺少真实临床症状标签，因此聚类结果只能称为潜在语音表型，不能直接对应运动型/非运动型或六类临床症状。聚类表型仍需要真实临床标签、纵向随访和外部样本验证。

十 结论

针对问题一，Dataset 1 的 PD/Healthy 辅助诊断模型表明，语音声学特征具有一定判别能力。Random Forest 的 accuracy 和 AUC 较高，但 specificity 较低，提示其偏向 PD 类；SVM-RBF 在 balanced accuracy 上表现相对稳健。

针对问题二，综合多种统计和模型解释证据，本文得到 Top20 候选语音生物标志物。其中特征主要集中于 Delta / Delta-delta 动态谱波动和 TQWT 多尺度声学结构，说明这些声学维度对 PD/Healthy 判别具有较强关联性，但不能解释为临床因果结论。

针对问题三，Dataset 2 按受试者 ID 分组交叉验证后，SVM-RBF 取得 accuracy 0.8375、balanced accuracy 0.8375 和 AUC 0.8906，是该数据集上的主推荐模型。该结论支持语音特征在独立数据集中的辅助诊断价值，但仍受样本量限制。

针对问题四，本文在 188 名 PD 受试者内部得到二类潜在语音表型。主方案 silhouette 为 0.3674，stability ARI mean 为 0.9854，显示较稳定的二类声学结构。该结构主要由动态谱波动特征和部分 TQWT 特征区分，但不是真实运动型/非运动型诊断。

针对问题五，本文固定 $k = 6$ 得到六类潜在语音表型。六簇主要由 Delta / Delta-delta 与 TQWT 特征区分，但其 silhouette 和稳定性弱于二类方案，因此只能作为探索性六类语音表型。后续若获得真实六类临床症状标签，应重新建立受试者级监督验证框架。

总体而言，语音声学特征对帕金森病辅助筛查、判别性特征识别和探索性分型具有应用潜力。本文结论应定位为辅助建模证据和探索性语音表型分析，而非临床确诊模型。

参考文献

- [1] Greenland J C, Barker R A. The differential diagnosis of Parkinson's disease[M]// Stoker T B, Greenland J C, editors. Parkinson's Disease: Pathogenesis and Clinical Aspects. Brisbane: Codon Publications, 2018.
- [2] Little M A, McSharry P E, Hunter E J, Spielman J, Ramig L O. Suitability of dysphonia measurements for telemonitoring of Parkinson's disease[J]. IEEE Transactions on Biomedical Engineering, 2009, 56(4): 1015–1022.
- [3] Tsanas A, Little M A, McSharry P E, Ramig L O. Accurate telemonitoring of Parkinson's disease progression by noninvasive speech tests[J]. IEEE Transactions on Biomedical Engineering, 2010, 57(4): 884–893.
- [4] Sakar B E, Isenkul M E, Sakar C O, Sertbas A, Gorgen F, Delil S, Apaydin H, Kursun O. Collection and analysis of a Parkinson speech dataset with multiple types of sound recordings[J]. IEEE Journal of Biomedical and Health Informatics, 2013, 17(4): 828–834.
- [5] Naranjo L, Perez C J, Campos-Roca Y, Martin J. Addressing voice recording replications for Parkinson's disease detection[J]. Expert Systems with Applications, 2016, 46: 286–292.
- [6] Naranjo L, Perez C J, Martin J, Campos-Roca Y. A two-stage variable selection and classification approach for Parkinson's disease detection by using voice recording replications[J]. Computer Methods and Programs in Biomedicine, 2017, 142: 147–156.
- [7] Cortes C, Vapnik V. Support-vector networks[J]. Machine Learning, 1995, 20(3): 273–297.
- [8] Breiman L. Random forests[J]. Machine Learning, 2001, 45(1): 5–32.
- [9] Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, et al. Scikit-learn: Machine learning in Python[J]. Journal of Machine Learning Research, 2011, 12: 2825–2830.
- [10] Benjamini Y, Hochberg Y. Controlling the false discovery rate: a practical and powerful approach to multiple testing[J]. Journal of the Royal Statistical Society: Series B, 1995, 57(1): 289–300.
- [11] Tibshirani R. Regression shrinkage and selection via the lasso[J]. Journal of the Royal Statistical Society: Series B, 1996, 58(1): 267–288.
- [12] Rousseeuw P J. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis[J]. Journal of Computational and Applied Mathematics, 1987, 20: 53–65.
- [13] Calinski T, Harabasz J. A dendrite method for cluster analysis[J]. Communications in Statistics, 1974, 3(1): 1–27.
- [14] Davies D L, Bouldin D W. A cluster separation measure[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1979, PAMI-1(2): 224–227.

- [15] Hubert L, Arabie P. Comparing partitions[J]. Journal of Classification, 1985, 2: 193–218.

A 附录：程序代码

本附录给出建模、特征识别、潜在表型分析与全流程审计所用的完整代码。为避免正文被程序实现细节打断，代码统一置于附录。

A.1 代码一：数据预处理与质量控制

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import matplotlib
6
7 matplotlib.use("Agg")
8
9 import matplotlib.pyplot as plt
10 import pandas as pd
11
12
13 ROOT = Path(__file__).resolve().parents[1]
14 DATA_DIR = ROOT / "data"
15 RESULTS_DIR = ROOT / "results"
16 FIGURES_DIR = RESULTS_DIR / "figures"
17
18 CSV_ENCODING = "utf-8"
19 OUTPUT_CSV_ENCODING = "utf-8-sig"
20
21 DATASET_CONFIG = {
22     "dataset1": {
23         "display_name": "Dataset 1",
24         "file_name": "pd_speech_features.csv",
25         "read_kwargs": {"header": 1, "encoding": CSV_ENCODING},
26         "read_method": 'pd.read_csv(path, header=1, encoding="utf-8")',
27         "id_col": "id",
28         "label_col": "class",
29         "gender_col": "gender",
30         "recording_col": None,
31         "metadata_cols": ["id", "gender", "class"],
32         "sex_male_rule": "sex_male = gender",
33     },
34     "dataset2": {
35         "display_name": "Dataset 2",
36         "file_name": "ReplicatedAcousticFeatures-ParkinsonDatabase.csv",
37         "read_kwargs": {"encoding": CSV_ENCODING},
38         "read_method": 'pd.read_csv(path, encoding="utf-8")',
39         "id_col": "ID",
40         "label_col": "Status",
41         "gender_col": "Gender",
42         "recording_col": "Recording",
43         "metadata_cols": ["ID", "Recording", "Status", "Gender"],
44         "sex_male_rule": "sex_male = 1 - Gender",
45     },
46 }
47
48 STAT_COLUMNS = ["mean", "std", "median", "min", "max", "IQR"]
49
50
51 def rel(path: Path) -> str:
52     return path.relative_to(ROOT).as_posix()
53
54
55 def find_source_csvs() -> dict[str, Path]:
56     paths = {path.name: path for path in DATA_DIR.rglob("*.csv")}
57     missing = [
58         config["file_name"]
59         for config in DATASET_CONFIG.values()
60         if config["file_name"] not in paths
61     ]
62     if missing:
63         raise FileNotFoundError(f"Missing source CSV files: {missing}")
64     return paths
65
66
67 def add_sex_male(df: pd.DataFrame, dataset_key: str) -> pd.DataFrame:
68     cfg = DATASET_CONFIG[dataset_key]
69     out = df.copy()
70     if dataset_key == "dataset1":
71         out["sex_male"] = out[cfg["gender_col"]]
72     elif dataset_key == "dataset2":
73         out["sex_male"] = 1 - out[cfg["gender_col"]]
74     else:
75         raise ValueError(f"Unknown dataset: {dataset_key}")
76     return out
77
78
79 def feature_columns(df: pd.DataFrame, dataset_key: str) -> list[str]:
80     cfg = DATASET_CONFIG[dataset_key]
81     excluded = set(cfg["metadata_cols"]) | {"sex_male"}
82     return [
83         column
```

```

84         for column in df.select_dtypes(include="number").columns
85         if column not in excluded
86     ]
87
88
89 def describe_features(
90     df: pd.DataFrame,
91     features: list[str],
92     dataset_key: str,
93     scope: str,
94     label_value: object = "",
95 ) -> pd.DataFrame:
96     numeric = df[features]
97     q1 = numeric.quantile(0.25)
98     q3 = numeric.quantile(0.75)
99     stats = pd.DataFrame(
100         {
101             "dataset": dataset_key,
102             "scope": scope,
103             "label_value": label_value,
104             "feature": features,
105             "n": numeric.count().reindex(features).astype(int).values,
106             "mean": numeric.mean().reindex(features).values,
107             "std": numeric.std(ddof=1).reindex(features).values,
108             "median": numeric.median().reindex(features).values,
109             "min": numeric.min().reindex(features).values,
110             "max": numeric.max().reindex(features).values,
111             "IQR": (q3 - q1).reindex(features).values,
112         }
113     )
114     return stats[["dataset", "scope", "label_value", "feature", "n"] + STAT_COLUMNS]
115
116
117 def record_level_stats(
118     df: pd.DataFrame, dataset_key: str, features: list[str]
119 ) -> pd.DataFrame:
120     cfg = DATASET_CONFIG[dataset_key]
121     frames = [describe_features(df, features, dataset_key, "record_overall")]
122     for label_value, group in df.groupby(cfg["label_col"], dropna=False):
123         frames.append(
124             describe_features(
125                 group, features, dataset_key, "record_by_label", label_value
126             )
127         )
128     return pd.concat(frames, ignore_index=True)
129
130
131 def subject_table(
132     df: pd.DataFrame, dataset_key: str, features: list[str]
133 ) -> pd.DataFrame:
134     cfg = DATASET_CONFIG[dataset_key]
135     id_col = cfg["id_col"]
136     label_col = cfg["label_col"]
137     gender_col = cfg["gender_col"]
138     feature_means = df.groupby(id_col, dropna=False)[features].mean().reset_index()
139     subject_meta = (
140         df.groupby(id_col, dropna=False)
141         .agg(
142             record_count=(id_col, "size"),
143             label_unique_count=(label_col, lambda s: s.nunique(dropna=False)),
144             gender_unique_count=(gender_col, lambda s: s.nunique(dropna=False)),
145             sex_male_unique_count=("sex_male", lambda s: s.nunique(dropna=False)),
146             label_value=(label_col, "first"),
147             raw_gender_value=(gender_col, "first"),
148             sex_male=("sex_male", "first"),
149         )
150         .reset_index()
151     )
152     return subject_meta.merge(feature_means, on=id_col, how="left")
153
154
155 def subject_level_stats(
156     subjects: pd.DataFrame, dataset_key: str, features: list[str]
157 ) -> pd.DataFrame:
158     frames = [describe_features(subjects, features, dataset_key, "subject_overall")]
159     for label_value, group in subjects.groupby("label_value", dropna=False):
160         frames.append(
161             describe_features(
162                 group, features, dataset_key, "subject_by_label", label_value
163             )
164         )
165     return pd.concat(frames, ignore_index=True)
166
167
168 def save_crosstab(
169     df: pd.DataFrame,
170     label_col: str,
171     output_path: Path,
172     index_name: str,
173 ) -> pd.DataFrame:
174     table = pd.crosstab(df[label_col], df["sex_male"], margins=True)
175     table.index.name = index_name

```

```

176     table.columns.name = "sex_male"
177     table.to_csv(output_path, encoding=OUTPUT_CSV_ENCODING)
178     return table
179
180
181 def save_distribution_figure(
182     record_counts: pd.Series,
183     subject_counts: pd.Series,
184     title: str,
185     x_label: str,
186     output_path: Path,
187 ) -> None:
188     fig, axes = plt.subplots(1, 2, figsize=(12, 6), dpi=120)
189     panels = [
190         ("Record level", record_counts, "#2f80ed"),
191         ("Subject level", subject_counts, "#27ae60"),
192     ]
193     for ax, (panel_title, counts, color) in zip(axes, panels):
194         counts = counts.sort_index()
195         labels = [str(label) for label in counts.index]
196         ax.bar(labels, counts.values, color=color)
197         ax.set_title(panel_title)
198         ax.set_xlabel(x_label)
199         ax.set_ylabel("Count")
200         ax.spines["top"].set_visible(False)
201         ax.spines["right"].set_visible(False)
202         for index, value in enumerate(counts.values):
203             ax.text(index, value, str(int(value)), ha="center", va="bottom")
204     fig.suptitle(title, fontsize=16, fontweight="bold")
205     fig.tight_layout(rect=(0, 0, 1, 0.94))
206     fig.savefig(output_path)
207     plt.close(fig)
208
209
210 def write_csv(df: pd.DataFrame, path: Path) -> None:
211     df.to_csv(path, encoding=OUTPUT_CSV_ENCODING, index=False)
212
213
214 def main() -> None:
215     RESULTS_DIR.mkdir(exist_ok=True)
216     FIGURES_DIR.mkdir(exist_ok=True)
217     source_paths = find_source_csvs()
218
219     loaded: dict[str, pd.DataFrame] = {}
220     subjects_by_dataset: dict[str, pd.DataFrame] = {}
221     feature_counts: dict[str, int] = {}
222     duplicate_counts: dict[str, int] = {}
223     label_conflicts: dict[str, int] = {}
224     gender_conflicts: dict[str, int] = {}
225     sex_conflicts: dict[str, int] = {}
226     crosstab_outputs: list[Path] = []
227     report_lines: list[str] = [
228         "# 第二阶段描述性统计与质量控制报告",
229         "",
230         " 本报告由 `src/stage2_qc_pipeline.py` 生成，所有文本使用 UTF-8 写入，CSV 使用 utf-8-sig 写入以兼容 Excel。",
231         "",
232         "## 统一数据配置",
233         "",
234         "``python",
235         "DATASET_CONFIG = {",
236         "    'dataset1': {'id_col': 'id', 'label_col': 'class', 'gender_col': 'gender', 'metadata_cols': ['id', 'gender',
237         ↪ 'class']},",
238         "    'dataset2': {'id_col': 'ID', 'label_col': 'Status', 'gender_col': 'Gender', 'recording_col': 'Recording',
239         ↪ 'metadata_cols': ['ID', 'Recording', 'Status', 'Gender']},",
240         "}",
241         "``",
242         "",
243         "## 性别编码统一",
244         "",
245         " 两个数据集原始性别编码方向不同。",
246         " 为避免解释混乱，本阶段新增统一变量 `sex_male`，不覆盖原始 `gender/Gender`。",
247         " `sex_male=1` 表示男性，`sex_male=0` 表示女性。",
248         " Dataset 1: 保留原始列 `gender`，新增 `sex_male = gender`。",
249         " Dataset 2: 保留原始列 `Gender`，新增 `sex_male = 1 - Gender`。",
250         " 后续跨数据集分析和交叉表均使用 `sex_male`；原始 `gender/Gender` 只用于数据审计和溯源，不进入统一建模特征。",
251         " 主模型应只使用语音特征；`sex_male` 不直接进入主模型，仅用于补充模型或敏感性分析，以检验性别混杂影响。",
252         "",
253     ]
254
255     for dataset_key, cfg in DATASET_CONFIG.items():
256         path = source_paths[cfg["file_name"]]
257         df_raw = pd.read_csv(path, **cfg["read_kwargs"])
258         df = add_sex_male(df_raw, dataset_key)
259         loaded[dataset_key] = df
260         features = feature_columns(df, dataset_key)
261         feature_counts[dataset_key] = len(features)
262         duplicate_counts[dataset_key] = int(df_raw.duplicated().sum())
263
264         record_stats = record_level_stats(df, dataset_key, features)
265         subjects = subject_table(df, dataset_key, features)
266         subjects_by_dataset[dataset_key] = subjects
267         subject_stats = subject_level_stats(subjects, dataset_key, features)

```

```

266
267 write_csv(record_stats, RESULTS_DIR / f"descriptive_statistics_{dataset_key}.csv")
268 write_csv(
269     subject_stats,
270     RESULTS_DIR / f"grouped_subject_statistics_{dataset_key}.csv",
271 )
272
273 for obsolete in RESULTS_DIR.glob(f"{dataset_key}_class_gender_crosstab_*.csv"):
274     obsolete.unlink()
275 record_crosstab_path = (
276     RESULTS_DIR / f"{dataset_key}_class_sex_male_crosstab_record.csv"
277 )
278 subject_crosstab_path = (
279     RESULTS_DIR / f"{dataset_key}_class_sex_male_crosstab_subject.csv"
280 )
281 save_crosstab(
282     df,
283     cfg["label_col"],
284     record_crosstab_path,
285     cfg["label_col"],
286 )
287 save_crosstab(
288     subjects.rename(columns={"label_value": cfg["label_col"]}),
289     cfg["label_col"],
290     subject_crosstab_path,
291     cfg["label_col"],
292 )
293 crosstab_outputs.extend([record_crosstab_path, subject_crosstab_path])
294
295 label_conflicts[dataset_key] = int(
296     (subjects["label_unique_count"] > 1).sum()
297 )
298 gender_conflicts[dataset_key] = int(
299     (subjects["gender_unique_count"] > 1).sum()
300 )
301 sex_conflicts[dataset_key] = int(
302     (subjects["sex_male_unique_count"] > 1).sum()
303 )
304
305 save_distribution_figure(
306     df[cfg["label_col"]].value_counts(),
307     subjects["label_value"].value_counts(),
308     f"{cfg['display_name']} class distribution",
309     cfg["label_col"],
310     FIGURES_DIR / f"class_distribution_{dataset_key}.png",
311 )
312 save_distribution_figure(
313     df["sex_male"].value_counts(),
314     subjects["sex_male"].value_counts(),
315     f"{cfg['display_name']} sex_male distribution",
316     "sex_male",
317     FIGURES_DIR / f"gender_distribution_{dataset_key}.png",
318 )
319
320 report_lines.extend(
321     [
322         f"### {cfg['display_name']}",
323         "",
324         f"- 源文件: `{rel(path)}`",
325         f"- 读取方式: `{cfg['read_method']}`",
326         f"- 原始形状: `{df_raw.shape}`",
327         f"- 元信息列: `{', '.join(cfg['metadata_cols'])}`",
328         f"- 统一性别变量规则: `{cfg['sex_male_rule']}`",
329         f"- 数值语音特征数量: `{len(features)}`",
330         f"- 完全重复行数量: `{duplicate_counts[dataset_key]}`",
331         f"- 受试者数量: `{subjects.shape[0]}`",
332         f"- 按受试者标签不一致数量: `{label_conflicts[dataset_key]}`",
333         f"- 按受试者原始性别不一致数量: `{gender_conflicts[dataset_key]}`",
334         f"- 按受试者 `sex_male` 不一致数量: `{sex_conflicts[dataset_key]}`",
335         f"- 记录级类别分布: `{df[cfg['label_col']].value_counts().sort_index().to_dict()}`",
336         f"- 受试者级类别分布: `{subjects['label_value'].value_counts().sort_index().to_dict()}`",
337         f"- 记录级 `sex_male` 分布: `{df['sex_male'].value_counts().sort_index().to_dict()}`",
338         f"- 受试者级 `sex_male` 分布: `{subjects['sex_male'].value_counts().sort_index().to_dict()}`",
339         "",
340         f"### 输出",
341         f"- 记录级描述统计: `results/descriptive_statistics_{dataset_key}.csv`",
342         f"- 受试者聚合描述统计: `results/grouped_subject_statistics_{dataset_key}.csv`",
343         f"- 记录级类别 × `sex_male` 交叉表: `results/{record_crosstab_path.name}`",
344         f"- 受试者级类别 × `sex_male` 交叉表: `results/{subject_crosstab_path.name}`",
345         "",
346     ]
347 )
348
349 if cfg["recording_col"]:
350     recording_col = cfg["recording_col"]
351     recording_counts = df[recording_col].value_counts().sort_index()
352     per_subject_recording = df.groupby(cfg["id_col"])[recording_col].apply(
353         lambda s: tuple(sorted(s.dropna().tolist()))
354     )
355     expected = (1, 2, 3)
356     report_lines.extend(
357         [

```

```

358         """### Recording 分布检查",
359         f"- 记录级 `Recording` 分布: `{recording_counts.to_dict()}`",
360         f"- 每名受试者均有且仅有一次 `Recording=1/2/3` 的数量: `{int((per_subject_recording ==
        ↪ expected).sum())}`",
361         f"- 不满足上述条件的受试者数量: `{int((per_subject_recording != expected).sum())}`",
362         """,
363     ]
364 )
365
366 dataset1 = loaded["dataset1"]
367 dataset2 = loaded["dataset2"]
368 dataset1_dedup = dataset1.drop_duplicates()
369 dataset1_dedup_path = RESULTS_DIR / "model_ready_dataset1_deduplicated.csv"
370 dataset2_prepared_path = RESULTS_DIR / "model_ready_dataset2_prepared.csv"
371 write_csv(dataset1_dedup, dataset1_dedup_path)
372 write_csv(dataset2, dataset2_prepared_path)
373
374 d1_dedup_record_counts = (
375     dataset1_dedup.groupby(DATASET_CONFIG["dataset1"]["id_col"])
376     .size()
377     .value_counts()
378     .sort_index()
379     .to_dict()
380 )
381
382 report_lines.extend(
383     [
384         """### Dataset 1 去重说明",
385         """,
386         "- 原始版保留 756 行, 用于数据描述。",
387         "- 去重版 755 行用于避免完全重复样本对建模产生重复加权。",
388         "- 使用去重版后, 不再严格满足每个受试者 3 条记录, 后续受试者级聚合必须显式说明。",
389         f"- 去重后受试者记录数分布: `{d1_dedup_record_counts}`。",
390         f"- 去重建模准备版: `results/{dataset1_dedup_path.name}`。",
391         f"- Dataset 2 prepared 建模准备版: `results/{dataset2_prepared_path.name}`。",
392         """,
393         """### 图像输出",
394         """,
395         "- `results/figures/class_distribution_dataset1.png`",
396         "- `results/figures/class_distribution_dataset2.png`",
397         "- `results/figures/gender_distribution_dataset1.png`",
398         "- `results/figures/gender_distribution_dataset2.png`",
399         """,
400         """### 边界说明",
401         """,
402         "本阶段只生成描述统计、受试者聚合、编码统一、交叉表、去重准备数据和基础分布图, 不生成预测结果、降维结果、合成采样结果、
        ↪ 无监督分组结果或特征排序结果。",
403         """,
404     ]
405 )
406
407 (RESULTS_DIR / "descriptive_report.md").write_text(
408     "\n".join(report_lines), encoding="utf-8"
409 )
410
411 print("Generated stage 2 QC outputs:")
412 for path in [
413     RESULTS_DIR / "descriptive_statistics_dataset1.csv",
414     RESULTS_DIR / "descriptive_statistics_dataset2.csv",
415     RESULTS_DIR / "grouped_subject_statistics_dataset1.csv",
416     RESULTS_DIR / "grouped_subject_statistics_dataset2.csv",
417     *crosstab_outputs,
418     dataset1_dedup_path,
419     dataset2_prepared_path,
420     RESULTS_DIR / "descriptive_report.md",
421 ]:
422     print(f"- {rel(path)}")
423
424
425 if __name__ == "__main__":
426     main()

```

A.2 代码二：二分类辅助诊断模型

```
1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from pathlib import Path
5 from typing import Any
6
7 import numpy as np
8 import pandas as pd
9 from sklearn.ensemble import RandomForestClassifier
10 from sklearn.linear_model import LogisticRegression
11 from sklearn.metrics import (
12     accuracy_score,
13     balanced_accuracy_score,
14     confusion_matrix,
15     f1_score,
16     precision_score,
17     recall_score,
18     roc_auc_score,
19 )
20 from sklearn.model_selection import GroupKFold
21 from sklearn.pipeline import Pipeline
22 from sklearn.preprocessing import StandardScaler
23 from sklearn.svm import SVC
24
25 try:
26     from sklearn.model_selection import StratifiedGroupKFold
27
28     HAS_STRATIFIED_GROUP_KFOLD = True
29     STRATIFIED_GROUP_KFOLD_ERROR = ""
30 except ImportError as exc:
31     StratifiedGroupKFold = None
32     HAS_STRATIFIED_GROUP_KFOLD = False
33     STRATIFIED_GROUP_KFOLD_ERROR = str(exc)
34
35
36 ROOT = Path(__file__).resolve().parents[1]
37 RESULTS_DIR = ROOT / "results"
38 OUTPUT_CSV_ENCODING = "utf-8-sig"
39 RANDOM_STATE = 42
40 N_SPLITS = 5
41 POS_LABEL = 1
42 NEG_LABEL = 0
43 THRESHOLD = 0.5
44
45
46 @dataclass(frozen=True)
47 class DatasetConfig:
48     key: str
49     display_name: str
50     path: Path
51     id_col: str
52     label_col: str
53     metadata_cols: tuple[str, ...]
54     expected_record_count_distribution: dict[int, int] | None = None
55
56
57 DATASET_CONFIGS = (
58     DatasetConfig(
59         key="dataset1",
60         display_name="Dataset 1",
61         path=RESULTS_DIR / "model_ready_dataset1_deduplicated.csv",
62         id_col="id",
63         label_col="class",
64         metadata_cols=("id", "gender", "class"),
65         expected_record_count_distribution={2: 1, 3: 251},
66     ),
67     DatasetConfig(
68         key="dataset2",
69         display_name="Dataset 2",
70         path=RESULTS_DIR / "model_ready_dataset2_prepared.csv",
71         id_col="ID",
72         label_col="Status",
73         metadata_cols=("ID", "Recording", "Status", "Gender"),
74     ),
75 )
76
77
78 METRIC_COLUMNS = (
79     "accuracy",
80     "balanced_accuracy",
81     "roc_auc",
82     "sensitivity",
83     "specificity",
84     "precision",
85     "recall",
86     "f1",
87 )
88
89
90 def rel(path: Path) -> str:
```

```

91     return path.relative_to(ROOT).as_posix()
92
93
94 def write_csv(df: pd.DataFrame, path: Path) -> None:
95     df.to_csv(path, index=False, encoding=OUTPUT_CSV_ENCODING)
96
97
98 def load_dataset(config: DatasetConfig) -> pd.DataFrame:
99     if not config.path.exists():
100         raise FileNotFoundError(f"Missing input file: {config.path}")
101     df = pd.read_csv(config.path)
102     required = set(config.metadata_cols) | {config.id_col, config.label_col, "sex_male"}
103     missing = sorted(required - set(df.columns))
104     if missing:
105         raise ValueError(f"{config.display_name} is missing required columns: {missing}")
106     if df[config.label_col].isna().any():
107         raise ValueError(f"{config.display_name} has missing labels")
108     if df[config.id_col].isna().any():
109         raise ValueError(f"{config.display_name} has missing subject IDs")
110     if df.isna().sum().sum() > 0:
111         raise ValueError(f"{config.display_name} contains missing values")
112     label_unique_by_subject = df.groupby(config.id_col)[config.label_col].nunique(
113         dropna=False
114     )
115     if int(label_unique_by_subject.max()) != 1:
116         raise ValueError(f"{config.display_name} has subjects with inconsistent labels")
117     return df
118
119
120 def acoustic_columns(df: pd.DataFrame, config: DatasetConfig) -> list[str]:
121     excluded = set(config.metadata_cols) | {"sex_male"}
122     return [
123         column
124         for column in df.select_dtypes(include="number").columns
125         if column not in excluded
126     ]
127
128
129 def feature_sets(df: pd.DataFrame, config: DatasetConfig) -> dict[str, list[str]]:
130     acoustic = acoustic_columns(df, config)
131     if not acoustic:
132         raise ValueError(f"{config.display_name} has no acoustic feature columns")
133     return {
134         "acoustic_only": acoustic,
135         "acoustic_plus_sex": acoustic + ["sex_male"],
136     }
137
138
139 def make_models() -> dict[str, Pipeline]:
140     return {
141         "Logistic Regression": Pipeline(
142             steps=[
143                 ("scaler", StandardScaler()),
144                 (
145                     "model",
146                     LogisticRegression(max_iter=5000, random_state=RANDOM_STATE),
147                 ),
148             ]
149         ),
150         "SVM-RBF": Pipeline(
151             steps=[
152                 ("scaler", StandardScaler()),
153                 (
154                     "model",
155                     SVC(
156                         kernel="rbf",
157                         probability=True,
158                         random_state=RANDOM_STATE,
159                     ),
160                 ),
161             ]
162         ),
163         "Random Forest": Pipeline(
164             steps=[
165                 (
166                     "model",
167                     RandomForestClassifier(
168                         n_estimators=500,
169                         random_state=RANDOM_STATE,
170                         n_jobs=-1,
171                     ),
172                 ),
173             ]
174         ),
175     }
176
177
178 def make_splitter() -> tuple[Any, str, str]:
179     if HAS_STRATIFIED_GROUP_KFOLD:
180         return (
181             StratifiedGroupKFold(
182                 n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE

```

```

183         ),
184         "StratifiedGroupKFold",
185         "",
186     )
187     return (
188         GroupKFold(n_splits=N_SPLITS),
189         "GroupKFold",
190         STRATIFIED_GROUP_KFOLD_ERROR,
191     )
192
193
194 def distribution(values: pd.Series | np.ndarray) -> dict[int, int]:
195     series = pd.Series(values).astype(int)
196     return {
197         int(label): int(count)
198         for label, count in series.value_counts().sort_index().items()
199     }
200
201
202 def subject_label_distribution(
203     df: pd.DataFrame, id_col: str, label_col: str
204 ) -> dict[int, int]:
205     labels = df.groupby(id_col, dropna=False)[label_col].first()
206     return distribution(labels)
207
208
209 def add_split_audit(
210     audit_rows: list[dict[str, Any]],
211     config: DatasetConfig,
212     df: pd.DataFrame,
213     train_idx: np.ndarray,
214     test_idx: np.ndarray,
215     fold: int,
216     splitter_name: str,
217     fallback_reason: str,
218 ) -> None:
219     train_df = df.iloc[train_idx]
220     test_df = df.iloc[test_idx]
221     train_subjects = set(train_df[config.id_col])
222     test_subjects = set(test_df[config.id_col])
223     overlap = train_subjects & test_subjects
224     if overlap:
225         raise ValueError(
226             f"{config.display_name} fold {fold} has subject leakage: {sorted(overlap)[:5]}"
227         )
228     audit_rows.append(
229         {
230             "dataset": config.key,
231             "dataset_display_name": config.display_name,
232             "fold": fold,
233             "splitter": splitter_name,
234             "fallback_reason": fallback_reason,
235             "train_subject_count": len(train_subjects),
236             "test_subject_count": len(test_subjects),
237             "train_record_count": len(train_df),
238             "test_record_count": len(test_df),
239             "train_record_class_distribution": distribution(train_df[config.label_col]),
240             "test_record_class_distribution": distribution(test_df[config.label_col]),
241             "train_subject_class_distribution": subject_label_distribution(
242                 train_df, config.id_col, config.label_col
243             ),
244             "test_subject_class_distribution": subject_label_distribution(
245                 test_df, config.id_col, config.label_col
246             ),
247             "overlap_subject_count": len(overlap),
248         }
249     )
250
251
252 def positive_probability(model: Pipeline, x_test: pd.DataFrame) -> np.ndarray:
253     if hasattr(model, "predict_proba"):
254         classes = list(model.classes_)
255         if POS_LABEL not in classes:
256             raise ValueError(f"Positive label {POS_LABEL} not found in model classes")
257         return model.predict_proba(x_test)[:, classes.index(POS_LABEL)]
258     if hasattr(model, "decision_function"):
259         scores = model.decision_function(x_test)
260         return 1.0 / (1.0 + np.exp(-scores))
261     raise ValueError("Model does not expose predict_proba or decision_function")
262
263
264 def compute_metrics(
265     y_true: pd.Series | np.ndarray, y_prob: np.ndarray
266 ) -> tuple[dict[str, float], dict[str, int]]:
267     y_true_array = np.asarray(y_true).astype(int)
268     y_pred = (np.asarray(y_prob) >= THRESHOLD).astype(int)
269     tn, fp, fn, tp = confusion_matrix(
270         y_true_array, y_pred, labels=[NEG_LABEL, POS_LABEL]
271     ).ravel()
272     roc_auc = (
273         float(roc_auc_score(y_true_array, y_prob))
274         if len(np.unique(y_true_array)) == 2

```

```

275         else np.nan
276     )
277     metrics = {
278         "accuracy": float(accuracy_score(y_true_array, y_pred)),
279         "balanced_accuracy": float(balanced_accuracy_score(y_true_array, y_pred)),
280         "roc_auc": roc_auc,
281         "sensitivity": float(tp / (tp + fn)) if (tp + fn) else np.nan,
282         "specificity": float(tn / (tn + fp)) if (tn + fp) else np.nan,
283         "precision": float(
284             precision_score(y_true_array, y_pred, pos_label=POS_LABEL, zero_division=0)
285         ),
286         "recall": float(
287             recall_score(y_true_array, y_pred, pos_label=POS_LABEL, zero_division=0)
288         ),
289         "f1": float(f1_score(y_true_array, y_pred, pos_label=POS_LABEL, zero_division=0)),
290     }
291     matrix = {"tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp)}
292     return metrics, matrix
293
294
295 def subject_level_predictions(
296     df_test: pd.DataFrame,
297     config: DatasetConfig,
298     record_prob: np.ndarray,
299 ) -> pd.DataFrame:
300     pred_df = pd.DataFrame(
301         {
302             "subject_id": df_test[config.id_col].to_numpy(),
303             "y_true": df_test[config.label_col].astype(int).to_numpy(),
304             "y_prob": record_prob,
305         }
306     )
307     subject_df = (
308         pred_df.groupby("subject_id", dropna=False)
309         .agg(y_true=("y_true", "first"), y_prob=("y_prob", "mean"))
310         .reset_index()
311     )
312     label_counts = pred_df.groupby("subject_id", dropna=False)["y_true"].nunique(
313         dropna=False
314     )
315     if int(label_counts.max()) != 1:
316         raise ValueError(f"{config.display_name} test fold has inconsistent labels")
317     return subject_df
318
319
320 def metric_row(
321     config: DatasetConfig,
322     feature_set: str,
323     model_name: str,
324     fold: int | str,
325     summary: str,
326     n_records: int,
327     n_subjects: int,
328     metrics: dict[str, float],
329 ) -> dict[str, Any]:
330     row: dict[str, Any] = {
331         "dataset": config.key,
332         "dataset_display_name": config.display_name,
333         "feature_set": feature_set,
334         "model": model_name,
335         "fold": fold,
336         "summary": summary,
337         "n_records": n_records,
338         "n_subjects": n_subjects,
339     }
340     row.update(metrics)
341     return row
342
343
344 def confusion_row(
345     config: DatasetConfig,
346     feature_set: str,
347     model_name: str,
348     level: str,
349     fold: int | str,
350     summary: str,
351     matrix: dict[str, int],
352 ) -> dict[str, Any]:
353     return {
354         "dataset": config.key,
355         "dataset_display_name": config.display_name,
356         "feature_set": feature_set,
357         "model": model_name,
358         "level": level,
359         "fold": fold,
360         "summary": summary,
361         **matrix,
362     }
363
364
365 def append_summary_rows(metrics_df: pd.DataFrame) -> pd.DataFrame:
366     group_cols = ["dataset", "dataset_display_name", "feature_set", "model"]

```

```

367 summary_rows: list[dict[str, Any]] = []
368 fold_rows = metrics_df[metrics_df["summary"] == "fold"]
369 for values, group in fold_rows.groupby(group_cols, dropna=False):
370     base = dict(zip(group_cols, values, strict=True))
371     for summary, values_by_metric in (
372         ("mean", group[list(METRIC_COLUMNS)].mean(numeric_only=True)),
373         ("std", group[list(METRIC_COLUMNS)].std(ddof=1, numeric_only=True)),
374     ):
375         row = {
376             **base,
377             "fold": "",
378             "summary": summary,
379             "n_records": "",
380             "n_subjects": "",
381         }
382         row.update({metric: values_by_metric[metric] for metric in METRIC_COLUMNS})
383         summary_rows.append(row)
384 return pd.concat([metrics_df, pd.DataFrame(summary_rows)], ignore_index=True)
385
386
387 def append_confusion_summary_rows(confusion_df: pd.DataFrame) -> pd.DataFrame:
388     group_cols = ["dataset", "dataset_display_name", "feature_set", "model", "level"]
389     matrix_cols = ["tn", "fp", "fn", "tp"]
390     summary_rows: list[dict[str, Any]] = []
391     fold_rows = confusion_df[confusion_df["summary"] == "fold"]
392     for values, group in fold_rows.groupby(group_cols, dropna=False):
393         row = dict(zip(group_cols, values, strict=True))
394         row.update(
395             {
396                 "fold": "",
397                 "summary": "aggregate",
398                 **{
399                     column: int(group[column].sum())
400                     for column in matrix_cols
401                 },
402             }
403         )
404         summary_rows.append(row)
405 return pd.concat([confusion_df, pd.DataFrame(summary_rows)], ignore_index=True)
406
407
408 def markdown_table(df: pd.DataFrame, floatfmt: str = ".4f") -> str:
409     headers = list(df.columns)
410     rows = []
411     for _, row in df.iterrows():
412         values = []
413         for column in headers:
414             value = row[column]
415             if isinstance(value, float) and not pd.isna(value):
416                 values.append(format(value, floatfmt))
417             elif pd.isna(value):
418                 values.append("")
419             else:
420                 values.append(str(value))
421         rows.append(values)
422
423     table = [
424         "| " + " | ".join(headers) + " |",
425         "| " + " | ".join(["---"] * len(headers)) + " |",
426     ]
427     table.extend("| " + " | ".join(row) + " |" for row in rows)
428     return "\n".join(table)
429
430
431 def run_dataset(
432     config: DatasetConfig,
433     record_metric_rows: list[dict[str, Any]],
434     subject_metric_rows: list[dict[str, Any]],
435     audit_rows: list[dict[str, Any]],
436     confusion_rows: list[dict[str, Any]],
437     report_lines: list[str],
438 ) -> None:
439     df = load_dataset(config)
440     features_by_name = feature_sets(df, config)
441     models = make_models()
442     splitter, splitter_name, fallback_reason = make_splitter()
443
444     y = df[config.label_col].astype(int)
445     groups = df[config.id_col]
446     splits = list(splitter.split(df, y, groups))
447     if len(splits) != N_SPLITS:
448         raise ValueError(f"{config.display_name} expected {N_SPLITS} folds")
449
450     record_count_distribution = (
451         df.groupby(config.id_col).size().value_counts().sort_index().to_dict()
452     )
453     if config.expected_record_count_distribution is not None:
454         expected = config.expected_record_count_distribution
455         if record_count_distribution != expected:
456             raise ValueError(
457                 f"{config.display_name} record count distribution changed: "
458                 f"{record_count_distribution}, expected {expected}"

```

```

459         )
460
461     report_lines.extend(
462         [
463             f"## {config.display_name}",
464             "",
465             f"- Input: `{rel(config.path)}`",
466             f"- Splitter: `{splitter_name}`",
467             f"- Subjects: `{df[config.id_col].nunique()}`",
468             f"- Records: `{len(df)}`",
469             f"- Subject record count distribution: `{record_count_distribution}`",
470             f"- Record-level class distribution: `{distribution(df[config.label_col])}`",
471             f"- Subject-level class distribution: "
472             f"{subject_label_distribution(df, config.id_col, config.label_col)}",
473             f"- Acoustic feature count: `{len(features_by_name['acoustic_only'])}`",
474             "- Feature sets: `acoustic_only` is the main analysis; "
475             "`acoustic_plus_sex` is supplemental.",
476         ]
477     )
478     if config.key == "dataset2":
479         report_lines.append(
480             "- Dataset 2 prepared file only contains metadata cleanup and sex_male "
481             "encoding; acoustic features are not globally z-score standardized before "
482             "cross-validation. Scaling for Logistic Regression and SVM-RBF is "
483             "performed only within sklearn Pipeline in each training fold."
484         )
485     report_lines.extend(["", "### Fold audit", ""])
486     if fallback_reason:
487         report_lines.extend(
488             [
489                 f"- StratifiedGroupKFold unavailable; fallback reason: `{fallback_reason}`",
490                 "",
491             ]
492         )
493
494     for fold, (train_idx, test_idx) in enumerate(splits, start=1):
495         add_split_audit(
496             audit_rows,
497             config,
498             df,
499             train_idx,
500             test_idx,
501             fold,
502             splitter_name,
503             fallback_reason,
504         )
505         train_df = df.iloc[train_idx]
506         test_df = df.iloc[test_idx]
507         report_lines.append(
508             f"- Fold {fold}: train subjects `{train_df[config.id_col].nunique()}`, "
509             f"test subjects `{test_df[config.id_col].nunique()}`, "
510             f"train records `{len(train_df)}`, test records `{len(test_df)}`, "
511             f"test subject classes "
512             f"{subject_label_distribution(test_df, config.id_col, config.label_col)}"
513         )
514
515         for feature_set_name, columns in features_by_name.items():
516             x_train = train_df[columns]
517             x_test = test_df[columns]
518             y_train = train_df[config.label_col].astype(int)
519             y_test = test_df[config.label_col].astype(int)
520
521             for model_name, model in models.items():
522                 model.fit(x_train, y_train)
523                 record_prob = positive_probability(model, x_test)
524
525                 record_metrics, record_matrix = compute_metrics(y_test, record_prob)
526                 record_metric_rows.append(
527                     metric_row(
528                         config,
529                         feature_set_name,
530                         model_name,
531                         fold,
532                         "fold",
533                         len(test_df),
534                         test_df[config.id_col].nunique(),
535                         record_metrics,
536                     )
537                 )
538                 confusion_rows.append(
539                     confusion_row(
540                         config,
541                         feature_set_name,
542                         model_name,
543                         "record",
544                         fold,
545                         "fold",
546                         record_matrix,
547                     )
548                 )
549
550     subject_df = subject_level_predictions(test_df, config, record_prob)

```

```

551         subject_metrics, subject_matrix = compute_metrics(
552             subject_df["y_true"], subject_df["y_prob"].to_numpy()
553         )
554         subject_metric_rows.append(
555             metric_row(
556                 config,
557                 feature_set_name,
558                 model_name,
559                 fold,
560                 "fold",
561                 len(test_df),
562                 len(subject_df),
563                 subject_metrics,
564             )
565         )
566         confusion_rows.append(
567             confusion_row(
568                 config,
569                 feature_set_name,
570                 model_name,
571                 "subject",
572                 fold,
573                 "fold",
574                 subject_matrix,
575             )
576         )
577     report_lines.append("")
578
579
580 def write_report(
581     report_lines: list[str],
582     record_metrics: pd.DataFrame,
583     subject_metrics: pd.DataFrame,
584 ) -> None:
585     report_lines.extend(
586         [
587             "## Main subject-level results",
588             "",
589             "The primary model comparison uses `acoustic_only` subject-level mean metrics.",
590             "",
591         ]
592     )
593     main = subject_metrics[
594         (subject_metrics["summary"] == "mean")
595         & (subject_metrics["feature_set"] == "acoustic_only")
596     ].copy()
597     display_cols = [
598         "dataset_display_name",
599         "model",
600         "accuracy",
601         "balanced_accuracy",
602         "roc_auc",
603         "sensitivity",
604         "specificity",
605         "precision",
606         "recall",
607         "f1",
608     ]
609     report_lines.append(markdown_table(main[display_cols], floatfmt=".4f"))
610     report_lines.extend(
611         [
612             "",
613             "## Output files",
614             "",
615             "- `results/stage3_baseline_metrics_record_level.csv`",
616             "- `results/stage3_baseline_metrics_subject_level.csv`",
617             "- `results/stage3_fold_split_audit.csv`",
618             "- `results/stage3_confusion_matrices.csv`",
619             "- `results/stage3_baseline_report.md`",
620             "",
621             "## Constraints",
622             "",
623             "- No row-wise random `train_test_split` was used.",
624             "- All train/test splits are grouped by subject ID.",
625             "- Standardization is only applied inside sklearn `Pipeline` objects.",
626             "- No SMOTE, PCA, feature selection, SHAP, clustering, or large grid search was used.",
627             "- `sex_male` is excluded from the main `acoustic_only` models.",
628             "",
629         ]
630     )
631     (RESULTS_DIR / "stage3_baseline_report.md").write_text(
632         "\n".join(report_lines), encoding="utf-8"
633     )
634
635
636 def main() -> None:
637     RESULTS_DIR.mkdir(exist_ok=True)
638     record_metric_rows: list[dict[str, Any]] = []
639     subject_metric_rows: list[dict[str, Any]] = []
640     audit_rows: list[dict[str, Any]] = []
641     confusion_rows: list[dict[str, Any]] = []
642     report_lines = [

```

```

643     "# Stage 3 Baseline PD/Healthy Classification Report",
644     "",
645     "This report is generated by `src/stage3_baseline_models.py`. ",
646     "",
647     "Positive class: `1 = PD`; negative class: `0 = Healthy`.",
648     "",
649     "Models: Logistic Regression, SVM-RBF, Random Forest.",
650     "",
651 ]
652
653 for config in DATASET_CONFIGS:
654     run_dataset(
655         config,
656         record_metric_rows,
657         subject_metric_rows,
658         audit_rows,
659         confusion_rows,
660         report_lines,
661     )
662
663 record_metrics = append_summary_rows(pd.DataFrame(record_metric_rows))
664 subject_metrics = append_summary_rows(pd.DataFrame(subject_metric_rows))
665 audit = pd.DataFrame(audit_rows)
666 confusion = append_confusion_summary_rows(pd.DataFrame(confusion_rows))
667
668 write_csv(
669     record_metrics, RESULTS_DIR / "stage3_baseline_metrics_record_level.csv"
670 )
671 write_csv(
672     subject_metrics, RESULTS_DIR / "stage3_baseline_metrics_subject_level.csv"
673 )
674 write_csv(audit, RESULTS_DIR / "stage3_fold_split_audit.csv")
675 write_csv(confusion, RESULTS_DIR / "stage3_confusion_matrices.csv")
676 write_report(report_lines, record_metrics, subject_metrics)
677
678
679 if __name__ == "__main__":
680     main()

```

A.3 代码三：判别性声学特征识别

```
1 from __future__ import annotations
2
3 from pathlib import Path
4 from typing import Any
5
6 import matplotlib
7
8 matplotlib.use("Agg")
9
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import pandas as pd
13 from scipy.stats import mannwhitneyu, ttest_ind
14 from sklearn.ensemble import ExtraTreesClassifier
15 from sklearn.metrics import balanced_accuracy_score
16 from sklearn.linear_model import LogisticRegressionCV
17 from sklearn.model_selection import StratifiedKFold
18 from sklearn.pipeline import Pipeline
19 from sklearn.preprocessing import StandardScaler
20
21
22 ROOT = Path(__file__).resolve().parents[1]
23 RESULTS_DIR = ROOT / "results"
24 FIGURES_DIR = RESULTS_DIR / "figures"
25 DATASET1_PATH = RESULTS_DIR / "model_ready_dataset1_deduplicated.csv"
26 DATASET2_REFERENCE_PATH = RESULTS_DIR / "model_ready_dataset2_prepared.csv"
27 OUTPUT_CSV_ENCODING = "utf-8-sig"
28
29 RANDOM_STATE = 42
30 N_SPLITS = 5
31 POS_LABEL = 1
32 NEG_LABEL = 0
33 NEAR_CONSTANT_STD_THRESHOLD = 1e-12
34 L1_CS = [0.01, 0.03, 0.1, 0.3, 1.0]
35 COEF_EPSILON = 1e-8
36 N_ESTIMATORS = 500
37 PERMUTATION_REPEATS = 1
38
39 METADATA_COLUMNS = {"id", "gender", "class", "sex_male"}
40
41
42 def rel(path: Path) -> str:
43     return path.relative_to(ROOT).as_posix()
44
45
46 def write_csv(df: pd.DataFrame, path: Path) -> None:
47     df.to_csv(path, index=False, encoding=OUTPUT_CSV_ENCODING)
48
49
50 def load_dataset1() -> pd.DataFrame:
51     if not DATASET1_PATH.exists():
52         raise FileNotFoundError(f"Missing input file: {DATASET1_PATH}")
53     df = pd.read_csv(DATASET1_PATH)
54     required = {"id", "gender", "class", "sex_male"}
55     missing = sorted(required - set(df.columns))
56     if missing:
57         raise ValueError(f"Dataset 1 is missing required columns: {missing}")
58     if df.isna().sum().sum() > 0:
59         raise ValueError("Dataset 1 contains missing values before aggregation")
60     if int(df.groupby("id")["class"].nunique(dropna=False).max()) != 1:
61         raise ValueError("Dataset 1 has subjects with inconsistent class labels")
62     if int(df.groupby("id")["sex_male"].nunique(dropna=False).max()) != 1:
63         raise ValueError("Dataset 1 has subjects with inconsistent sex_male values")
64     return df
65
66
67 def acoustic_columns(df: pd.DataFrame) -> list[str]:
68     columns = [
69         column
70         for column in df.select_dtypes(include="number").columns
71         if column not in METADATA_COLUMNS
72     ]
73     if len(columns) != 752:
74         raise ValueError(f"Expected 752 acoustic features, found {len(columns)}")
75     forbidden = METADATA_COLUMNS & set(columns)
76     if forbidden:
77         raise ValueError(f"Forbidden metadata columns in acoustic features: {forbidden}")
78     return columns
79
80
81 def build_subject_table(df: pd.DataFrame, features: list[str]) -> pd.DataFrame:
82     feature_means = df.groupby("id", dropna=False)[features].mean().reset_index()
83     subject_meta = (
84         df.groupby("id", dropna=False)
85         .agg(
86             class_label=("class", "first"),
87             sex_male=("sex_male", "first"),
88             record_count=("id", "size"),
89         )
90         .reset_index()
```

```

91     .rename(columns={"class_label": "class"})
92 )
93 subject_table = subject_meta.merge(feature_means, on="id", how="left")
94 if subject_table.shape[0] != 252:
95     raise ValueError(f"Expected 252 subjects, found {subject_table.shape[0]}")
96 return subject_table
97
98
99 def audit_subject_table(
100     subject_table: pd.DataFrame, features: list[str]
101 ) -> dict[str, Any]:
102     feature_audit = build_feature_audit_table(subject_table, features)
103     return {
104         "subject_count": int(subject_table.shape[0]),
105         "class_distribution": {
106             int(k): int(v)
107             for k, v in subject_table["class"].value_counts().sort_index().items()
108         },
109         "acoustic_feature_count": len(features),
110         "missing_value_count": int(subject_table.isna().sum().sum()),
111         "constant_feature_count": int(feature_audit["is_constant"].sum()),
112         "near_constant_feature_count": int(
113             feature_audit["is_near_constant"].sum()
114         ),
115         "near_constant_features": feature_audit.loc[
116             feature_audit["is_near_constant"], "feature"
117         ].tolist(),
118     }
119
120
121 def build_feature_audit_table(
122     subject_table: pd.DataFrame, features: list[str]
123 ) -> pd.DataFrame:
124     feature_values = subject_table[features]
125     stds = feature_values.std(ddof=0)
126     unique_counts = feature_values.nunique(dropna=False)
127     missing_counts = feature_values.isna().sum()
128     audit = pd.DataFrame(
129         {
130             "feature": features,
131             "category": [categorize_feature(feature) for feature in features],
132             "subject_level_std": stds.reindex(features).to_numpy(),
133             "subject_level_unique_count": unique_counts.reindex(features).to_numpy(),
134             "subject_level_missing_count": missing_counts.reindex(features).to_numpy(),
135         }
136     )
137     audit["is_constant"] = audit["subject_level_unique_count"] <= 1
138     audit["is_near_constant"] = (
139         audit["subject_level_std"] <= NEAR_CONSTANT_STD_THRESHOLD
140     )
141     return audit
142
143
144 def merge_feature_audit(df: pd.DataFrame, feature_audit: pd.DataFrame) -> pd.DataFrame:
145     audit_cols = [
146         "feature",
147         "subject_level_std",
148         "subject_level_unique_count",
149         "subject_level_missing_count",
150         "is_constant",
151         "is_near_constant",
152     ]
153     return df.merge(feature_audit[audit_cols], on="feature", how="left")
154
155
156 def add_q_value_display_columns(df: pd.DataFrame) -> pd.DataFrame:
157     out = df.copy()
158     for column in ["q_mannwhitney", "q_welch"]:
159         if column in out.columns:
160             out[f"{column}_scientific"] = out[column].map(
161                 lambda value: "" if pd.isna(value) else f"{value:.3e}"
162             )
163     return out
164
165
166 def fdr_bh(p_values: np.ndarray) -> np.ndarray:
167     p = np.asarray(p_values, dtype=float)
168     adjusted = np.full(p.shape, np.nan, dtype=float)
169     valid = np.isfinite(p)
170     if not valid.any():
171         return adjusted
172     valid_p = p[valid]
173     order = np.argsort(valid_p)
174     ordered_p = valid_p[order]
175     n = len(ordered_p)
176     ordered_q = ordered_p * n / np.arange(1, n + 1)
177     ordered_q = np.minimum.accumulate(ordered_q[::-1])[::-1]
178     ordered_q = np.clip(ordered_q, 0, 1)
179     q = np.empty_like(ordered_q)
180     q[order] = ordered_q
181     adjusted[valid] = q
182     return adjusted

```

```

183
184
185 def cohens_d(pd_values: np.ndarray, healthy_values: np.ndarray) -> float:
186     n_pd = len(pd_values)
187     n_healthy = len(healthy_values)
188     if n_pd < 2 or n_healthy < 2:
189         return np.nan
190     var_pd = np.var(pd_values, ddof=1)
191     var_healthy = np.var(healthy_values, ddof=1)
192     pooled = ((n_pd - 1) * var_pd + (n_healthy - 1) * var_healthy) / (
193         n_pd + n_healthy - 2
194     )
195     if pooled <= 0:
196         return 0.0
197     return float((np.mean(pd_values) - np.mean(healthy_values)) / np.sqrt(pooled))
198
199
200 def cliffs_delta(pd_values: np.ndarray, healthy_values: np.ndarray) -> float:
201     comparisons = pd_values[:, None] - healthy_values[None, :]
202     greater = np.sum(comparisons > 0)
203     lesser = np.sum(comparisons < 0)
204     return float((greater - lesser) / comparisons.size)
205
206
207 def run_univariate_statistics(
208     subject_table: pd.DataFrame, features: list[str]
209 ) -> pd.DataFrame:
210     y = subject_table["class"].astype(int).to_numpy()
211     rows: list[dict[str, Any]] = []
212     for feature in features:
213         values = subject_table[feature].to_numpy(dtype=float)
214         pd_values = values[y == POS_LABEL]
215         healthy_values = values[y == NEG_LABEL]
216         try:
217             u_stat, mw_p = mannwhitneyu(
218                 pd_values, healthy_values, alternative="two-sided"
219             )
220         except ValueError:
221             u_stat, mw_p = np.nan, np.nan
222         try:
223             t_stat, t_p = ttest_ind(
224                 pd_values, healthy_values, equal_var=False, nan_policy="omit"
225             )
226         except ValueError:
227             t_stat, t_p = np.nan, np.nan
228         d = cohens_d(pd_values, healthy_values)
229         delta = cliffs_delta(pd_values, healthy_values)
230         rows.append(
231             {
232                 "feature": feature,
233                 "category": categorize_feature(feature),
234                 "mean_pd": float(np.mean(pd_values)),
235                 "mean_healthy": float(np.mean(healthy_values)),
236                 "median_pd": float(np.median(pd_values)),
237                 "median_healthy": float(np.median(healthy_values)),
238                 "mannwhitney_u": float(u_stat),
239                 "p_mannwhitney": float(mw_p),
240                 "welch_t": float(t_stat),
241                 "p_welch": float(t_p),
242                 "cohens_d": d,
243                 "abs_cohens_d": abs(d),
244                 "cliffs_delta": delta,
245                 "abs_cliffs_delta": abs(delta),
246             }
247         )
248     stats = pd.DataFrame(rows)
249     stats["q_mannwhitney"] = fdr_bh(stats["p_mannwhitney"].to_numpy())
250     stats["q_welch"] = fdr_bh(stats["p_welch"].to_numpy())
251     return stats.sort_values(["q_mannwhitney", "p_mannwhitney"], na_position="last")
252
253
254 def stratified_folds(y: np.ndarray) -> StratifiedKFold:
255     return StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE)
256
257
258 def run_l1_stability(subject_table: pd.DataFrame, features: list[str]) -> pd.DataFrame:
259     x = subject_table[features]
260     y = subject_table["class"].astype(int).to_numpy()
261     selected_counts = pd.Series(0, index=features, dtype=int)
262     abs_coef_sums = pd.Series(0.0, index=features, dtype=float)
263
264     for fold, (train_idx, _) in enumerate(stratified_folds(y).split(x, y), start=1):
265         pipeline = Pipeline(
266             steps=[
267                 ("scaler", StandardScaler()),
268                 (
269                     "model",
270                     LogisticRegressionCV(
271                         Cs=L1_CS,
272                         cv=3,
273                         penalty="l1",
274                         solver="liblinear",

```

```

275         scoring="balanced_accuracy",
276         max_iter=5000,
277         random_state=RANDOM_STATE + fold,
278         n_jobs=1,
279     ),
280 ),
281 ],
282 )
283 pipeline.fit(x.iloc[train_idx], y[train_idx])
284 coefs = np.abs(pipeline.named_steps["model"].coef_.ravel())
285 selected = coefs > COEF_EPSILON
286 selected_counts += selected.astype(int)
287 abs_coef_sums += coefs
288
289 out = pd.DataFrame(
290     {
291         "feature": features,
292         "category": [categorize_feature(feature) for feature in features],
293         "selected_folds": selected_counts.to_numpy(),
294         "selection_frequency": selected_counts.to_numpy() / N_SPLITS,
295         "mean_abs_l1_coefficient": abs_coef_sums.to_numpy() / N_SPLITS,
296     }
297 )
298 return out.sort_values(
299     ["selection_frequency", "mean_abs_l1_coefficient"],
300     ascending=[False, False],
301 )
302
303
304 def run_tree_and_permutation(
305     subject_table: pd.DataFrame, features: list[str]
306 ) -> tuple[pd.DataFrame, pd.DataFrame]:
307     x = subject_table[features].to_numpy(dtype=float)
308     y = subject_table["class"].astype(int).to_numpy()
309     tree_fold_importances: list[np.ndarray] = []
310     permutation_fold_importances: list[np.ndarray] = []
311     permutation_fold_stdts: list[np.ndarray] = []
312
313     for fold, (train_idx, test_idx) in enumerate(stratified_folds(y).split(x, y), start=1):
314         model = ExtraTreesClassifier(
315             n_estimators=N_ESTIMATORS,
316             random_state=RANDOM_STATE + fold,
317             n_jobs=1,
318         )
319         model.fit(x[train_idx], y[train_idx])
320         tree_fold_importances.append(model.feature_importances_)
321
322         permutation_mean, permutation_std = heldout_permutation_importance(
323             model,
324             x[test_idx],
325             y[test_idx],
326             random_state=RANDOM_STATE + fold,
327         )
328         permutation_fold_importances.append(permutation_mean)
329         permutation_fold_stdts.append(permutation_std)
330
331     tree_array = np.vstack(tree_fold_importances)
332     permutation_array = np.vstack(permutation_fold_importances)
333     permutation_std_array = np.vstack(permutation_fold_stdts)
334
335     tree_df = pd.DataFrame(
336         {
337             "feature": features,
338             "category": [categorize_feature(feature) for feature in features],
339             "tree_importance_mean": tree_array.mean(axis=0),
340             "tree_importance_std": tree_array.std(axis=0, ddof=1),
341         }
342     ).sort_values("tree_importance_mean", ascending=False)
343
344     permutation_df = pd.DataFrame(
345         {
346             "feature": features,
347             "category": [categorize_feature(feature) for feature in features],
348             "permutation_importance_mean": permutation_array.mean(axis=0),
349             "permutation_importance_std_across_folds": permutation_array.std(
350                 axis=0, ddof=1
351             ),
352             "mean_within_fold_permutation_std": permutation_std_array.mean(axis=0),
353         }
354     ).sort_values("permutation_importance_mean", ascending=False)
355     return tree_df, permutation_df
356
357
358 def heldout_permutation_importance(
359     model: ExtraTreesClassifier,
360     x_test: np.ndarray,
361     y_test: np.ndarray,
362     random_state: int,
363 ) -> tuple[np.ndarray, np.ndarray]:
364     rng = np.random.default_rng(random_state)
365     baseline = balanced_accuracy_score(y_test, model.predict(x_test))
366     importances = np.zeros((x_test.shape[1], PERMUTATION_REPEATS), dtype=float)

```

```

367     for feature_index in range(x_test.shape[1]):
368         for repeat in range(PERMUTATION_REPEATS):
369             x_permuted = x_test.copy()
370             x_permuted[:, feature_index] = rng.permutation(x_permuted[:, feature_index])
371             score = balanced_accuracy_score(y_test, model.predict(x_permuted))
372             importances[feature_index, repeat] = baseline - score
373     ddof = 1 if PERMUTATION_REPEATS > 1 else 0
374     return importances.mean(axis=1), importances.std(axis=1, ddof=ddof)
375
376
377 def categorize_feature(feature: str) -> str:
378     name = feature.lower()
379     if "tqwt" in name:
380         return "TQWT"
381     if any(token in name for token in ["wavelet", "_lt_", "ed_", "ed2_", "entropy"]):
382         return "Wavelet"
383     if "jitter" in name:
384         return "Jitter"
385     if "shimmer" in name or "shim_" in name or "apq" in name:
386         return "Shimmer"
387     if "hnr" in name or "harmonicity" in name or "harmtonoise" in name:
388         return "HNR / Harmonicity"
389     if "intensity" in name:
390         return "Intensity"
391     if (
392         name.startswith("f")
393         and len(name) == 2
394         and name[1].isdigit()
395         or name.startswith("b")
396         and len(name) == 2
397         and name[1].isdigit()
398     ):
399         return "Formant / Bandwidth"
400     if "mfcc" in name:
401         return "MFCC"
402     if "delta_delta" in name or "delta" in name:
403         return "Delta / Delta-delta"
404     if any(token in name for token in ["rpde", "dfa", "ppe", "gne"]):
405         return "Nonlinear"
406     return "Other"
407
408
409 def rank_desc(series: pd.Series) -> pd.Series:
410     return series.rank(ascending=False, method="average", na_option="bottom")
411
412
413 def rank_asc(series: pd.Series) -> pd.Series:
414     return series.rank(ascending=True, method="average", na_option="bottom")
415
416
417 def build_rank_summary(
418     univariate: pd.DataFrame,
419     l1: pd.DataFrame,
420     tree: pd.DataFrame,
421     permutation: pd.DataFrame,
422 ) -> pd.DataFrame:
423     summary = (
424         univariate[
425             [
426                 "feature",
427                 "category",
428                 "p_mannwhitney",
429                 "q_mannwhitney",
430                 "cohens_d",
431                 "abs_cohens_d",
432                 "cliffs_delta",
433                 "abs_cliffs_delta",
434                 "q_mannwhitney_scientific",
435                 "subject_level_std",
436                 "subject_level_unique_count",
437                 "is_constant",
438                 "is_near_constant",
439             ]
440         ]
441         .merge(
442             l1[
443                 [
444                     "feature",
445                     "selected_folds",
446                     "selection_frequency",
447                     "mean_abs_l1_coefficient",
448                     "l1_selected_any",
449                     "l1_selected_near_constant",
450                 ]
451             ],
452             on="feature",
453             how="left",
454         )
455         .merge(
456             tree[["feature", "tree_importance_mean", "tree_importance_std"]],
457             on="feature",
458             how="left",

```

```

459     )
460     .merge(
461         permutation[
462             [
463                 "feature",
464                 "permutation_importance_mean",
465                 "permutation_importance_std_across_folds",
466                 "permutation_importance_positive",
467             ]
468         ],
469         on="feature",
470         how="left",
471     )
472 )
473 summary["rank_q_mannwhitney"] = rank_asc(summary["q_mannwhitney"])
474 summary["rank_abs_cohens_d"] = rank_desc(summary["abs_cohens_d"])
475 summary["rank_abs_cliffs_delta"] = rank_desc(summary["abs_cliffs_delta"])
476 summary["rank_selection_frequency"] = rank_desc(summary["selection_frequency"])
477 summary["rank_permutation_importance"] = rank_desc(
478     summary["permutation_importance_mean"]
479 )
480 summary["rank_tree_importance"] = rank_desc(summary["tree_importance_mean"])
481 rank_cols = [
482     "rank_q_mannwhitney",
483     "rank_abs_cohens_d",
484     "rank_abs_cliffs_delta",
485     "rank_selection_frequency",
486     "rank_permutation_importance",
487     "rank_tree_importance",
488 ]
489 summary["mean_rank"] = summary[rank_cols].mean(axis=1)
490 summary["rank_score"] = 1.0 / summary["mean_rank"]
491 summary = summary.sort_values(["mean_rank", "feature"]).reset_index(drop=True)
492 summary["included_in_top10"] = False
493 summary["included_in_top20"] = False
494 summary["included_in_top50"] = False
495 summary.loc[summary.index[:10], "included_in_top10"] = True
496 summary.loc[summary.index[:20], "included_in_top20"] = True
497 summary.loc[summary.index[:50], "included_in_top50"] = True
498 near_top50 = summary.loc[
499     summary["included_in_top50"] & summary["is_near_constant"], "feature"
500 ].tolist()
501 if near_top50:
502     raise ValueError(f"Near-constant features entered Top 50: {near_top50}")
503 return summary
504
505
506 def save_top20_rank_plot(summary: pd.DataFrame) -> None:
507     top20 = summary.head(20).sort_values("mean_rank", ascending=True)
508     fig, ax = plt.subplots(figsize=(11, 8), dpi=150)
509     ax.barh(top20["feature"], top20["rank_score"], color="#2f80ed")
510     ax.set_xlabel("Rank score (1 / mean rank)")
511     ax.set_ylabel("Feature")
512     ax.set_title("Stage 4 Top 20 Biomarker Candidates")
513     ax.invert_yaxis()
514     fig.tight_layout()
515     fig.savefig(FIGURES_DIR / "stage4_top20_biomarker_rank.png")
516     plt.close(fig)
517
518
519 def save_top20_effect_size_plot(summary: pd.DataFrame) -> None:
520     top20 = summary.head(20).copy()
521     top20["effect_direction"] = np.where(top20["cohens_d"] >= 0, "PD higher", "Healthy higher")
522     colors = top20["effect_direction"].map(
523         {"PD higher": "#d62728", "Healthy higher": "#1f77b4"}
524     )
525     top20 = top20.sort_values("cohens_d")
526     fig, ax = plt.subplots(figsize=(11, 8), dpi=150)
527     ax.barh(top20["feature"], top20["cohens_d"], color=colors.loc[top20.index])
528     ax.axvline(0, color="#333333", linewidth=0.8)
529     ax.set_xlabel("Cohen's d (PD - Healthy)")
530     ax.set_ylabel("Feature")
531     ax.set_title("Stage 4 Top 20 Effect Sizes")
532     fig.tight_layout()
533     fig.savefig(FIGURES_DIR / "stage4_top20_effect_size.png")
534     plt.close(fig)
535
536
537 def save_top50_category_plot(summary: pd.DataFrame) -> None:
538     counts = summary.head(50)["category"].value_counts().sort_values()
539     fig, ax = plt.subplots(figsize=(10, 6), dpi=150)
540     ax.barh(counts.index, counts.values, color="#27ae60")
541     ax.set_xlabel("Count in Top 50")
542     ax.set_ylabel("Feature category")
543     ax.set_title("Stage 4 Feature Category Counts in Top 50")
544     for index, value in enumerate(counts.values):
545         ax.text(value, index, f"{int(value)}", va="center")
546     fig.tight_layout()
547     fig.savefig(FIGURES_DIR / "stage4_feature_category_counts_top50.png")
548     plt.close(fig)
549
550

```

```

551 def save_volcano_plot(univariate: pd.DataFrame) -> None:
552     plot_df = univariate.copy()
553     min_positive_q = plot_df.loc[
554         plot_df["q_mannwhitney"] > 0, "q_mannwhitney"
555     ].min()
556     if pd.isna(min_positive_q):
557         min_positive_q = 1e-300
558     plot_df["safe_q"] = plot_df["q_mannwhitney"].replace(0, min_positive_q)
559     plot_df["neg_log10_q"] = -np.log10(plot_df["safe_q"])
560     fig, ax = plt.subplots(figsize=(9, 7), dpi=150)
561     ax.scatter(
562         plot_df["cohens_d"],
563         plot_df["neg_log10_q"],
564         s=18,
565         alpha=0.75,
566         color="#4b5563",
567         edgecolors="none",
568     )
569     ax.axvline(0, color="#333333", linewidth=0.8)
570     ax.set_xlabel("Cohen's d (PD - Healthy)")
571     ax.set_ylabel("-log10(FDR q-value)")
572     ax.set_title("Stage 4 Dataset 1 Volcano Plot")
573     fig.tight_layout()
574     fig.savefig(FIGURES_DIR / "stage4_volcano_plot_dataset1.png")
575     plt.close(fig)
576
577
578 def write_report(
579     audit: dict[str, Any],
580     summary: pd.DataFrame,
581     univariate: pd.DataFrame,
582     l1: pd.DataFrame,
583 ) -> None:
584     dataset2_ref = (
585         f"~{rel(DATASET2_REFERENCE_PATH)}~ exists"
586         if DATASET2_REFERENCE_PATH.exists()
587         else f"~{rel(DATASET2_REFERENCE_PATH)}~ not found"
588     )
589     top20_cols = [
590         "feature",
591         "category",
592         "mean_rank",
593         "rank_score",
594         "q_mannwhitney",
595         "cohens_d",
596         "cliffs_delta",
597         "selection_frequency",
598         "permutation_importance_mean",
599         "tree_importance_mean",
600     ]
601     top10 = summary.head(10)[top20_cols].copy()
602     top20 = summary.head(20)[top20_cols].copy()
603     top50 = summary.head(50)[["feature", "category", "mean_rank", "rank_score"]].copy()
604     near_constant_features = set(audit["near_constant_features"])
605     near_constant_in_top50 = sorted(near_constant_features & set(summary.head(50)[["feature"]]))
606     near_constant_l1_selected = l1[
607         l1["feature"].isin(near_constant_features) & (l1["selected_folds"] > 0)
608     ]
609     nonzero_permutation_count = int(
610         (summary["permutation_importance_mean"] > 0).sum()
611     )
612     lines = [
613         "# Stage 4 Biomarker Candidate Report",
614         "",
615         "Main analysis: Dataset 1 subject-level PD/Healthy comparison.",
616         "",
617         "Dataset 2 is only noted as a reference prepared file and is not used in the main biomarker ranking.",
618         f"Dataset 2 reference status: {dataset2_ref}.",
619         "",
620         "## Subject-level audit",
621         "",
622         f"- Subject count: `{audit['subject_count']}`",
623         f"- Class distribution: `{audit['class_distribution']}`",
624         f"- Acoustic feature count: `{audit['acoustic_feature_count']}`",
625         f"- Missing value count: `{audit['missing_value_count']}`",
626         f"- Constant feature count: `{audit['constant_feature_count']}`",
627         f"- Near-constant feature count (`std` <= {NEAR_CONSTANT_STD_THRESHOLD}): `{audit['near_constant_feature_count']}`",
628         f"- Near-constant features in Top 50: `{len(near_constant_in_top50)}`",
629         f"- Near-constant features selected by L1 at least once: `{len(near_constant_l1_selected)}`",
630         "- Main features exclude `id`, `gender`, `class`, and `sex_male`.",
631         "",
632         "## Methods",
633         "",
634         "- Dataset 1 records were aggregated to one row per subject before all statistical tests and models.",
635         "- Univariate screening used Mann-Whitney U, Welch t-test, FDR-BH correction, Cohen's d, and Cliff's delta.",
636         "- L1 stability selection used subject-level StratifiedKfold; scaling and L1 Logistic were fit only inside training
637         ↪ folds.",
638         "- ExtraTrees and permutation importance used subject-level StratifiedKfold; permutation importance was computed on
639         ↪ held-out folds.",
640         "- SHAP was not run because it is optional and unavailable in the current project environment.",
641         "- These features are exploratory biomarker candidates for modeling interpretation and require independent
642         ↪ validation.",

```

```

640     "- Some near-constant features were selected by L1, so L1 stability is treated as one evidence source rather than a
    ↪ standalone final conclusion.",
641     f"- Permutation importance is sparse ({nonzero_permutation_count}` of `{summary.shape[0]}` features have positive
    ↪ mean importance), which may reflect substitution effects among high-dimensional correlated acoustic features.",
642     "- The final Top 20 are candidate voice biomarker features, not clinical causal mechanisms.",
643     "",
644     "## Top 10 biomarker candidates",
645     "",
646     markdown_table(top10, floatfmt=".5f"),
647     "",
648     "## Top 20 biomarker candidates",
649     "",
650     markdown_table(top20, floatfmt=".5f"),
651     "",
652     "## Top 50 biomarker candidates",
653     "",
654     markdown_table(top50, floatfmt=".5f"),
655     "",
656     "## Output files",
657     "",
658     "- `results/stage4_subject_level_feature_table_dataset1.csv`",
659     "- `results/stage4_feature_audit_dataset1.csv`",
660     "- `results/stage4_univariate_statistics_dataset1.csv`",
661     "- `results/stage4_l1_stability_selection_dataset1.csv`",
662     "- `results/stage4_tree_importance_dataset1.csv`",
663     "- `results/stage4_permutation_importance_dataset1.csv`",
664     "- `results/stage4_biomarker_rank_summary_dataset1.csv`",
665     "- `results/stage4_top10_biomarkers_dataset1.csv`",
666     "- `results/stage4_top20_biomarkers_dataset1.csv`",
667     "- `results/stage4_top50_biomarkers_dataset1.csv`",
668     "- `results/figures/stage4_top20_biomarker_rank.png`",
669     "- `results/figures/stage4_top20_effect_size.png`",
670     "- `results/figures/stage4_feature_category_counts_top50.png`",
671     "- `results/figures/stage4_volcano_plot_dataset1.png`",
672     "",
673     "## Guardrails",
674     "",
675     "- The 755 recordings were not treated as independent samples for significance testing.",
676     "- Metadata and sex variables were excluded from the main acoustic feature list.",
677     "- Final ranking combines univariate, stability, permutation, and tree-based evidence rather than relying only on
    ↪ tree importance.",
678     "- The report uses associative interpretation only.",
679     "",
680     f"Univariate features with FDR q < 0.05: `{int((univariate['q_mannwhitney'] < 0.05).sum())}`",
681 ]
682 (RESULTS_DIR / "stage4_biomarker_report.md").write_text(
683     "\n".join(lines), encoding="utf-8"
684 )
685
686
687 def markdown_table(df: pd.DataFrame, floatfmt: str = ".4f") -> str:
688     headers = list(df.columns)
689     rows = []
690     for _, row in df.iterrows():
691         values = []
692         for column in headers:
693             value = row[column]
694             if isinstance(value, float) and not pd.isna(value):
695                 if column.startswith("q_"):
696                     values.append(format(value, ".3e"))
697                 else:
698                     values.append(format(value, floatfmt))
699             elif pd.isna(value):
700                 values.append("")
701             else:
702                 values.append(str(value))
703         rows.append(values)
704     table = [
705         "| " + " | ".join(headers) + " |",
706         "| " + " | ".join(["---"] * len(headers)) + " |",
707     ]
708     table.extend("| " + " | ".join(row) + " |" for row in rows)
709     return "\n".join(table)
710
711
712 def main() -> None:
713     RESULTS_DIR.mkdir(exist_ok=True)
714     FIGURES_DIR.mkdir(exist_ok=True)
715
716     df = load_dataset1()
717     features = acoustic_columns(df)
718     subject_table = build_subject_table(df, features)
719     audit = audit_subject_table(subject_table, features)
720     feature_audit = build_feature_audit_table(subject_table, features)
721
722     univariate = run_univariate_statistics(subject_table, features)
723     univariate = add_q_value_display_columns(
724         merge_feature_audit(univariate, feature_audit)
725     )
726     l1 = merge_feature_audit(run_l1_stability(subject_table, features), feature_audit)
727     l1["l1_selected_any"] = l1["selected_folds"] > 0
728     l1["l1_selected_near_constant"] = l1["l1_selected_any"] & l1["is_near_constant"]

```

```

729     tree, permutation = run_tree_and_permutation(subject_table, features)
730     tree = merge_feature_audit(tree, feature_audit)
731     permutation = merge_feature_audit(permutation, feature_audit)
732     permutation["permutation_importance_positive"] = (
733         permutation["permutation_importance_mean"] > 0
734     )
735     summary = build_rank_summary(univariate, l1, tree, permutation)
736     top20 = summary.head(20).copy()
737     top10 = summary.head(10).copy()
738     top50 = summary.head(50).copy()
739
740     write_csv(
741         subject_table,
742         RESULTS_DIR / "stage4_subject_level_feature_table_dataset1.csv",
743     )
744     write_csv(
745         feature_audit,
746         RESULTS_DIR / "stage4_feature_audit_dataset1.csv",
747     )
748     write_csv(
749         univariate,
750         RESULTS_DIR / "stage4_univariate_statistics_dataset1.csv",
751     )
752     write_csv(
753         l1,
754         RESULTS_DIR / "stage4_l1_stability_selection_dataset1.csv",
755     )
756     write_csv(
757         tree,
758         RESULTS_DIR / "stage4_tree_importance_dataset1.csv",
759     )
760     write_csv(
761         permutation,
762         RESULTS_DIR / "stage4_permutation_importance_dataset1.csv",
763     )
764     write_csv(
765         summary,
766         RESULTS_DIR / "stage4_biomarker_rank_summary_dataset1.csv",
767     )
768     write_csv(
769         top10,
770         RESULTS_DIR / "stage4_top10_biomarkers_dataset1.csv",
771     )
772     write_csv(
773         top20,
774         RESULTS_DIR / "stage4_top20_biomarkers_dataset1.csv",
775     )
776     write_csv(
777         top50,
778         RESULTS_DIR / "stage4_top50_biomarkers_dataset1.csv",
779     )
780
781     save_top20_rank_plot(summary)
782     save_top20_effect_size_plot(summary)
783     save_top50_category_plot(summary)
784     save_volcano_plot(univariate)
785     write_report(audit, summary, univariate, l1)
786
787
788 if __name__ == "__main__":
789     main()

```

A.4 代码四：二类潜在语音表型建模

```
1 from __future__ import annotations
2
3 from pathlib import Path
4 from typing import Any
5
6 import matplotlib
7
8 matplotlib.use("Agg")
9
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import pandas as pd
13 from scipy.stats import mannwhitneyu
14 from sklearn.cluster import AgglomerativeClustering, KMeans
15 from sklearn.decomposition import PCA
16 from sklearn.ensemble import RandomForestClassifier
17 from sklearn.metrics import (
18     accuracy_score,
19     adjusted_rand_score,
20     balanced_accuracy_score,
21     calinski_harabasz_score,
22     davies_bouldin_score,
23     f1_score,
24     roc_auc_score,
25     silhouette_score,
26 )
27 from sklearn.mixture import GaussianMixture
28 from sklearn.model_selection import StratifiedKFold
29 from sklearn.pipeline import Pipeline
30 from sklearn.preprocessing import StandardScaler
31 from sklearn.linear_model import LogisticRegression
32 from sklearn.svm import SVC
33
34
35 ROOT = Path(__file__).resolve().parents[1]
36 RESULTS_DIR = ROOT / "results"
37 FIGURES_DIR = RESULTS_DIR / "figures"
38
39 SUBJECT_TABLE_PATH = RESULTS_DIR / "stage4_subject_level_feature_table_dataset1.csv"
40 TOP20_PATH = RESULTS_DIR / "stage4_top20_biomarkers_dataset1.csv"
41 TOP50_PATH = RESULTS_DIR / "stage4_top50_biomarkers_dataset1.csv"
42
43 OUTPUT_CSV_ENCODING = "utf-8-sig"
44 RANDOM_STATE = 42
45 PD_LABEL = 1
46 EXPECTED_SUBJECTS = 252
47 EXPECTED_PD_SUBJECTS = 188
48 EXPECTED_ACOUSTIC_FEATURES = 752
49 STABILITY_RUNS = 50
50 STABILITY_SAMPLE_FRACTION = 0.8
51
52 METADATA_COLUMNS = {
53     "id",
54     "gender",
55     "class",
56     "sex_male",
57     "record_count",
58 }
59 LABEL_COLUMN_TOKENS = ("cluster", "label", "prediction", "phenotype")
60
61
62 def rel(path: Path) -> str:
63     return path.relative_to(ROOT).as_posix()
64
65
66 def write_csv(df: pd.DataFrame, path: Path) -> None:
67     df.to_csv(path, index=False, encoding=OUTPUT_CSV_ENCODING)
68
69
70 def require_file(path: Path) -> None:
71     if not path.exists():
72         raise FileNotFoundError(f"Missing required Stage 4 input file: {path}")
73
74
75 def load_inputs() -> tuple[pd.DataFrame, list[str], list[str]]:
76     for path in [SUBJECT_TABLE_PATH, TOP20_PATH, TOP50_PATH]:
77         require_file(path)
78
79     subject_table = pd.read_csv(SUBJECT_TABLE_PATH)
80     top20 = pd.read_csv(TOP20_PATH)
81     top50 = pd.read_csv(TOP50_PATH)
82
83     required_subject_columns = {"id", "class", "sex_male"}
84     missing_subject_columns = sorted(required_subject_columns - set(subject_table.columns))
85     if missing_subject_columns:
86         raise ValueError(f"Subject table missing required columns: {missing_subject_columns}")
87
88     if subject_table.shape[0] != EXPECTED_SUBJECTS:
89         raise ValueError(
90             f"Expected {EXPECTED_SUBJECTS} subject rows, found {subject_table.shape[0]}"
91         )
```

```

91     )
92     duplicated_id_count = int(subject_table["id"].duplicated().sum())
93     if duplicated_id_count:
94         raise ValueError(
95             f"Subject table must contain one row per subject; duplicated ids: {duplicated_id_count}"
96         )
97
98     for label, table, expected_rows in [
99         ("Top20", top20, 20),
100        ("Top50", top50, 50),
101    ]:
102        if "feature" not in table.columns:
103            raise ValueError(f"{label} input is missing the required 'feature' column")
104        if table.shape[0] != expected_rows:
105            raise ValueError(f"{label} input expected {expected_rows} rows, found {table.shape[0]}")
106
107        top20_features = top20["feature"].astype(str).tolist()
108        top50_features = top50["feature"].astype(str).tolist()
109        missing_top20 = sorted(set(top20_features) - set(subject_table.columns))
110        missing_top50 = sorted(set(top50_features) - set(subject_table.columns))
111        if missing_top20 or missing_top50:
112            raise ValueError(
113                "Stage 4 biomarker features missing from subject-level table: "
114                f"top20={missing_top20}, top50={missing_top50}"
115            )
116
117    return subject_table, top20_features, top50_features
118
119
120 def is_forbidden_feature_column(column: str) -> bool:
121     name = column.lower()
122     return column in METADATA_COLUMNS or any(token in name for token in LABEL_COLUMN_TOKENS)
123
124
125 def acoustic_columns(subject_table: pd.DataFrame) -> list[str]:
126     columns = [
127         column
128         for column in subject_table.select_dtypes(include="number").columns
129         if not is_forbidden_feature_column(column)
130     ]
131     if len(columns) != EXPECTED_ACOUSTIC_FEATURES:
132         raise ValueError(
133             f"Expected {EXPECTED_ACOUSTIC_FEATURES} all-acoustic features after metadata "
134             f"exclusion, found {len(columns)}"
135         )
136     forbidden = sorted(set(columns) & METADATA_COLUMNS)
137     if forbidden:
138         raise ValueError(f"Forbidden metadata columns entered acoustic features: {forbidden}")
139     return columns
140
141
142 def validate_pd_subjects(subject_table: pd.DataFrame) -> pd.DataFrame:
143     before_counts = subject_table["class"].value_counts().sort_index().to_dict()
144     if int(sum(before_counts.values())) != EXPECTED_SUBJECTS:
145         raise ValueError("Class distribution does not sum to the expected subject count")
146
147     pd_subjects = subject_table.loc[subject_table["class"] == PD_LABEL].copy()
148     if pd_subjects.shape[0] != EXPECTED_PD_SUBJECTS:
149         raise ValueError(
150             f"Expected {EXPECTED_PD_SUBJECTS} PD subjects, found {pd_subjects.shape[0]}"
151         )
152     return pd_subjects
153
154
155 def write_input_audit(
156     subject_table: pd.DataFrame,
157     pd_subjects: pd.DataFrame,
158     all_features: list[str],
159     top20_features: list[str],
160     top50_features: list[str],
161 ) -> None:
162     excluded_columns = [
163         column if column in subject_table.columns else f"{column} (not present)"
164         for column in ["id", "gender", "class", "sex_male", "record_count"]
165     ]
166     audit = pd.DataFrame(
167         [
168             {
169                 "input_file": rel(SUBJECT_TABLE_PATH),
170                 "total_subjects_before_filter": int(subject_table.shape[0]),
171                 "pd_subjects_after_filter": int(pd_subjects.shape[0]),
172                 "class_distribution_before_filter": str(
173                     {
174                         int(k): int(v)
175                         for k, v in subject_table["class"]
176                             .value_counts()
177                             .sort_index()
178                             .items()
179                     }
180                 ),
181                 "class_distribution_after_filter": str(
182                     {

```

```

183             int(k): int(v)
184             for k, v in pd_subjects["class"].value_counts().sort_index().items()
185         }
186     ),
187     "missing_value_count": int(subject_table.isna().sum().sum()),
188     "duplicated_id_count": int(subject_table["id"].duplicated().sum()),
189     "acoustic_feature_count": int(len(all_features)),
190     "excluded_columns": "; ".join(excluded_columns),
191     "feature_sets_used": "top20_biomarkers; top50_biomarkers; all_acoustic",
192     "top20_feature_count": int(len(top20_features)),
193     "top50_feature_count": int(len(top50_features)),
194     "all_acoustic_feature_count": int(len(all_features)),
195 }
196 ]
197 )
198 write_csv(audit, RESULTS_DIR / "stage5_pd_subject_clustering_input_audit.csv")
199
200
201 def categorize_feature(feature: str) -> str:
202     name = feature.lower()
203     if "tqwt" in name:
204         return "TQWT"
205     if any(token in name for token in ["wavelet", "_lt_", "ed_", "ed2_", "entropy"]):
206         return "Wavelet"
207     if "jitter" in name:
208         return "Jitter"
209     if "shimmer" in name or "shim_" in name or "apq" in name:
210         return "Shimmer"
211     if "hnr" in name or "harmonicity" in name or "harmtonoise" in name:
212         return "HNR / Harmonicity"
213     if "intensity" in name:
214         return "Intensity"
215     if (
216         name.startswith("f")
217         and len(name) == 2
218         and name[1].isdigit()
219         or name.startswith("b")
220         and len(name) == 2
221         and name[1].isdigit()
222     ):
223         return "Formant / Bandwidth"
224     if "mfcc" in name:
225         return "MFCC"
226     if "delta_delta" in name or "delta" in name:
227         return "Delta / Delta-delta"
228     if any(token in name for token in ["rpde", "dfa", "ppe", "gne"]):
229         return "Nonlinear"
230     return "Other"
231
232
233 def build_representation(
234     data: pd.DataFrame, representation: str
235 ) -> tuple[np.ndarray, int, float]:
236     scaled = StandardScaler().fit_transform(data.to_numpy(dtype=float))
237     if representation == "scaled_only":
238         return scaled, scaled.shape[1], np.nan
239
240     if representation == "pca_90":
241         pca = PCA(n_components=0.90, random_state=RANDOM_STATE)
242     elif representation.startswith("pca_"):
243         pca = PCA(n_components=int(representation.split("_", maxsplit=1)[1]), random_state=RANDOM_STATE)
244     else:
245         raise ValueError(f"Unknown representation: {representation}")
246
247     projected = pca.fit_transform(scaled)
248     return projected, projected.shape[1], float(np.sum(pca.explained_variance_ratio_))
249
250
251 def cluster_labels(
252     matrix: np.ndarray, method: str, random_state: int = RANDOM_STATE
253 ) -> np.ndarray:
254     if method == "KMeans":
255         return KMeans(
256             n_clusters=2,
257             n_init=50,
258             random_state=random_state,
259         ).fit_predict(matrix)
260     if method == "GaussianMixture":
261         return GaussianMixture(
262             n_components=2,
263             covariance_type="full",
264             reg_covar=1e-6,
265             random_state=random_state,
266         ).fit_predict(matrix)
267     if method == "AgglomerativeClustering":
268         return AgglomerativeClustering(n_clusters=2, linkage="ward").fit_predict(matrix)
269     raise ValueError(f"Unknown clustering method: {method}")
270
271
272 def clustering_quality(matrix: np.ndarray, labels: np.ndarray) -> tuple[float, float, float, str]:
273     unique_labels = np.unique(labels)
274     if unique_labels.size < 2:

```

```

275         return np.nan, np.nan, np.nan, "single cluster produced"
276     try:
277         silhouette = float(silhouette_score(matrix, labels))
278         calinski = float(calinski_harabasz_score(matrix, labels))
279         daves = float(davies_bouldin_score(matrix, labels))
280     except ValueError as exc:
281         return np.nan, np.nan, np.nan, f"metric calculation failed: {exc}"
282     return silhouette, calinski, daves, ""
283
284
285 def stability_analysis(
286     data: pd.DataFrame,
287     representation: str,
288     method: str,
289     full_labels: np.ndarray,
290 ) -> tuple[float, float]:
291     rng = np.random.default_rng(RANDOM_STATE)
292     n_subjects = data.shape[0]
293     sample_size = int(round(n_subjects * STABILITY_SAMPLE_FRACTION))
294     scores: list[float] = []
295
296     for run_index in range(STABILITY_RUNS):
297         sample_index = np.sort(rng.choice(n_subjects, size=sample_size, replace=False))
298         sample_data = data.iloc[sample_index]
299         sample_matrix, _, _ = build_representation(sample_data, representation)
300         sample_labels = cluster_labels(
301             sample_matrix,
302             method,
303             random_state=RANDOM_STATE + run_index + 1,
304         )
305         scores.append(float(adjusted_rand_score(full_labels[sample_index], sample_labels)))
306
307     return float(np.mean(scores)), float(np.std(scores, ddof=1))
308
309
310 def run_clustering_candidates(
311     pd_subjects: pd.DataFrame, feature_sets: dict[str, list[str]]
312 ) -> tuple[pd.DataFrame, dict[tuple[str, str, str], dict[str, Any]]]:
313     representations = {
314         "top20_biomarkers": ["scaled_only", "pca_5", "pca_10"],
315         "top50_biomarkers": ["pca_10", "pca_90"],
316         "all_acoustic": ["pca_20", "pca_90"],
317     }
318     methods = ["KMeans", "GaussianMixture", "AgglomerativeClustering"]
319
320     rows: list[dict[str, Any]] = []
321     candidate_artifacts: dict[tuple[str, str, str], dict[str, Any]] = {}
322
323     for feature_set, features in feature_sets.items():
324         data = pd_subjects[features]
325         for representation in representations[feature_set]:
326             matrix, n_components, explained_variance = build_representation(data, representation)
327             for method in methods:
328                 labels = cluster_labels(matrix, method)
329                 counts = pd.Series(labels).value_counts().reindex([0, 1], fill_value=0)
330                 cluster_0_size = int(counts.loc[0])
331                 cluster_1_size = int(counts.loc[1])
332                 smaller_cluster_size = int(min(cluster_0_size, cluster_1_size))
333                 smaller_cluster_ratio = float(smaller_cluster_size / len(labels))
334
335                 silhouette, calinski, daves, metric_warning = clustering_quality(matrix, labels)
336                 stability_mean, stability_std = stability_analysis(
337                     data,
338                     representation,
339                     method,
340                     labels,
341                 )
342
343                 warnings = []
344                 if metric_warning:
345                     warnings.append(metric_warning)
346                 if smaller_cluster_ratio < 0.10:
347                     warnings.append("smaller_cluster_ratio < 0.10; not eligible for recommendation")
348
349                 key = (feature_set, representation, method)
350                 rows.append(
351                     {
352                         "feature_set": feature_set,
353                         "representation": representation,
354                         "clustering_method": method,
355                         "n_subjects": int(len(labels)),
356                         "n_features_original": int(len(features)),
357                         "n_components_after_pca": int(n_components),
358                         "pca_explained_variance_sum": explained_variance,
359                         "cluster_0_size": cluster_0_size,
360                         "cluster_1_size": cluster_1_size,
361                         "smaller_cluster_size": smaller_cluster_size,
362                         "smaller_cluster_ratio": smaller_cluster_ratio,
363                         "silhouette": silhouette,
364                         "calinski_harabasz": calinski,
365                         "davies_bouldin": daves,
366                         "stability_ari_mean": stability_mean,

```

```

367         "stability_ari_std": stability_std,
368         "stability_n_runs": STABILITY_RUNS,
369         "recommended_flag": False,
370         "exclusion_or_warning": "; ".join(warnings),
371     }
372 )
373     candidate_artifacts[key] = {
374         "labels": labels,
375         "matrix": matrix,
376         "features": features,
377         "n_components_after_pca": n_components,
378         "pca_explained_variance_sum": explained_variance,
379     }
380
381 metrics = pd.DataFrame(rows)
382 return metrics, candidate_artifacts
383
384
385 def minmax(series: pd.Series, higher_is_better: bool = True) -> pd.Series:
386     values = series.astype(float)
387     valid = values.replace([np.inf, -np.inf], np.nan)
388     min_value = valid.min(skipna=True)
389     max_value = valid.max(skipna=True)
390     if pd.isna(min_value) or pd.isna(max_value) or max_value == min_value:
391         return pd.Series(0.5, index=series.index)
392     scaled = (valid - min_value) / (max_value - min_value)
393     if not higher_is_better:
394         scaled = 1 - scaled
395     return scaled.fillna(0.0)
396
397
398 def choose_recommended(metrics: pd.DataFrame) -> tuple[pd.DataFrame, tuple[str, str, str]]:
399     out = metrics.copy()
400     eligible = out["smaller_cluster_ratio"] >= 0.10
401     if not eligible.any():
402         raise ValueError("No clustering candidate has smaller_cluster_ratio >= 0.10")
403
404     balance = 1 - (0.5 - out["smaller_cluster_ratio"]).abs() / 0.5
405     score = (
406         0.25 * minmax(out["silhouette"], higher_is_better=True)
407         + 0.15 * minmax(out["calinski_harabasz"], higher_is_better=True)
408         + 0.15 * minmax(out["davies_bouldin"], higher_is_better=False)
409         + 0.30 * minmax(out["stability_ari_mean"], higher_is_better=True)
410         + 0.15 * balance
411     )
412     score += out["feature_set"].map(
413         {
414             "top20_biomarkers": 0.08,
415             "top50_biomarkers": 0.04,
416             "all_acoustic": -0.02,
417         }
418     )
419     score = score.where(eligible, -np.inf)
420     out["selection_score"] = score
421
422     best_index = int(out["selection_score"].idxmax())
423     best_score = float(out.loc[best_index, "selection_score"])
424
425     top20_close = out[
426         (out["feature_set"] == "top20_biomarkers")
427         & eligible
428         & (out["selection_score"] >= best_score - 0.03)
429     ]
430     if not top20_close.empty:
431         best_index = int(top20_close["selection_score"].idxmax())
432     else:
433         interpretable_close = out[
434             (out["feature_set"].isin(["top20_biomarkers", "top50_biomarkers"]))
435             & eligible
436             & (out["selection_score"] >= best_score - 0.08)
437         ]
438         if not interpretable_close.empty and out.loc[best_index, "feature_set"] == "all_acoustic":
439             best_index = int(interpretable_close["selection_score"].idxmax())
440
441     out["recommended_flag"] = False
442     out.loc[best_index, "recommended_flag"] = True
443     key = (
444         str(out.loc[best_index, "feature_set"]),
445         str(out.loc[best_index, "representation"]),
446         str(out.loc[best_index, "clustering_method"]),
447     )
448     return out, key
449
450
451 def pca_2d_coordinates(data: pd.DataFrame) -> np.ndarray:
452     scaled = StandardScaler().fit_transform(data.to_numpy(dtype=float))
453     return PCA(n_components=2, random_state=RANDOM_STATE).fit_transform(scaled)
454
455
456 def write_cluster_assignments(
457     pd_subjects: pd.DataFrame,
458     recommended_key: tuple[str, str, str],

```

```

459     recommended_artifact: dict[str, Any],
460 ) -> pd.DataFrame:
461     feature_set, representation, method = recommended_key
462     labels = np.asarray(recommended_artifact["labels"], dtype=int)
463     counts = pd.Series(labels).value_counts().to_dict()
464     coords = pca_2d_coordinates(pd_subjects[recommended_artifact["features"]])
465     assignments = pd.DataFrame(
466         {
467             "id": pd_subjects["id"].to_numpy(),
468             "class": pd_subjects["class"].to_numpy(),
469             "sex_male": pd_subjects["sex_male"].to_numpy(),
470             "cluster": labels,
471             "cluster_size": [int(counts[label]) for label in labels],
472             "feature_set": feature_set,
473             "representation": representation,
474             "clustering_method": method,
475             "pca_1": coords[:, 0],
476             "pca_2": coords[:, 1],
477         }
478     )
479     write_csv(assignments, RESULTS_DIR / "stage5_cluster_assignments_dataset1_pd.csv")
480     return assignments
481
482
483 def fdr_bh(p_values: np.ndarray) -> np.ndarray:
484     p = np.asarray(p_values, dtype=float)
485     adjusted = np.full(p.shape, np.nan, dtype=float)
486     valid = np.isfinite(p)
487     if not valid.any():
488         return adjusted
489     valid_p = p[valid]
490     order = np.argsort(valid_p)
491     ordered_p = valid_p[order]
492     n = len(ordered_p)
493     ordered_q = ordered_p * n / np.arange(1, n + 1)
494     ordered_q = np.minimum.accumulate(ordered_q[::-1])[::-1]
495     ordered_q = np.clip(ordered_q, 0, 1)
496     q = np.empty_like(ordered_q)
497     q[order] = ordered_q
498     adjusted[valid] = q
499     return adjusted
500
501
502 def cohens_d(values_1: np.ndarray, values_0: np.ndarray) -> float:
503     n_1 = len(values_1)
504     n_0 = len(values_0)
505     if n_1 < 2 or n_0 < 2:
506         return np.nan
507     var_1 = np.var(values_1, ddof=1)
508     var_0 = np.var(values_0, ddof=1)
509     pooled = ((n_1 - 1) * var_1 + (n_0 - 1) * var_0) / (n_1 + n_0 - 2)
510     if pooled <= 0:
511         return 0.0
512     return float((np.mean(values_1) - np.mean(values_0)) / np.sqrt(pooled))
513
514
515 def cluster_feature_differences(
516     pd_subjects: pd.DataFrame,
517     labels: np.ndarray,
518     features: list[str],
519 ) -> tuple[pd.DataFrame, pd.DataFrame]:
520     raw = pd_subjects[features]
521     scaled = pd.DataFrame(
522         StandardScaler().fit_transform(raw.to_numpy(dtype=float)),
523         columns=features,
524         index=raw.index,
525     )
526     rows: list[dict[str, Any]] = []
527     cluster_0_mask = labels == 0
528     cluster_1_mask = labels == 1
529
530     for feature in features:
531         raw_0 = raw.loc[cluster_0_mask, feature].to_numpy(dtype=float)
532         raw_1 = raw.loc[cluster_1_mask, feature].to_numpy(dtype=float)
533         z_0 = scaled.loc[cluster_0_mask, feature].to_numpy(dtype=float)
534         z_1 = scaled.loc[cluster_1_mask, feature].to_numpy(dtype=float)
535
536         try:
537             u_stat, p_value = mannwhitneyu(raw_0, raw_1, alternative="two-sided")
538         except ValueError:
539             u_stat, p_value = np.nan, np.nan
540
541         mean_difference = float(np.mean(raw_1) - np.mean(raw_0))
542         z_difference = float(np.mean(z_1) - np.mean(z_0))
543         d_value = cohens_d(raw_1, raw_0)
544
545         rows.append(
546             {
547                 "feature": feature,
548                 "category": categorize_feature(feature),
549                 "cluster_0_mean": float(np.mean(raw_0)),
550                 "cluster_1_mean": float(np.mean(raw_1)),

```

```

551         "mean_difference_cluster1_minus_cluster0": mean_difference,
552         "cluster_0_zmean": float(np.mean(z_0)),
553         "cluster_1_zmean": float(np.mean(z_1)),
554         "zmean_difference_cluster1_minus_cluster0": z_difference,
555         "cohens_d_cluster1_minus_cluster0": d_value,
556         "mannwhitney_u": float(u_stat),
557         "p_mannwhitney": float(p_value),
558         "abs_cohens_d": float(abs(d_value)) if np.isfinite(d_value) else np.nan,
559         "abs_zmean_difference": float(abs(z_difference)),
560         "direction": (
561             "higher_in_cluster_1"
562             if z_difference > 0
563             else "higher_in_cluster_0"
564             if z_difference < 0
565             else "no_difference"
566         ),
567     }
568 )
569
570 differences = pd.DataFrame(rows)
571 differences["q_mannwhitney"] = fdr_bh(differences["p_mannwhitney"].to_numpy())
572 ordered_columns = [
573     "feature",
574     "category",
575     "cluster_0_mean",
576     "cluster_1_mean",
577     "mean_difference_cluster1_minus_cluster0",
578     "cluster_0_zmean",
579     "cluster_1_zmean",
580     "zmean_difference_cluster1_minus_cluster0",
581     "cohens_d_cluster1_minus_cluster0",
582     "mannwhitney_u",
583     "p_mannwhitney",
584     "q_mannwhitney",
585     "abs_cohens_d",
586     "abs_zmean_difference",
587     "direction",
588 ]
589 differences = differences[ordered_columns]
590 differences = differences.sort_values(
591     ["abs_zmean_difference", "abs_cohens_d", "q_mannwhitney"],
592     ascending=[False, False, True],
593 ).reset_index(drop=True)
594 top_features = differences.head(20).copy()
595
596 write_csv(differences, RESULTS_DIR / "stage5_cluster_feature_differences.csv")
597 write_csv(top_features, RESULTS_DIR / "stage5_cluster_interpretation_top_features.csv")
598 return differences, top_features
599
600
601 def plot_pca_clusters(
602     pd_subjects: pd.DataFrame,
603     features: list[str],
604     labels: np.ndarray,
605     output_path: Path,
606     title: str,
607 ) -> None:
608     coords = pca_2d_coordinates(pd_subjects[features])
609     fig, ax = plt.subplots(figsize=(8, 6))
610     for cluster in sorted(np.unique(labels)):
611         mask = labels == cluster
612         ax.scatter(
613             coords[mask, 0],
614             coords[mask, 1],
615             s=42,
616             alpha=0.82,
617             label=f"Cluster {cluster}",
618         )
619     ax.set_title(title)
620     ax.set_xlabel("PCA 1")
621     ax.set_ylabel("PCA 2")
622     ax.legend(title="Main cluster label")
623     fig.tight_layout()
624     fig.savefig(output_path, dpi=220)
625     plt.close(fig)
626
627
628 def plot_cluster_sizes(metrics: pd.DataFrame) -> None:
629     plot_data = metrics.copy()
630     plot_data["candidate"] = (
631         plot_data["feature_set"]
632         + "\n"
633         + plot_data["representation"]
634         + "\n"
635         + plot_data["clustering_method"]
636     )
637     colors = np.where(plot_data["recommended_flag"], "#d55e00", "#4c78a8")
638     fig, ax = plt.subplots(figsize=(12, 7))
639     ax.bar(np.arange(plot_data.shape[0]), plot_data["smaller_cluster_ratio"], color=colors)
640     ax.axhline(0.10, color="#cc3311", linestyle="--", linewidth=1.2, label="0.10 threshold")
641     ax.set_xticks(np.arange(plot_data.shape[0]))
642     ax.set_xticklabels(plot_data["candidate"], rotation=75, ha="right", fontsize=7)

```

```

643     ax.set_ylabel("Smaller cluster ratio")
644     ax.set_title("Stage 5 Candidate Cluster Size Balance")
645     ax.legend()
646     fig.tight_layout()
647     fig.savefig(FIGURES_DIR / "stage5_cluster_size_comparison.png", dpi=220)
648     plt.close(fig)
649
650
651 def plot_feature_difference_top20(top_features: pd.DataFrame) -> None:
652     plot_data = top_features.sort_values("cohens_d_cluster1_minus_cluster0")
653     colors = np.where(
654         plot_data["cohens_d_cluster1_minus_cluster0"] >= 0,
655         "#2f80ed",
656         "#e07a5f",
657     )
658     fig, ax = plt.subplots(figsize=(9, 7))
659     ax.barh(
660         plot_data["feature"],
661         plot_data["cohens_d_cluster1_minus_cluster0"],
662         color=colors,
663     )
664     ax.axvline(0, color="#333333", linewidth=0.8)
665     ax.set_xlabel("Cohen's d (cluster 1 minus cluster 0)")
666     ax.set_ylabel("Feature")
667     ax.set_title("Stage 5 Top 20 Cluster-Distinguishing Voice Features")
668     fig.tight_layout()
669     fig.savefig(FIGURES_DIR / "stage5_cluster_feature_difference_top20.png", dpi=220)
670     plt.close(fig)
671
672
673 def run_cluster_label_classifier(
674     pd_subjects: pd.DataFrame,
675     features: list[str],
676     labels: np.ndarray,
677 ) -> pd.DataFrame:
678     x = pd_subjects[features].to_numpy(dtype=float)
679     y = labels.astype(int)
680     cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
681     models: dict[str, Any] = {
682         "Logistic Regression": Pipeline(
683             [
684                 ("scaler", StandardScaler()),
685                 (
686                     "model",
687                     LogisticRegression(
688                         max_iter=5000,
689                         class_weight="balanced",
690                         random_state=RANDOM_STATE,
691                     ),
692                 ),
693             ]
694         ),
695         "SVM-RBF": Pipeline(
696             [
697                 ("scaler", StandardScaler()),
698                 (
699                     "model",
700                     SVC(
701                         kernel="rbf",
702                         probability=True,
703                         class_weight="balanced",
704                         random_state=RANDOM_STATE,
705                     ),
706                 ),
707             ]
708         ),
709         "Random Forest": RandomForestClassifier(
710             n_estimators=500,
711             random_state=RANDOM_STATE,
712             class_weight="balanced",
713         ),
714     }
715
716     rows: list[dict[str, Any]] = []
717     for model_name, model in models.items():
718         fold_rows: list[dict[str, Any]] = []
719         for fold_index, (train_index, test_index) in enumerate(cv.split(x, y), start=1):
720             model.fit(x[train_index], y[train_index])
721             predictions = model.predict(x[test_index])
722             try:
723                 probabilities = model.predict_proba(x[test_index])[:, 1]
724                 auc = float(roc_auc_score(y[test_index], probabilities))
725             except (AttributeError, ValueError):
726                 auc = np.nan
727
728             fold_row = {
729                 "model": model_name,
730                 "fold": fold_index,
731                 "accuracy": float(accuracy_score(y[test_index], predictions)),
732                 "balanced_accuracy": float(
733                     balanced_accuracy_score(y[test_index], predictions)
734                 ),

```

```

735         "f1_macro": float(f1_score(y[test_index], predictions, average="macro")),
736         "f1_weighted": float(
737             f1_score(y[test_index], predictions, average="weighted")
738         ),
739         "roc_auc": auc,
740     }
741     rows.append(fold_row)
742     fold_rows.append(fold_row)
743
744     fold_metrics = pd.DataFrame(fold_rows)
745     for summary_name, reducer in [("mean", np.nanmean), ("std", np.nanstd)]:
746         rows.append(
747             {
748                 "model": model_name,
749                 "fold": summary_name,
750                 "accuracy": float(reducer(fold_metrics["accuracy"])),
751                 "balanced_accuracy": float(
752                     reducer(fold_metrics["balanced_accuracy"])
753                 ),
754                 "f1_macro": float(reducer(fold_metrics["f1_macro"])),
755                 "f1_weighted": float(reducer(fold_metrics["f1_weighted"])),
756                 "roc_auc": float(reducer(fold_metrics["roc_auc"])),
757             }
758         )
759
760     classifier_metrics = pd.DataFrame(rows)
761     write_csv(
762         classifier_metrics,
763         RESULTS_DIR / "stage5_cluster_label_classifier_metrics.csv",
764     )
765     return classifier_metrics
766
767
768 def markdown_table(df: pd.DataFrame, columns: list[str], max_rows: int | None = None) -> str:
769     table = df.loc[:, columns].copy()
770     table.columns = [str(column) for column in table.columns]
771     if max_rows is not None:
772         table = table.head(max_rows)
773     for column in table.columns:
774         if pd.api.types.is_float_dtype(table[column]):
775             table[column] = table[column].map(
776                 lambda value: "" if pd.isna(value) else f"{value:.4f}"
777             )
778     header = "| " + " | ".join(table.columns) + " |"
779     separator = "| " + " | ".join(["---" * len(table.columns)]) + " |"
780     rows = [
781         "| " + " | ".join(str(value) for value in row) + " |"
782         for row in table.to_numpy()
783     ]
784     return "\n".join([header, separator, *rows])
785
786
787 def report_selection_reason(metrics: pd.DataFrame, recommended: pd.Series) -> str:
788     high_metric = metrics.sort_values("selection_score", ascending=False).head(3)
789     return (
790         f"主推荐方案为 {recommended['feature_set']} / {recommended['representation']} / "
791         f"{recommended['clustering_method']}`。该方案最小簇比例为 "
792         f"{recommended['smaller_cluster_ratio']:.3f}，稳定性 ARI 均值为 "
793         f"{recommended['stability_ari_mean']:.3f}，在聚类质量、簇均衡性、稳定性与解释性之间取得"
794         f"较好平衡。Top20/Top50 候选优先于 all_acoustic，因为其特征来源于 Stage 4 候选生物标志物，"
795         f"更适合论文解释；all_acoustic 结果作为参考，避免把高维噪声依赖方案作为默认主结论。\\n\\n"
796         f"选择时没有只看单一指标。若某些候选方案单项 silhouette 或 Calinski-Harabasz 较高，"
797         f"但簇规模过度失衡、稳定性较弱、或依赖更高维声学空间，则不作为主推荐方案。\\n\\n"
798         f"Selection score 前三候选: \\n\\n"
799         + markdown_table(
800             high_metric,
801             [
802                 "feature_set",
803                 "representation",
804                 "clustering_method",
805                 "selection_score",
806                 "smaller_cluster_ratio",
807                 "silhouette",
808                 "stability_ari_mean",
809             ],
810         )
811     )
812
813
814 def write_report(
815     subject_table: pd.DataFrame,
816     pd_subjects: pd.DataFrame,
817     metrics: pd.DataFrame,
818     assignments: pd.DataFrame,
819     top_features: pd.DataFrame,
820     classifier_metrics: pd.DataFrame,
821 ) -> None:
822     recommended = metrics.loc[metrics["recommended_flag"]].iloc[0]
823     cluster_counts = assignments["cluster"].value_counts().sort_index()
824     cluster_ratio = (cluster_counts / cluster_counts.sum()).round(4)
825     sex_table = pd.crosstab(assignments["cluster"], assignments["sex_male"], margins=True)
826     category_counts = top_features["category"].value_counts().reset_index()

```

```

827 category_counts.columns = ["category", "top_feature_count"]
828 classifier_summary = classifier_metrics[classifier_metrics["fold"].isin(["mean", "std"])]
829
830 required_boundary_text = (
831     " 由于附件数据未提供运动型与非运动型的真实临床标签，本文不直接构建监督分类模型，"
832     " 而是在帕金森病受试者内部进行无监督二类聚类，以获得基于语音特征的潜在分型。"
833     " 随后训练的分类器预测的是聚类标签，而非临床确诊标签。因此，该模型只能作为"
834     " 运动型/非运动型辅助诊断的探索性参考，仍需真实临床症状标签进一步验证。"
835 )
836
837 report = [
838     "# Stage 5 Dataset 1 PD Two-Cluster Voice Phenotyping",
839     "",
840     "## 1. Stage 5 目标与边界",
841     "本阶段只在 Dataset 1 的 PD 受试者内部进行 k=2 无监督聚类，目标是得到基于语音特征的二类潜在表型，并训练一个预测聚类标签的"
842     "↪ cluster-label classifier。",
843     "所有结论均为探索性辅助分型，不是临床确诊结果。",
844     "",
845     "## 2. 为什么不能做真实运动型/非运动型监督诊断",
846     required_boundary_text,
847     "",
848     "## 3. 输入文件与样本审计",
849     f"- 输入 subject-level 文件: `{rel(SUBJECT_TABLE_PATH)}`",
850     f"- 总受试者数: `{subject_table.shape[0]}`",
851     f"- PD 受试者数: `{pd_subjects.shape[0]}`",
852     f"- class 分布: `{subject_table['class'].value_counts().sort_index().to_dict()}`",
853     f"- 缺失值总数: `{int(subject_table.isna().sum().sum())}`",
854     f"- duplicated id 数: `{int(subject_table['id'].duplicated().sum())}`",
855     "",
856     "## 4. 特征集与预处理",
857     "使用 Stage 4 的 Top20、Top50 与 all_acoustic 三套特征集。所有候选方案先执行 StandardScaler; PCA 表示均在标准化之后生成。"
858     "↪ ",
859     "`id`, `gender`, `class`, `sex_male`, `record_count` 及任何标签或后续生成列均不进入聚类输入。`sex_male` 仅用于事后描述。"
860     "↪ ",
861     "",
862     "## 5. 聚类候选方案完整指标表",
863     markdown_table(
864         metrics,
865         [
866             "feature_set",
867             "representation",
868             "clustering_method",
869             "cluster_0_size",
870             "cluster_1_size",
871             "smaller_cluster_ratio",
872             "silhouette",
873             "calinski_harabasz",
874             "davies_bouldin",
875             "stability_ari_mean",
876             "recommended_flag",
877         ],
878     ),
879     "",
880     "## 6. 稳定性分析结果",
881     f"每个候选方案执行 `{STABILITY_RUNS}` 次 80% 受试者重采样，重新标准化、PCA 与聚类，并在可比较样本上计算 Adjusted Rand"
882     "↪ Index。",
883     "稳定性均值和标准差已写入 `results/stage5_clustering_metrics.csv`。",
884     "",
885     "## 7. 主推荐方案选择理由",
886     report_selection_reason(metrics, recommended),
887     "",
888     "## 8. 主方案两簇人数与比例",
889     markdown_table(
890         pd.DataFrame(
891             {
892                 "cluster": cluster_counts.index,
893                 "n_subjects": cluster_counts.values,
894                 "ratio": cluster_ratio.values,
895             }
896         ),
897         ["cluster", "n_subjects", "ratio"],
898     ),
899     "",
900     "## 9. 两簇 Top 区分语音特征",
901     markdown_table(
902         top_features,
903         [
904             "feature",
905             "category",
906             "zmean_difference_cluster1_minus_cluster0",
907             "cohens_d_cluster1_minus_cluster0",
908             "q_mannwhitney",
909             "direction",
910         ],
911     ),
912     "",
913     "## 10. 特征类别分布与解释",
914     markdown_table(category_counts, ["category", "top_feature_count"]),
915     "Top 区分特征类别分布用于描述两个潜在语音表型的声学差异来源，不构成临床病因解释。",
916     "",
917     "## 11. 性别分布的事后描述",
918     markdown_table(sex_table.reset_index(), list(sex_table.reset_index().columns)),

```

```

915     "`sex_male` 未进入聚类或 cluster-label classifier, 只作为事后描述, 不能作因果解释。",
916     "",
917     "### 12. cluster-label classifier 结果",
918     markdown_table(
919         classifier_summary,
920         [
921             "model",
922             "fold",
923             "accuracy",
924             "balanced_accuracy",
925             "f1_macro",
926             "f1_weighted",
927             "roc_auc",
928         ],
929     ),
930     "该分类器学习的是无监督聚类标签, 用于将潜在语音表型分型规则模型化; 它不是训练自真实临床运动型/非运动型标签,
    ↳ 因此不能解释为真实临床症状分类性能。",
931     "",
932     "### 13. 局限性",
933     "聚类标签来自语音特征内部结构, 没有外部临床症状标签验证。",
934     "Mann-Whitney U 与 FDR 是聚类后的描述性比较, 不是外部验证假设检验, 也不是临床显著性证据。",
935     "cluster 编号本身没有临床含义; 任何 potential motor-like voice phenotype 或 potential nonmotor-like voice phenotype
    ↳ 命名都只是解释性命名。",
936     "all_acoustic 高维结果可能受冗余与噪声影响, 因此不优先作为论文主解释方案。",
937     "",
938     "### 14. 论文建议表述",
939     "建议写作: 本文在 PD 受试者内部基于 Stage 4 筛选的语音特征进行 k=2 无监督聚类, 得到两个基于语音特征的潜在表型簇,
    ↳ 并进一步训练 cluster-label classifier 以复现该探索性分型规则。",
940     "不建议写作: 将本阶段结果表述为真实临床亚型诊断性能。",
941     "",
942     "## 输出文件",
943     "- `results/stage5_pd_subject_clustering_input_audit.csv`",
944     "- `results/stage5_clustering_metrics.csv`",
945     "- `results/stage5_cluster_assignments_dataset1_pd.csv`",
946     "- `results/stage5_cluster_feature_differences.csv`",
947     "- `results/stage5_cluster_interpretation_top_features.csv`",
948     "- `results/stage5_cluster_label_classifier_metrics.csv`",
949     "- `results/stage5_two_cluster_report.md`",
950     "- `results/figures/stage5_pca_k2_clusters_top20.png`",
951     "- `results/figures/stage5_pca_k2_clusters_top50.png`",
952     "- `results/figures/stage5_cluster_size_comparison.png`",
953     "- `results/figures/stage5_cluster_feature_difference_top20.png`",
954 ]
955 (RESULTS_DIR / "stage5_two_cluster_report.md").write_text(
956     "\n".join(report),
957     encoding="utf-8",
958 )
959
960
961 def main() -> None:
962     RESULTS_DIR.mkdir(exist_ok=True)
963     FIGURES_DIR.mkdir(exist_ok=True)
964
965     subject_table, top20_features, top50_features = load_inputs()
966     all_features = acoustic_columns(subject_table)
967     pd_subjects = validate_pd_subjects(subject_table)
968     write_input_audit(
969         subject_table,
970         pd_subjects,
971         all_features,
972         top20_features,
973         top50_features,
974     )
975
976     feature_sets = {
977         "top20_biomarkers": top20_features,
978         "top50_biomarkers": top50_features,
979         "all_acoustic": all_features,
980     }
981     metrics, artifacts = run_clustering_candidates(pd_subjects, feature_sets)
982     metrics, recommended_key = choose_recommended(metrics)
983     write_csv(metrics, RESULTS_DIR / "stage5_clustering_metrics.csv")
984
985     recommended_artifact = artifacts[recommended_key]
986     labels = np.asarray(recommended_artifact["labels"], dtype=int)
987     assignments = write_cluster_assignments(
988         pd_subjects,
989         recommended_key,
990         recommended_artifact,
991     )
992
993     differences, top_features = cluster_feature_differences(
994         pd_subjects,
995         labels,
996         recommended_artifact["features"],
997     )
998     classifier_metrics = run_cluster_label_classifier(
999         pd_subjects,
1000         recommended_artifact["features"],
1001         labels,
1002     )
1003

```

```

1004     plot_pca_clusters(
1005         pd_subjects,
1006         top20_features,
1007         labels,
1008         FIGURES_DIR / "stage5_pca_k2_clusters_top20.png",
1009         "Stage 5 Main Cluster Labels Projected on Top20 Biomarker PCA",
1010     )
1011     plot_pca_clusters(
1012         pd_subjects,
1013         top50_features,
1014         labels,
1015         FIGURES_DIR / "stage5_pca_k2_clusters_top50.png",
1016         "Stage 5 Main Cluster Labels Projected on Top50 Biomarker PCA",
1017     )
1018     plot_cluster_sizes(metrics)
1019     plot_feature_difference_top20(top_features)
1020
1021     write_report(
1022         subject_table,
1023         pd_subjects,
1024         metrics,
1025         assignments,
1026         top_features,
1027         classifier_metrics,
1028     )
1029
1030
1031 if __name__ == "__main__":
1032     main()

```

A.5 代码五：六类潜在语音表型建模

```
1 from __future__ import annotations
2
3 from pathlib import Path
4 from typing import Any
5
6 import matplotlib
7
8 matplotlib.use("Agg")
9
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import pandas as pd
13 from scipy.stats import kruskal
14 from sklearn.cluster import AgglomerativeClustering, KMeans
15 from sklearn.decomposition import PCA
16 from sklearn.ensemble import RandomForestClassifier
17 from sklearn.linear_model import LogisticRegression
18 from sklearn.metrics import (
19     accuracy_score,
20     adjusted_rand_score,
21     balanced_accuracy_score,
22     calinski_harabasz_score,
23     confusion_matrix,
24     davies_bouldin_score,
25     f1_score,
26     normalized_mutual_info_score,
27     precision_score,
28     recall_score,
29     roc_auc_score,
30     silhouette_score,
31 )
32 from sklearn.mixture import GaussianMixture
33 from sklearn.model_selection import StratifiedKFold
34 from sklearn.pipeline import Pipeline
35 from sklearn.preprocessing import StandardScaler
36 from sklearn.svm import SVC
37
38
39 ROOT = Path(__file__).resolve().parents[1]
40 RESULTS_DIR = ROOT / "results"
41 FIGURES_DIR = RESULTS_DIR / "figures"
42
43 SUBJECT_TABLE_PATH = RESULTS_DIR / "stage4_subject_level_feature_table_dataset1.csv"
44 TOP20_PATH = RESULTS_DIR / "stage4_top20_biomarkers_dataset1.csv"
45 TOP50_PATH = RESULTS_DIR / "stage4_top50_biomarkers_dataset1.csv"
46 RANK_SUMMARY_PATH = RESULTS_DIR / "stage4_biomarker_rank_summary_dataset1.csv"
47 STAGE5_ASSIGNMENTS_PATH = RESULTS_DIR / "stage5_cluster_assignments_dataset1_pd.csv"
48 STAGE5_DIFFERENCES_PATH = RESULTS_DIR / "stage5_cluster_feature_differences.csv"
49
50 OUTPUT_CSV_ENCODING = "utf-8-sig"
51 RANDOM_STATE = 42
52 EXPECTED_SUBJECTS = 252
53 EXPECTED_PD_SUBJECTS = 188
54 EXPECTED_ACOUSTIC_FEATURES = 752
55 PD_LABEL = 1
56 MAIN_K = 6
57 STABILITY_RUNS = 50
58 STABILITY_SAMPLE_FRACTION = 0.8
59
60 METADATA_COLUMNS = {"id", "gender", "class", "sex_male", "record_count"}
61 LABEL_COLUMN_TOKENS = ("cluster", "label", "prediction", "phenotype")
62 METHODS = ["KMeans", "GaussianMixture", "AgglomerativeClustering"]
63 FEATURE_SET_REPRESENTATIONS = {
64     "top20_biomarkers": ["scaled_only", "pca_5", "pca_10"],
65     "top50_biomarkers": ["pca_10", "pca_90"],
66     "all_acoustic": ["pca_20", "pca_90"],
67 }
68 SAFE_LABEL_SUFFIXES = ["A", "B", "C", "D", "E", "F"]
69
70
71 def rel(path: Path) -> str:
72     return path.relative_to(ROOT).as_posix()
73
74
75 def write_csv(df: pd.DataFrame, path: Path) -> None:
76     df.to_csv(path, index=False, encoding=OUTPUT_CSV_ENCODING)
77
78
79 def require_file(path: Path) -> None:
80     if not path.exists():
81         raise FileNotFoundError(f"Missing required input file: {path}")
82
83
84 def is_forbidden_feature_column(column: str) -> bool:
85     name = column.lower()
86     return column in METADATA_COLUMNS or any(token in name for token in LABEL_COLUMN_TOKENS)
87
88
89 def categorize_feature(feature: str) -> str:
90     name = feature.lower()
```

```

91     if "tqwt" in name:
92         return "TQWT"
93     if any(token in name for token in ["wavelet", "_lt_", "ed_", "ed2_", "entropy"]):
94         return "Wavelet"
95     if "jitter" in name:
96         return "Jitter"
97     if "shimmer" in name or "shim_" in name or "apq" in name:
98         return "Shimmer"
99     if "hnr" in name or "harmonicity" in name or "harmtonoise" in name:
100         return "HNR / Harmonicity"
101     if "intensity" in name:
102         return "Intensity"
103     if (
104         name.startswith("f")
105         and len(name) == 2
106         and name[1].isdigit()
107         or name.startswith("b")
108         and len(name) == 2
109         and name[1].isdigit()
110     ):
111         return "Formant / Bandwidth"
112     if "mfcc" in name:
113         return "MFCC"
114     if "delta_delta" in name or "delta" in name:
115         return "Delta / Delta-delta"
116     if any(token in name for token in ["rpde", "dfa", "ppe", "gne"]):
117         return "Nonlinear"
118     return "Other"
119
120
121 def load_inputs() -> tuple[pd.DataFrame, list[str], list[str], pd.DataFrame, pd.DataFrame, pd.DataFrame]:
122     required_paths = [
123         SUBJECT_TABLE_PATH,
124         TOP20_PATH,
125         TOP50_PATH,
126         RANK_SUMMARY_PATH,
127         STAGE5_ASSIGNMENTS_PATH,
128         STAGE5_DIFFERENCES_PATH,
129     ]
130     for path in required_paths:
131         require_file(path)
132
133     subject_table = pd.read_csv(SUBJECT_TABLE_PATH)
134     top20 = pd.read_csv(TOP20_PATH)
135     top50 = pd.read_csv(TOP50_PATH)
136     rank_summary = pd.read_csv(RANK_SUMMARY_PATH)
137     stage5_assignments = pd.read_csv(STAGE5_ASSIGNMENTS_PATH)
138     stage5_differences = pd.read_csv(STAGE5_DIFFERENCES_PATH)
139
140     required_subject_columns = {"id", "class", "sex_male"}
141     missing = sorted(required_subject_columns - set(subject_table.columns))
142     if missing:
143         raise ValueError(f"Subject-level table missing required columns: {missing}")
144     if subject_table.shape[0] != EXPECTED_SUBJECTS:
145         raise ValueError(f"Expected {EXPECTED_SUBJECTS} subjects, found {subject_table.shape[0]}")
146     duplicated_id_count = int(subject_table["id"].duplicated().sum())
147     if duplicated_id_count:
148         raise ValueError(f"Subject-level table must have one row per subject; duplicated ids={duplicated_id_count}")
149
150     for label, table, expected_rows in [("Top20", top20, 20), ("Top50", top50, 50)]:
151         if "feature" not in table.columns:
152             raise ValueError(f"{label} file is missing the required 'feature' column")
153         if table.shape[0] != expected_rows:
154             raise ValueError(f"{label} expected {expected_rows} rows, found {table.shape[0]}")
155
156     required_stage5_columns = {"id", "cluster"}
157     missing_stage5 = sorted(required_stage5_columns - set(stage5_assignments.columns))
158     if missing_stage5:
159         raise ValueError(f"Stage 5 assignments missing required columns: {missing_stage5}")
160
161     top20_features = top20["feature"].astype(str).tolist()
162     top50_features = top50["feature"].astype(str).tolist()
163     missing_top20 = sorted(set(top20_features) - set(subject_table.columns))
164     missing_top50 = sorted(set(top50_features) - set(subject_table.columns))
165     if missing_top20 or missing_top50:
166         raise ValueError(
167             "Stage 4 biomarker features missing from subject-level table: "
168             f"top20={missing_top20}, top50={missing_top50}"
169         )
170
171     return subject_table, top20_features, top50_features, rank_summary, stage5_assignments, stage5_differences
172
173
174 def acoustic_columns(subject_table: pd.DataFrame) -> list[str]:
175     columns = [
176         column
177         for column in subject_table.select_dtypes(include="number").columns
178         if not is_forbidden_feature_column(column)
179     ]
180     if len(columns) != EXPECTED_ACOUSTIC_FEATURES:
181         raise ValueError(f"Expected {EXPECTED_ACOUSTIC_FEATURES} acoustic features, found {len(columns)}")
182     return columns

```

```

183
184
185 def validate_pd_subjects(subject_table: pd.DataFrame, stage5_assignments: pd.DataFrame) -> pd.DataFrame:
186     pd_subjects = subject_table.loc[subject_table["class"] == PD_LABEL].copy()
187     if pd_subjects.shape[0] != EXPECTED_PD_SUBJECTS:
188         raise ValueError(f"Expected {EXPECTED_PD_SUBJECTS} PD subjects, found {pd_subjects.shape[0]}")
189     overlap = set(pd_subjects["id"]).intersection(set(stage5_assignments["id"]))
190     if len(overlap) != EXPECTED_PD_SUBJECTS:
191         raise ValueError(f"Stage 5 assignments overlap {len(overlap)} PD subjects, expected {EXPECTED_PD_SUBJECTS}")
192     return pd_subjects
193
194
195 def write_input_audit(
196     subject_table: pd.DataFrame,
197     pd_subjects: pd.DataFrame,
198     all_features: list[str],
199     top20_features: list[str],
200     top50_features: list[str],
201     stage5_assignments: pd.DataFrame,
202 ) -> None:
203     excluded_columns = [
204         column if column in subject_table.columns else f"{column} (not present)"
205         for column in ["id", "gender", "class", "sex_male", "record_count"]
206     ]
207     overlap = len(set(pd_subjects["id"]).intersection(set(stage5_assignments["id"])))
208     audit = pd.DataFrame(
209         [
210             {
211                 "input_file": rel(SUBJECT_TABLE_PATH),
212                 "total_subjects_before_filter": int(subject_table.shape[0]),
213                 "pd_subjects_after_filter": int(pd_subjects.shape[0]),
214                 "class_distribution_before_filter": str(
215                     {int(k): int(v) for k, v in subject_table["class"].value_counts().sort_index().items()}
216                 ),
217                 "class_distribution_after_filter": str(
218                     {int(k): int(v) for k, v in pd_subjects["class"].value_counts().sort_index().items()}
219                 ),
220                 "missing_value_count": int(subject_table.isna().sum().sum()),
221                 "duplicated_id_count": int(subject_table["id"].duplicated().sum()),
222                 "acoustic_feature_count": int(len(all_features)),
223                 "excluded_columns": "; ".join(excluded_columns + ["stage5_cluster", "stage6_cluster"]),
224                 "feature_sets_used": "top20_biomarkers; top50_biomarkers; all_acoustic",
225                 "top20_feature_count": int(len(top20_features)),
226                 "top50_feature_count": int(len(top50_features)),
227                 "all_acoustic_feature_count": int(len(all_features)),
228                 "stage5_assignment_file_used": rel(STAGE5_ASSIGNMENTS_PATH),
229                 "stage5_label_overlap_subject_count": int(overlap),
230             }
231         ]
232     )
233     write_csv(audit, RESULTS_DIR / "stage6_six_cluster_input_audit.csv")
234
235
236 def build_representation(data: pd.DataFrame, representation: str) -> tuple[np.ndarray, int, float]:
237     scaled = StandardScaler().fit_transform(data.to_numpy(dtype=float))
238     if representation == "scaled_only":
239         return scaled, scaled.shape[1], np.nan
240     if representation == "pca_90":
241         pca = PCA(n_components=0.90, random_state=RANDOM_STATE)
242     elif representation.startswith("pca_"):
243         pca = PCA(n_components=int(representation.split("_", maxsplit=1)[1]), random_state=RANDOM_STATE)
244     else:
245         raise ValueError(f"Unknown representation: {representation}")
246     projected = pca.fit_transform(scaled)
247     return projected, projected.shape[1], float(np.sum(pca.explained_variance_ratio_))
248
249
250 def cluster_labels(matrix: np.ndarray, method: str, k: int, random_state: int = RANDOM_STATE) -> np.ndarray:
251     if method == "KMeans":
252         return KMeans(n_clusters=k, n_init=100, random_state=random_state).fit_predict(matrix)
253     if method == "GaussianMixture":
254         return GaussianMixture(
255             n_components=k,
256             covariance_type="full",
257             reg_covar=1e-6,
258             random_state=random_state,
259         ).fit_predict(matrix)
260     if method == "AgglomerativeClustering":
261         return AgglomerativeClustering(n_clusters=k, linkage="ward").fit_predict(matrix)
262     raise ValueError(f"Unknown clustering method: {method}")
263
264
265 def clustering_quality(matrix: np.ndarray, labels: np.ndarray) -> tuple[float, float, float, str]:
266     if np.unique(labels).size < 2:
267         return np.nan, np.nan, np.nan, "single cluster produced"
268     try:
269         return (
270             float(silhouette_score(matrix, labels)),
271             float(calinski_harabasz_score(matrix, labels)),
272             float(davies_bouldin_score(matrix, labels)),
273             "",
274         )

```

```

275     except ValueError as exc:
276         return np.nan, np.nan, np.nan, f"metric calculation failed: {exc}"
277
278
279 def stability_analysis(
280     data: pd.DataFrame,
281     representation: str,
282     method: str,
283     k: int,
284     full_labels: np.ndarray,
285     n_runs: int = STABILITY_RUNS,
286 ) -> tuple[float, float]:
287     rng = np.random.default_rng(RANDOM_STATE + k)
288     n_subjects = data.shape[0]
289     sample_size = int(round(n_subjects * STABILITY_SAMPLE_FRACTION))
290     scores: list[float] = []
291     for run_index in range(n_runs):
292         sample_index = np.sort(rng.choice(n_subjects, size=sample_size, replace=False))
293         sample_matrix, _, _ = build_representation(data.iloc[sample_index], representation)
294         sample_labels = cluster_labels(sample_matrix, method, k, random_state=RANDOM_STATE + run_index + 1)
295         scores.append(float(adjusted_rand_score(full_labels[sample_index], sample_labels)))
296     return float(np.mean(scores)), float(np.std(scores, ddof=1))
297
298
299 def cluster_size_stats(labels: np.ndarray, k: int) -> dict[str, Any]:
300     counts = pd.Series(labels).value_counts().reindex(range(k), fill_value=0).astype(int)
301     min_size = int(counts.min())
302     max_size = int(counts.max())
303     min_ratio = float(min_size / len(labels))
304     max_ratio = float(max_size / len(labels))
305     return {
306         "cluster_sizes": ";".join(f"{cluster}:{int(size)}" for cluster, size in counts.items()),
307         "min_cluster_size": min_size,
308         "max_cluster_size": max_size,
309         "min_cluster_ratio": min_ratio,
310         "max_cluster_ratio": max_ratio,
311         "size_imbalance_ratio": float(max_size / min_size) if min_size > 0 else np.inf,
312     }
313
314
315 def run_clustering_candidates(
316     pd_subjects: pd.DataFrame,
317     feature_sets: dict[str, list[str]],
318     k: int = MAIN_K,
319 ) -> tuple[pd.DataFrame, dict[tuple[str, str, str, int], dict[str, Any]]]:
320     rows: list[dict[str, Any]] = []
321     artifacts: dict[tuple[str, str, str, int], dict[str, Any]] = {}
322
323     for feature_set, features in feature_sets.items():
324         data = pd_subjects[features]
325         for representation in FEATURE_SET_REPRESENTATIONS[feature_set]:
326             matrix, n_components, explained_variance = build_representation(data, representation)
327             for method in METHODS:
328                 labels = cluster_labels(matrix, method, k)
329                 size_stats = cluster_size_stats(labels, k)
330                 silhouette, calinski, davies, metric_warning = clustering_quality(matrix, labels)
331                 stability_mean, stability_std = stability_analysis(data, representation, method, k, labels)
332
333                 warnings = []
334                 if metric_warning:
335                     warnings.append(metric_warning)
336                 if size_stats["min_cluster_size"] < 8:
337                     warnings.append("min_cluster_size < 8; not eligible for recommendation")
338                 if size_stats["min_cluster_ratio"] < 0.05:
339                     warnings.append("min_cluster_ratio < 0.05; not eligible for recommendation")
340
341                 key = (feature_set, representation, method, k)
342                 rows.append(
343                     {
344                         "feature_set": feature_set,
345                         "representation": representation,
346                         "clustering_method": method,
347                         "k": int(k),
348                         "n_subjects": int(len(labels)),
349                         "n_features_original": int(len(features)),
350                         "n_components_after_pca": int(n_components),
351                         "pca_explained_variance_sum": explained_variance,
352                         **size_stats,
353                         "silhouette": silhouette,
354                         "calinski_harabasz": calinski,
355                         "davies_bouldin": davies,
356                         "stability_ari_mean": stability_mean,
357                         "stability_ari_std": stability_std,
358                         "stability_n_runs": STABILITY_RUNS,
359                         "recommended_flag": False,
360                         "exclusion_or_warning": "; ".join(warnings),
361                     }
362                 )
363                 artifacts[key] = {
364                     "labels": labels,
365                     "matrix": matrix,
366                     "features": features,

```

```

367         "n_components_after_pca": n_components,
368         "pca_explained_variance_sum": explained_variance,
369     }
370     return pd.DataFrame(rows), artifacts
371
372
373 def minmax(series: pd.Series, higher_is_better: bool = True) -> pd.Series:
374     valid = series.astype(float).replace([np.inf, -np.inf], np.nan)
375     min_value = valid.min(skipna=True)
376     max_value = valid.max(skipna=True)
377     if pd.isna(min_value) or pd.isna(max_value) or max_value == min_value:
378         return pd.Series(0.5, index=series.index)
379     scaled = (valid - min_value) / (max_value - min_value)
380     if not higher_is_better:
381         scaled = 1 - scaled
382     return scaled.fillna(0.0)
383
384
385 def stage5_alignment_score(labels: np.ndarray, stage5_labels: np.ndarray) -> float:
386     nmi = normalized_mutual_info_score(stage5_labels, labels)
387     return float(nmi)
388
389
390 def choose_recommended(
391     metrics: pd.DataFrame,
392     artifacts: dict[tuple[str, str, str, int], dict[str, Any]],
393     stage5_labels: np.ndarray,
394 ) -> tuple[pd.DataFrame, tuple[str, str, str, int]]:
395     out = metrics.copy()
396     alignments = []
397     for _, row in out.iterrows():
398         key = (row["feature_set"], row["representation"], row["clustering_method"], int(row["k"]))
399         alignments.append(stage5_alignment_score(artifacts[key]["labels"], stage5_labels))
400     out["stage5_nmi"] = alignments
401
402     eligible = (out["min_cluster_size"] >= 8) & (out["min_cluster_ratio"] >= 0.05)
403     if not eligible.any():
404         raise ValueError("No k=6 candidate is eligible after min cluster constraints")
405
406     balance = 1 - (out["max_cluster_ratio"] - out["min_cluster_ratio"]).clip(lower=0)
407     score = (
408         0.20 * minmax(out["silhouette"], True)
409         + 0.12 * minmax(out["calinski_harabasz"], True)
410         + 0.13 * minmax(out["davies_bouldin"], False)
411         + 0.30 * minmax(out["stability_ari_mean"], True)
412         + 0.15 * balance
413         + 0.10 * minmax(out["stage5_nmi"], True)
414     )
415     score += out["feature_set"].map(
416         {"top20_biomarkers": 0.04, "top50_biomarkers": 0.08, "all_acoustic": -0.03}
417     )
418     score = score.where(eligible, -np.inf)
419     out["selection_score"] = score
420     best_index = int(out["selection_score"].idxmax())
421     best_score = float(out.loc[best_index, "selection_score"])
422
423     interpretable_close = out[
424         (out["feature_set"].isin(["top20_biomarkers", "top50_biomarkers"]))
425         & eligible
426         & (out["selection_score"] >= best_score - 0.08)
427     ]
428     if not interpretable_close.empty and out.loc[best_index, "feature_set"] == "all_acoustic":
429         best_index = int(interpretable_close["selection_score"].idxmax())
430
431     top50_close = out[
432         (out["feature_set"] == "top50_biomarkers")
433         & eligible
434         & (out["selection_score"] >= float(out.loc[best_index, "selection_score"]) - 0.03)
435     ]
436     if not top50_close.empty and out.loc[best_index, "feature_set"] == "top20_biomarkers":
437         best_index = int(top50_close["selection_score"].idxmax())
438
439     out["recommended_flag"] = False
440     out.loc[best_index, "recommended_flag"] = True
441     key = (
442         str(out.loc[best_index, "feature_set"]),
443         str(out.loc[best_index, "representation"]),
444         str(out.loc[best_index, "clustering_method"]),
445         int(out.loc[best_index, "k"]),
446     )
447     return out, key
448
449
450 def run_k_scan(
451     pd_subjects: pd.DataFrame,
452     features: list[str],
453     feature_set: str,
454     representation: str,
455 ) -> pd.DataFrame:
456     rows: list[dict[str, Any]] = []
457     data = pd_subjects[features]
458     for k in range(2, 9):

```

```

459     matrix, n_components, explained_variance = build_representation(data, representation)
460     for method in METHODS:
461         labels = cluster_labels(matrix, method, k)
462         size_stats = cluster_size_stats(labels, k)
463         silhouette, calinski, davies, metric_warning = clustering_quality(matrix, labels)
464         stability_mean, stability_std = stability_analysis(data, representation, method, k, labels)
465         rows.append(
466             {
467                 "feature_set": feature_set,
468                 "representation": representation,
469                 "clustering_method": method,
470                 "k": int(k),
471                 "n_subjects": int(len(labels)),
472                 "n_features_original": int(len(features)),
473                 "n_components_after_pca": int(n_components),
474                 "pca_explained_variance_sum": explained_variance,
475                 **size_stats,
476                 "silhouette": silhouette,
477                 "calinski_harabasz": calinski,
478                 "davies_bouldin": davies,
479                 "stability_ari_mean": stability_mean,
480                 "stability_ari_std": stability_std,
481                 "stability_n_runs": STABILITY_RUNS,
482                 "exclusion_or_warning": metric_warning,
483             }
484         )
485     k_scan = pd.DataFrame(rows)
486     write_csv(k_scan, RESULTS_DIR / "stage6_k_scan_metrics.csv")
487     return k_scan
488
489 def consensus_kmeans_summary(data: pd.DataFrame, representation: str, labels: np.ndarray) -> dict[str, float]:
490     n_subjects = data.shape[0]
491     coassociation = np.zeros((n_subjects, n_subjects), dtype=float)
492     coobserved = np.zeros((n_subjects, n_subjects), dtype=float)
493     rng = np.random.default_rng(RANDOM_STATE + 600)
494     sample_size = int(round(n_subjects * STABILITY_SAMPLE_FRACTION))
495     for run_index in range(STABILITY_RUNS):
496         sample_index = np.sort(rng.choice(n_subjects, size=sample_size, replace=False))
497         sample_matrix, _, _ = build_representation(data.iloc[sample_index], representation)
498         sample_labels = cluster_labels(sample_matrix, "KMeans", MAIN_K, random_state=RANDOM_STATE + run_index + 1)
499         for i_pos, i_subject in enumerate(sample_index):
500             same = sample_labels == labels[sample_index[i_pos]]
501             coassociation[i_subject, sample_index[same]] += 1
502             coobserved[i_subject, sample_index[same]] += 1
503     with np.errstate(divide="ignore", invalid="ignore"):
504         consensus = np.divide(coassociation, coobserved, out=np.full_like(coassociation, np.nan), where=coobserved > 0)
505     same_mask = labels[:, None] == labels[None, :]
506     diff_mask = labels[:, None] != labels[None, :]
507     return {
508         "consensus_within_cluster_mean": float(np.nanmean(consensus[same_mask])),
509         "consensus_between_cluster_mean": float(np.nanmean(consensus[diff_mask])),
510         "consensus_separation": float(np.nanmean(consensus[same_mask]) - np.nanmean(consensus[diff_mask])),
511     }
512
513 def pca_2d_coordinates(data: pd.DataFrame) -> np.ndarray:
514     scaled = StandardScaler().fit_transform(data.to_numpy(dtype=float))
515     return PCA(n_components=2, random_state=RANDOM_STATE).fit_transform(scaled)
516
517 def aligned_stage5(stage5_assignments: pd.DataFrame, pd_subjects: pd.DataFrame) -> pd.Series:
518     stage5 = stage5_assignments[["id", "cluster"]].rename(columns={"cluster": "stage5_cluster"})
519     merged = pd_subjects[["id"]].merge(stage5, on="id", how="left")
520     if merged["stage5_cluster"].isna().any():
521         raise ValueError("Missing Stage 5 cluster labels after aligning to PD subject order")
522     return merged["stage5_cluster"].astype(int)
523
524 def write_assignments(
525     pd_subjects: pd.DataFrame,
526     stage5_labels: pd.Series,
527     recommended_key: tuple[str, str, str, int],
528     artifact: dict[str, Any],
529 ) -> pd.DataFrame:
530     feature_set, representation, method, _ = recommended_key
531     labels = np.asarray(artifact["labels"], dtype=int)
532     counts = pd.Series(labels).value_counts().to_dict()
533     coords = pca_2d_coordinates(pd_subjects[artifact["features"]])
534     assignments = pd.DataFrame(
535         {
536             "id": pd_subjects["id"].to_numpy(),
537             "class": pd_subjects["class"].to_numpy(),
538             "sex_male": pd_subjects["sex_male"].to_numpy(),
539             "stage5_cluster": stage5_labels.to_numpy(),
540             "stage6_cluster": labels,
541             "stage6_cluster_size": [int(counts[label]) for label in labels],
542             "stage6_cluster_ratio": [float(counts[label] / len(labels)) for label in labels],
543             "feature_set": feature_set,
544             "representation": representation,
545             "clustering_method": method,
546             "pca_1": coords[:, 0],
547         }
548     )

```

```

551         "pca_2": coords[:, 1],
552     }
553 )
554 write_csv(assignments, RESULTS_DIR / "stage6_six_cluster_assignments_dataset1_pd.csv")
555 return assignments
556
557
558 def fdr_bh(p_values: np.ndarray) -> np.ndarray:
559     p = np.asarray(p_values, dtype=float)
560     adjusted = np.full(p.shape, np.nan, dtype=float)
561     valid = np.isfinite(p)
562     if not valid.any():
563         return adjusted
564     valid_p = p[valid]
565     order = np.argsort(valid_p)
566     ordered_p = valid_p[order]
567     n = len(ordered_p)
568     ordered_q = ordered_p * n / np.arange(1, n + 1)
569     ordered_q = np.minimum.accumulate(ordered_q[::-1])[:-1]
570     ordered_q = np.clip(ordered_q, 0, 1)
571     q = np.empty_like(ordered_q)
572     q[order] = ordered_q
573     adjusted[valid] = q
574     return adjusted
575
576
577 def cohens_d(values_1: np.ndarray, values_0: np.ndarray) -> float:
578     n_1 = len(values_1)
579     n_0 = len(values_0)
580     if n_1 < 2 or n_0 < 2:
581         return np.nan
582     var_1 = np.var(values_1, ddof=1)
583     var_0 = np.var(values_0, ddof=1)
584     pooled = ((n_1 - 1) * var_1 + (n_0 - 1) * var_0) / (n_1 + n_0 - 2)
585     if pooled <= 0:
586         return 0.0
587     return float((np.mean(values_1) - np.mean(values_0)) / np.sqrt(pooled))
588
589
590 def build_feature_profiles(
591     pd_subjects: pd.DataFrame,
592     labels: np.ndarray,
593     features: list[str],
594 ) -> tuple[pd.DataFrame, pd.DataFrame]:
595     raw = pd_subjects[features]
596     scaled = pd.DataFrame(
597         StandardScaler().fit_transform(raw.to_numpy(dtype=float)),
598         columns=features,
599         index=raw.index,
600     )
601     rows: list[dict[str, Any]] = []
602     for feature in features:
603         row: dict[str, Any] = {"feature": feature, "category": categorize_feature(feature)}
604         cluster_values = []
605         for cluster in range(MAIN_K):
606             values = raw.loc[labels == cluster, feature].to_numpy(dtype=float)
607             z_values = scaled.loc[labels == cluster, feature].to_numpy(dtype=float)
608             row[f"cluster_{cluster}_mean"] = float(np.mean(values))
609             row[f"cluster_{cluster}_zmean"] = float(np.mean(z_values))
610             cluster_values.append(values)
611         zmeans = np.array([row[f"cluster_{cluster}_zmean"] for cluster in range(MAIN_K)], dtype=float)
612         try:
613             h_stat, p_value = kruskal(*cluster_values)
614         except ValueError:
615             h_stat, p_value = np.nan, np.nan
616         row["max_abs_zmean_difference"] = float(np.max(zmeans) - np.min(zmeans))
617         row["kruskal_h"] = float(h_stat)
618         row["p_kruskal"] = float(p_value)
619         rows.append(row)
620
621     profiles = pd.DataFrame(rows)
622     profiles["q_kruskal"] = fdr_bh(profiles["p_kruskal"].to_numpy())
623     ordered_columns = (
624         ["feature", "category"]
625         + [f"cluster_{cluster}_mean" for cluster in range(MAIN_K)]
626         + [f"cluster_{cluster}_zmean" for cluster in range(MAIN_K)]
627         + ["max_abs_zmean_difference", "kruskal_h", "p_kruskal", "q_kruskal"]
628     )
629     profiles = profiles[ordered_columns].sort_values(
630         ["max_abs_zmean_difference", "q_kruskal"],
631         ascending=[False, True],
632     ).reset_index(drop=True)
633
634     top_rows: list[dict[str, Any]] = []
635     for cluster in range(MAIN_K):
636         other_mask = labels != cluster
637         cluster_mask = labels == cluster
638         for _, row in profiles.iterrows():
639             feature = str(row["feature"])
640             zmean = float(row[f"cluster_{cluster}_zmean"])
641             other_zmean = float(np.mean([row[f"cluster_{other}_zmean"] for other in range(MAIN_K) if other != cluster]))
642             contrast = zmean - other_zmean

```

```

643     d_value = cohens_d(
644         raw.loc[cluster_mask, feature].to_numpy(dtype=float),
645         raw.loc[other_mask, feature].to_numpy(dtype=float),
646     )
647     top_rows.append(
648         {
649             "stage6_cluster": cluster,
650             "feature": feature,
651             "category": row["category"],
652             "cluster_zmean": zmean,
653             "other_clusters_mean_zmean": other_zmean,
654             "one_vs_rest_zmean_difference": contrast,
655             "one_vs_rest_cohens_d": d_value,
656             "abs_one_vs_rest_zmean_difference": abs(contrast),
657             "abs_one_vs_rest_cohens_d": abs(d_value) if np.isfinite(d_value) else np.nan,
658             "q_kruskal": row["q_kruskal"],
659             "direction": "positive" if contrast >= 0 else "negative",
660         }
661     )
662     top_by_cluster = (
663         pd.DataFrame(top_rows)
664         .sort_values(
665             ["stage6_cluster", "abs_one_vs_rest_zmean_difference", "abs_one_vs_rest_cohens_d", "q_kruskal"],
666             ascending=[True, False, False, True],
667         )
668         .groupby("stage6_cluster", group_keys=False)
669         .head(10)
670         .reset_index(drop=True)
671     )
672
673     write_csv(profiles, RESULTS_DIR / "stage6_six_cluster_feature_profiles.csv")
674     write_csv(top_by_cluster, RESULTS_DIR / "stage6_six_cluster_top_features_by_cluster.csv")
675     return profiles, top_by_cluster
676
677
678 def category_phrase(category: str) -> str:
679     mapping = {
680         "Delta / Delta-delta": "dynamic_delta",
681         "TQWT": "tqwt",
682         "MFCC": "mfcc",
683         "Shimmer": "shimmer",
684         "Jitter": "jitter",
685         "HNR / Harmonicity": "harmonicity",
686         "Nonlinear": "nonlinear",
687         "Intensity": "intensity",
688         "Formant / Bandwidth": "formant_bandwidth",
689         "Wavelet": "wavelet",
690         "Other": "mixed_acoustic",
691     }
692     return mapping.get(category, "mixed_acoustic")
693
694
695 def build_cluster_profiles(
696     assignments: pd.DataFrame,
697     top_by_cluster: pd.DataFrame,
698 ) -> pd.DataFrame:
699     rows: list[dict[str, Any]] = []
700     for cluster in range(MAIN_K):
701         cluster_assignments = assignments.loc[assignments["stage6_cluster"] == cluster]
702         cluster_features = top_by_cluster.loc[top_by_cluster["stage6_cluster"] == cluster]
703         category_counts = cluster_features["category"].value_counts()
704         dominant_categories = "; ".join(f"{cat}:{int(count)}" for cat, count in category_counts.items())
705         positives = cluster_features.loc[cluster_features["one_vs_rest_zmean_difference"] >= 0].head(5)
706         negatives = cluster_features.loc[cluster_features["one_vs_rest_zmean_difference"] < 0].head(5)
707         if positives.empty:
708             positives = cluster_features.sort_values("one_vs_rest_zmean_difference", ascending=False).head(5)
709         if negatives.empty:
710             negatives = cluster_features.sort_values("one_vs_rest_zmean_difference", ascending=True).head(5)
711         dominant_category = category_counts.index[0] if not category_counts.empty else "Other"
712         direction = "high" if positives["one_vs_rest_zmean_difference"].mean() >=
713             ↪ abs(negatives["one_vs_rest_zmean_difference"].mean()) else "low"
714         label =
715             ↪ f"latent_voice_phenotype_{SAFE_LABEL_SUFFIXES[cluster]}_{category_phrase(str(dominant_category))}_{direction}"
716         rows.append(
717             {
718                 "stage6_cluster": cluster,
719                 "n_subjects": int(cluster_assignments.shape[0]),
720                 "ratio": float(cluster_assignments.shape[0] / assignments.shape[0]),
721                 "dominant_feature_categories": dominant_categories,
722                 "top_positive_features": "; ".join(positives["feature"].astype(str).tolist()),
723                 "top_negative_features": "; ".join(negatives["feature"].astype(str).tolist()),
724                 "stage5_cluster_distribution": str(
725                     {int(k): int(v) for k, v in cluster_assignments["stage5_cluster"].value_counts().sort_index().items()}
726                 ),
727                 "sex_male_distribution": str(
728                     {int(k): int(v) for k, v in cluster_assignments["sex_male"].value_counts().sort_index().items()}
729                 ),
730                 "tentative_voice_phenotype_label": label,
731                 "interpretation_note": "Exploratory acoustic phenotype label; not a clinical symptom diagnosis.",
732             }
733         )
734     profiles = pd.DataFrame(rows)

```

```

733 write_csv(profiles, RESULTS_DIR / "stage6_six_cluster_profiles.csv")
734 return profiles
735
736
737 def stage5_crosswalk(assignments: pd.DataFrame) -> pd.DataFrame:
738     crosstab = pd.crosstab(assignments["stage5_cluster"], assignments["stage6_cluster"])
739     nmi = float(normalized_mutual_info_score(assignments["stage5_cluster"], assignments["stage6_cluster"]))
740     ari = float(adjusted_rand_score(assignments["stage5_cluster"], assignments["stage6_cluster"]))
741     majority_map = {}
742     for stage6_cluster in sorted(assignments["stage6_cluster"].unique()):
743         subset = assignments.loc[assignments["stage6_cluster"] == stage6_cluster, "stage5_cluster"]
744         majority_map[int(stage6_cluster)] = int(subset.value_counts().idxmax())
745     collapsed = assignments["stage6_cluster"].map(majority_map)
746     collapsed_ari = float(adjusted_rand_score(assignments["stage5_cluster"], collapsed))
747
748     rows: list[dict[str, Any]] = []
749     for stage5_cluster in crosstab.index:
750         for stage6_cluster in crosstab.columns:
751             count = int(crosstab.loc[stage5_cluster, stage6_cluster])
752             row_total = int(crosstab.loc[stage5_cluster].sum())
753             col_total = int(crosstab[stage6_cluster].sum())
754             rows.append(
755                 {
756                     "stage5_cluster": int(stage5_cluster),
757                     "stage6_cluster": int(stage6_cluster),
758                     "count": count,
759                     "row_percentage": float(count / row_total) if row_total else 0.0,
760                     "column_percentage": float(count / col_total) if col_total else 0.0,
761                     "adjusted_rand_index_stage5_vs_stage6": ari,
762                     "normalized_mutual_info_stage5_vs_stage6": nmi,
763                     "collapsed_stage6_to_stage5_majority_ari": collapsed_ari,
764                 }
765             )
766     crosswalk = pd.DataFrame(rows)
767     write_csv(crosswalk, RESULTS_DIR / "stage6_stage5_crosswalk.csv")
768     return crosswalk
769
770
771 def run_classifier(pd_subjects: pd.DataFrame, features: list[str], labels: np.ndarray) -> tuple[pd.DataFrame, pd.DataFrame]:
772     x = pd_subjects[features].to_numpy(dtype=float)
773     y = labels.astype(int)
774     min_cluster_size = int(pd.Series(y).value_counts().min())
775     n_splits = min(5, min_cluster_size)
776     if n_splits < 2:
777         raise ValueError("six-cluster-label classifier requires at least 2 samples per class")
778
779     cv = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=RANDOM_STATE)
780     models: dict[str, Any] = {
781         "Multinomial Logistic Regression": Pipeline(
782             [
783                 ("scaler", StandardScaler()),
784                 (
785                     "model",
786                     LogisticRegression(
787                         max_iter=5000,
788                         class_weight="balanced",
789                         solver="lbfgs",
790                         random_state=RANDOM_STATE,
791                     ),
792                 ),
793             ]
794         ),
795         "SVM-RBF": Pipeline(
796             [
797                 ("scaler", StandardScaler()),
798                 (
799                     "model",
800                     SVC(
801                         kernel="rbf",
802                         probability=True,
803                         class_weight="balanced",
804                         random_state=RANDOM_STATE,
805                     ),
806                 ),
807             ]
808         ),
809         "Random Forest": RandomForestClassifier(
810             n_estimators=500,
811             class_weight="balanced",
812             random_state=RANDOM_STATE,
813         ),
814     }
815
816     metric_rows: list[dict[str, Any]] = []
817     cm_rows: list[dict[str, Any]] = []
818     labels_order = list(range(MAIN_K))
819
820     for model_name, model in models.items():
821         fold_rows: list[dict[str, Any]] = []
822         for fold_index, (train_index, test_index) in enumerate(cv.split(x, y), start=1):
823             model.fit(x[train_index], y[train_index])
824             predictions = model.predict(x[test_index])

```

```

825     try:
826         probabilities = model.predict_proba(x[test_index])
827         if probabilities.shape[1] == MAIN_K and set(np.unique(y[test_index])) == set(labels_order):
828             auc = float(roc_auc_score(y[test_index], probabilities, multi_class="ovr", labels=labels_order))
829         else:
830             auc = np.nan
831     except (AttributeError, ValueError):
832         auc = np.nan
833     row = {
834         "model": model_name,
835         "fold": fold_index,
836         "n_splits": n_splits,
837         "accuracy": float(accuracy_score(y[test_index], predictions)),
838         "balanced_accuracy": float(balanced_accuracy_score(y[test_index], predictions)),
839         "f1_macro": float(f1_score(y[test_index], predictions, average="macro", zero_division=0)),
840         "f1_weighted": float(f1_score(y[test_index], predictions, average="weighted", zero_division=0)),
841         "precision_macro": float(precision_score(y[test_index], predictions, average="macro", zero_division=0)),
842         "recall_macro": float(recall_score(y[test_index], predictions, average="macro", zero_division=0)),
843         "multiclass_roc_auc_ovr": auc,
844     }
845     metric_rows.append(row)
846     fold_rows.append(row)
847     matrix = confusion_matrix(y[test_index], predictions, labels=labels_order)
848     for true_label in labels_order:
849         for predicted_label in labels_order:
850             cm_rows.append(
851                 {
852                     "model": model_name,
853                     "fold": fold_index,
854                     "true_stage6_cluster": true_label,
855                     "predicted_stage6_cluster": predicted_label,
856                     "count": int(matrix[true_label, predicted_label]),
857                 }
858             )
859     fold_metrics = pd.DataFrame(fold_rows)
860     for summary_name, reducer in [("mean", np.nanmean), ("std", np.nanstd)]:
861         metric_rows.append(
862             {
863                 "model": model_name,
864                 "fold": summary_name,
865                 "n_splits": n_splits,
866                 "accuracy": float(reducer(fold_metrics["accuracy"])),
867                 "balanced_accuracy": float(reducer(fold_metrics["balanced_accuracy"])),
868                 "f1_macro": float(reducer(fold_metrics["f1_macro"])),
869                 "f1_weighted": float(reducer(fold_metrics["f1_weighted"])),
870                 "precision_macro": float(reducer(fold_metrics["precision_macro"])),
871                 "recall_macro": float(reducer(fold_metrics["recall_macro"])),
872                 "multiclass_roc_auc_ovr": float(reducer(fold_metrics["multiclass_roc_auc_ovr"])),
873             }
874         )
875     metrics = pd.DataFrame(metric_rows)
876     confusion = pd.DataFrame(cm_rows)
877     write_csv(metrics, RESULTS_DIR / "stage6_six_cluster_label_classifier_metrics.csv")
878     write_csv(confusion, RESULTS_DIR / "stage6_six_cluster_label_classifier_confusion_matrices.csv")
879     return metrics, confusion
880
881
882
883 def markdown_table(df: pd.DataFrame, columns: list[str], max_rows: int | None = None) -> str:
884     table = df.loc[:, columns].copy()
885     table.columns = [str(column) for column in table.columns]
886     if max_rows is not None:
887         table = table.head(max_rows)
888     for column in table.columns:
889         if pd.api.types.is_float_dtype(table[column]):
890             table[column] = table[column].map(lambda value: "" if pd.isna(value) else f"{value:.4f}")
891     header = "| " + " | ".join(table.columns) + " |"
892     separator = "| " + " | ".join(["---"] * len(table.columns)) + " |"
893     rows = ["| " + " | ".join(str(value) for value in row) + " |" for row in table.to_numpy()]
894     return "\n".join([header, separator, *rows])
895
896
897 def plot_pca_clusters(assignments: pd.DataFrame) -> None:
898     fig, ax = plt.subplots(figsize=(8, 6))
899     for cluster in sorted(assignments["stage6_cluster"].unique()):
900         subset = assignments.loc[assignments["stage6_cluster"] == cluster]
901         ax.scatter(subset["pca_1"], subset["pca_2"], s=44, alpha=0.85, label=f"Cluster {cluster}")
902     ax.set_title("Stage 6 Main k=6 Latent Voice Phenotype Clusters")
903     ax.set_xlabel("PCA 1")
904     ax.set_ylabel("PCA 2")
905     ax.legend(title="Stage 6 cluster", ncol=2)
906     fig.tight_layout()
907     fig.savefig(FIGURES_DIR / "stage6_pca_k6_clusters_main.png", dpi=220)
908     plt.close(fig)
909
910
911 def plot_cluster_size(assignments: pd.DataFrame) -> None:
912     counts = assignments["stage6_cluster"].value_counts().sort_index()
913     ratios = counts / counts.sum()
914     fig, ax = plt.subplots(figsize=(8, 5))
915     bars = ax.bar(counts.index.astype(str), counts.values, color="#4c78a8")
916     for bar, ratio in zip(bars, ratios):

```

```

917         ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height(), f"{ratio:.1%}", ha="center", va="bottom")
918     ax.set_title("Stage 6 Cluster Size Distribution")
919     ax.set_xlabel("Stage 6 cluster")
920     ax.set_ylabel("Number of PD subjects")
921     fig.tight_layout()
922     fig.savefig(FIGURES_DIR / "stage6_cluster_size_distribution.png", dpi=220)
923     plt.close(fig)
924
925
926 def plot_candidate_metrics(metrics: pd.DataFrame) -> None:
927     plot_data = metrics.copy()
928     plot_data["candidate"] = (
929         plot_data["feature_set"] + "\n" + plot_data["representation"] + "\n" + plot_data["clustering_method"]
930     )
931     fig, axes = plt.subplots(3, 1, figsize=(12, 10), sharex=True)
932     for ax, metric, title in [
933         (axes[0], "silhouette", "Silhouette"),
934         (axes[1], "davies_bouldin", "Davies-Bouldin"),
935         (axes[2], "stability_ari_mean", "Stability ARI Mean"),
936     ]:
937         colors = np.where(plot_data["recommended_flag"], "#d55e00", "#4c78a8")
938         ax.bar(np.arange(plot_data.shape[0]), plot_data[metric], color=colors)
939         ax.set_ylabel(title)
940         ax.set_title(f"Stage 6 Candidate {title}")
941     axes[-1].set_xticks(np.arange(plot_data.shape[0]))
942     axes[-1].set_xticklabels(plot_data["candidate"], rotation=75, ha="right", fontsize=7)
943     fig.tight_layout()
944     fig.savefig(FIGURES_DIR / "stage6_candidate_metrics_comparison.png", dpi=220)
945     plt.close(fig)
946
947
948 def plot_feature_heatmap(feature_profiles: pd.DataFrame) -> None:
949     top = feature_profiles.head(20)
950     matrix = top[[f"cluster_{cluster}_zmean" for cluster in range(MAIN_K)]].to_numpy(dtype=float)
951     fig, ax = plt.subplots(figsize=(9, 8))
952     image = ax.imshow(matrix, aspect="auto", cmap="coolwarm")
953     ax.set_yticks(np.arange(top.shape[0]))
954     ax.set_yticklabels(top["feature"], fontsize=7)
955     ax.set_xticks(np.arange(MAIN_K))
956     ax.set_xticklabels([f"Cluster {cluster}" for cluster in range(MAIN_K)])
957     ax.set_title("Stage 6 Cluster z-Mean Heatmap for Top Features")
958     ax.set_xlabel("Stage 6 cluster")
959     ax.set_ylabel("Feature")
960     fig.colorbar(image, ax=ax, label="Cluster z-mean")
961     fig.tight_layout()
962     fig.savefig(FIGURES_DIR / "stage6_cluster_feature_heatmap.png", dpi=220)
963     plt.close(fig)
964
965
966 def plot_crosswalk_heatmap(crosswalk: pd.DataFrame) -> None:
967     pivot = crosswalk.pivot(index="stage5_cluster", columns="stage6_cluster", values="count").fillna(0)
968     fig, ax = plt.subplots(figsize=(8, 4))
969     image = ax.imshow(pivot.to_numpy(dtype=float), aspect="auto", cmap="Blues")
970     ax.set_xticks(np.arange(pivot.shape[1]))
971     ax.set_xticklabels([f"Stage 6 {column}" for column in pivot.columns])
972     ax.set_yticks(np.arange(pivot.shape[0]))
973     ax.set_yticklabels([f"Stage 5 {index}" for index in pivot.index])
974     for i in range(pivot.shape[0]):
975         for j in range(pivot.shape[1]):
976             ax.text(j, i, int(pivot.iloc[i, j]), ha="center", va="center", color="#111111")
977     ax.set_title("Stage 5 Two-Cluster vs Stage 6 Six-Cluster Crosswalk")
978     ax.set_xlabel("Stage 6 cluster")
979     ax.set_ylabel("Stage 5 cluster")
980     fig.colorbar(image, ax=ax, label="Subject count")
981     fig.tight_layout()
982     fig.savefig(FIGURES_DIR / "stage6_stage5_crosswalk_heatmap.png", dpi=220)
983     plt.close(fig)
984
985
986 def plot_top_features_by_cluster(top_by_cluster: pd.DataFrame) -> None:
987     fig, axes = plt.subplots(3, 2, figsize=(14, 12))
988     axes = axes.flatten()
989     for cluster, ax in enumerate(axes):
990         data = top_by_cluster.loc[top_by_cluster["stage6_cluster"] == cluster].copy()
991         data = data.sort_values("one_vs_rest_zmean_difference")
992         colors = np.where(data["one_vs_rest_zmean_difference"] >= 0, "#2f80ed", "#e07a5f")
993         ax.barh(data["feature"], data["one_vs_rest_zmean_difference"], color=colors)
994         ax.axvline(0, color="#333333", linewidth=0.8)
995         ax.set_title(f"Cluster {cluster}")
996         ax.set_xlabel("One-vs-rest z-mean difference")
997         ax.tick_params(axis="y", labelsiz=7)
998     fig.suptitle("Stage 6 Top Distinguishing Features by Cluster", y=0.995)
999     fig.tight_layout()
1000     fig.savefig(FIGURES_DIR / "stage6_top_features_by_cluster.png", dpi=220)
1001     plt.close(fig)
1002
1003
1004 def report_selection_reason(metrics: pd.DataFrame, recommended: pd.Series) -> str:
1005     top = metrics.sort_values("selection_score", ascending=False).head(5)
1006     text = (
1007         f"主推荐方案为 `{recommended['feature_set']}` / `{recommended['representation']}` / "
1008         f"`{recommended['clustering_method']}`。该方案的最小簇人数为 `{int(recommended['min_cluster_size'])}`，"

```

```

1009         f" 最小簇比例为 `{recommended['min_cluster_ratio']:.3f}`，稳定性 ARI 均值为 "
1010         f"`{recommended['stability_ari_mean']:.3f}`。选择时同时考虑了簇大小、稳定性、聚类指标、"
1011         " 特征集可解释性、是否依赖高维噪声，以及与 Stage 5 二类语音表型的关系。\\n\\n"
1012         " 没有只按单项指标选择方案。若某些候选 silhouette 或 Calinski-Harabasz 较高，但存在小簇、"
1013         " 不稳定或高维噪声依赖，则不作为论文主解释方案。\\n\\n"
1014     )
1015     return text + markdown_table(
1016         top,
1017         [
1018             "feature_set",
1019             "representation",
1020             "clustering_method",
1021             "selection_score",
1022             "min_cluster_size",
1023             "min_cluster_ratio",
1024             "silhouette",
1025             "stability_ari_mean",
1026             "stage5_nmi",
1027             "recommended_flag",
1028         ],
1029     )
1030
1031
1032 def write_report(
1033     subject_table: pd.DataFrame,
1034     pd_subjects: pd.DataFrame,
1035     metrics: pd.DataFrame,
1036     k_scan: pd.DataFrame,
1037     assignments: pd.DataFrame,
1038     feature_profiles: pd.DataFrame,
1039     top_by_cluster: pd.DataFrame,
1040     cluster_profiles: pd.DataFrame,
1041     crosswalk: pd.DataFrame,
1042     classifier_metrics: pd.DataFrame,
1043     consensus: dict[str, float],
1044 ) -> None:
1045     recommended = metrics.loc[metrics["recommended_flag"]].iloc[0]
1046     classifier_summary = classifier_metrics[classifier_metrics["fold"].isin(["mean", "std"])]
1047     cluster_size_table = (
1048         assignments.groupby("stage6_cluster")
1049         .size()
1050         .reset_index(name="n_subjects")
1051     )
1052     cluster_size_table["ratio"] = cluster_size_table["n_subjects"] / assignments.shape[0]
1053     sex_table = pd.crosstab(assignments["stage6_cluster"], assignments["sex_male"], margins=True)
1054     category_counts = top_by_cluster["category"].value_counts().reset_index()
1055     category_counts.columns = ["category", "top_feature_count"]
1056     boundary_text = (
1057         " 由于附件数据未提供静止性震颤、运动迟缓、肌肉僵直、疼痛、痴呆、睡眠障碍的真实临床标签，"
1058         " 本文不直接构建六类症状监督分类模型，而是在帕金森病受试者内部进行 k=6 无监督聚类，"
1059         " 以获得基于语音特征的六类潜在表型。随后训练的分类器预测的是聚类标签，而非临床确诊标签。"
1060         " 因此，该模型只能作为六类帕金森病症状辅助诊断的探索性语音表型参考，仍需真实临床症状标签进一步验证。"
1061     )
1062     report = [
1063         "# Stage 6 Dataset 1 PD Six-Cluster Voice Phenotyping",
1064         "",
1065         "## 1. Stage 6 目标与边界",
1066         "本阶段在 PD 受试者内部进行 k=6 无监督聚类，输出基于语音特征的六类潜在表型，并训练 six-cluster-label classifier  

1067         ↳ 预测聚类标签。",
1068         " 六个 cluster 编号没有临床症状含义。",
1069         "",
1070         "## 2. 为什么不能做真实六类临床症状监督诊断",
1071         boundary_text,
1072         "",
1073         "## 3. 输入文件与样本审计",
1074         f"- Subject-level input: `{rel(SUBJECT_TABLE_PATH)}`",
1075         f"- Total subjects: `{subject_table.shape[0]}`",
1076         f"- PD subjects: `{pd_subjects.shape[0]}`",
1077         f"- Class distribution: `{subject_table['class'].value_counts().sort_index().to_dict()}`",
1078         f"- Missing values: `{int(subject_table.isna().sum().sum())}`",
1079         f"- Duplicated id count: `{int(subject_table['id'].duplicated().sum())}`",
1080         f"- Stage 5 assignment file: `{rel(STAGE5_ASSIGNMENTS_PATH)}`",
1081         "",
1082         "## 4. 特征集与预处理",
1083         "使用 Top20、Top50 和 all_acoustic 三套特征集，所有表示均先 StandardScaler，再按需要 PCA。`id`，`gender`，`class`，  

1084         ↳ `sex_male`，`record_count`，Stage 5 cluster 和 Stage 6 cluster 不进入聚类或分类器输入。",
1085         "",
1086         "## 5. k=6 聚类候选方案完整指标表",
1087         markdown_table(
1088             metrics,
1089             [
1090                 "feature_set",
1091                 "representation",
1092                 "clustering_method",
1093                 "cluster_sizes",
1094                 "min_cluster_size",
1095                 "min_cluster_ratio",
1096                 "silhouette",
1097                 "davies_bouldin",
1098                 "stability_ari_mean",
1099                 "recommended_flag",
1100             ],
1101         ),
1102     ]

```

```

1099     ),
1100     "",
1101     "### 6. k=2 到 k=8 参考扫描结果",
1102     "k 扫描只用于说明 k=6 的相对聚类质量和稳定性; k=6 是由题目六类设定驱动的探索性语音表型划分, 不是由真实六类临床标签监督得到。",
1103     ↪ " ",
1104     markdown_table(
1105         k_scan,
1106         [
1107             "k",
1108             "clustering_method",
1109             "min_cluster_size",
1110             "silhouette",
1111             "davies_bouldin",
1112             "stability_ari_mean",
1113         ],
1114         max_rows=21,
1115     ),
1116     "",
1117     "### 7. 稳定性分析结果",
1118     f" 每个 k=6 候选和 k 扫描方案均执行 `{STABILITY_RUNS}` 次 80% PD 受试者重采样, 重新标准化、PCA 与聚类, 并计算 ARI。",
1119     f"主方案 Consensus KMeans 参考: within-cluster mean `{consensus['consensus_within_cluster_mean']:.3f}`,",
1120     ↪ " between-cluster mean `{consensus['consensus_between_cluster_mean']:.3f}`, separation",
1121     ↪ "`{consensus['consensus_separation']:.3f}`。",
1122     "",
1123     "### 8. 主推荐方案选择理由",
1124     report_selection_reason(metrics, recommended),
1125     "",
1126     "### 9. 六个簇的人数与比例",
1127     markdown_table(cluster_size_table, ["stage6_cluster", "n_subjects", "ratio"]),
1128     "",
1129     "### 10. 六个簇的声学特征画像",
1130     markdown_table(
1131         cluster_profiles,
1132         [
1133             "stage6_cluster",
1134             "n_subjects",
1135             "ratio",
1136             "dominant_feature_categories",
1137             "tentative_voice_phenotype_label",
1138         ],
1139     ),
1140     "",
1141     "### 11. 每个簇的 Top 区分语音特征",
1142     markdown_table(
1143         top_by_cluster,
1144         [
1145             "stage6_cluster",
1146             "feature",
1147             "category",
1148             "one_vs_rest_zmean_difference",
1149             "one_vs_rest_cohens_d",
1150             "q_kruskal",
1151         ],
1152         max_rows=60,
1153     ),
1154     "",
1155     "### 12. 特征类别分布与解释",
1156     markdown_table(category_counts, ["category", "top_feature_count"]),
1157     " 这些类别分布用于描述六类潜在语音表型的声学差异来源, 不是临床病因解释。",
1158     "",
1159     "### 13. Stage 5 二类表型与 Stage 6 六类表型关系",
1160     markdown_table(
1161         crosswalk,
1162         [
1163             "stage5_cluster",
1164             "stage6_cluster",
1165             "count",
1166             "row_percentage",
1167             "column_percentage",
1168             "normalized_mutual_info_stage5_vs_stage6",
1169             "collapsed_stage6_to_stage5_majority_ari",
1170         ],
1171     ),
1172     " 该交叉表用于观察六类潜在表型是否可视为二类潜在语音表型的细分; 不能解释为真实临床层级诊断。",
1173     "",
1174     "### 14. 性别分布的事后描述",
1175     markdown_table(sex_table.reset_index(), list(sex_table.reset_index().columns)),
1176     " `sex_male` 未进入 Stage 6 聚类或 six-cluster-label classifier, 只用于事后描述, 不能作为因果解释。",
1177     "",
1178     "### 15. six-cluster-label classifier 结果",
1179     markdown_table(
1180         classifier_summary,
1181         [
1182             "model",
1183             "fold",
1184             "accuracy",
1185             "balanced_accuracy",
1186             "f1_macro",
1187             "f1_weighted",
1188             "precision_macro",
1189             "recall_macro",
1190             "multiclass_roc_auc_ovr",

```

```

1188     ],
1189 ),
1190 "该分类器学习的是无监督六类聚类标签，用于将潜在语音表型分型规则模型化；它不是训练自真实静止性震颤、运动迟缓、肌肉僵直、疼痛、
    ↳ 痴呆或睡眠障碍临床标签，因此不能解释为真实六类症状诊断性能。",
1191 "",
1192 "## 16. 局限性",
1193 "- 六类标签来自语音特征内部结构，没有外部临床症状标签验证。",
1194 "- Kruskal-Wallis 和 FDR 是聚类后的描述性比较，不是外部临床验证假设检验。",
1195 "- cluster 编号和 tentative label 均为声学解释命名，不对应具体临床症状。",
1196 "- k=6 由题目设定驱动，k 扫描只提供参考。",
1197 "",
1198 "## 17. 论文建议表述",
1199 "建议写作：本文在 PD 受试者内部基于 Stage 4 筛选的声学特征进行 k=6 无监督聚类，得到六个基于语音特征的潜在表型，并训练
    ↳ six-cluster-label classifier 复现该探索性分型规则。",
1200 "不建议写作：将本阶段结果表述为真实六类临床症状诊断性能。",
1201 "",
1202 "## 输出文件",
1203 "- `results/stage6_six_cluster_input_audit.csv`",
1204 "- `results/stage6_six_cluster_metrics.csv`",
1205 "- `results/stage6_k_scan_metrics.csv`",
1206 "- `results/stage6_six_cluster_assignments_dataset1_pd.csv`",
1207 "- `results/stage6_six_cluster_feature_profiles.csv`",
1208 "- `results/stage6_six_cluster_top_features_by_cluster.csv`",
1209 "- `results/stage6_six_cluster_profiles.csv`",
1210 "- `results/stage6_stage5_crosswalk.csv`",
1211 "- `results/stage6_six_cluster_label_classifier_metrics.csv`",
1212 "- `results/stage6_six_cluster_label_classifier_confusion_matrices.csv`",
1213 ]
1214 (RESULTS_DIR / "stage6_six_cluster_report.md").write_text("\n".join(report), encoding="utf-8")
1215
1216
1217 def main() -> None:
1218     RESULTS_DIR.mkdir(exist_ok=True)
1219     FIGURES_DIR.mkdir(exist_ok=True)
1220
1221     subject_table, top20_features, top50_features, _, stage5_assignments, _ = load_inputs()
1222     all_features = acoustic_columns(subject_table)
1223     pd_subjects = validate_pd_subjects(subject_table, stage5_assignments)
1224     write_input_audit(subject_table, pd_subjects, all_features, top20_features, top50_features, stage5_assignments)
1225
1226     feature_sets = {
1227         "top20_biomarkers": top20_features,
1228         "top50_biomarkers": top50_features,
1229         "all_acoustic": all_features,
1230     }
1231     stage5_labels = aligned_stage5(stage5_assignments, pd_subjects)
1232     metrics, artifacts = run_clustering_candidates(pd_subjects, feature_sets, MAIN_K)
1233     metrics, recommended_key = choose_recommended(metrics, artifacts, stage5_labels.to_numpy())
1234     write_csv(metrics, RESULTS_DIR / "stage6_six_cluster_metrics.csv")
1235
1236     recommended_artifact = artifacts[recommended_key]
1237     recommended_labels = np.asarray(recommended_artifact["labels"], dtype=int)
1238     assignments = write_assignments(pd_subjects, stage5_labels, recommended_key, recommended_artifact)
1239     k_scan = run_k_scan(
1240         pd_subjects,
1241         recommended_artifact["features"],
1242         recommended_key[0],
1243         recommended_key[1],
1244     )
1245     consensus = consensus_kmeans_summary(
1246         pd_subjects[recommended_artifact["features"]],
1247         recommended_key[1],
1248         recommended_labels,
1249     )
1250
1251     feature_profiles, top_by_cluster = build_feature_profiles(
1252         pd_subjects,
1253         recommended_labels,
1254         recommended_artifact["features"],
1255     )
1256     cluster_profiles = build_cluster_profiles(assignments, top_by_cluster)
1257     crosswalk = stage5_crosswalk(assignments)
1258     classifier_metrics, _ = run_classifier(
1259         pd_subjects,
1260         recommended_artifact["features"],
1261         recommended_labels,
1262     )
1263
1264     plot_pca_clusters(assignments)
1265     plot_cluster_size(assignments)
1266     plot_candidate_metrics(metrics)
1267     plot_feature_heatmap(feature_profiles)
1268     plot_crosswalk_heatmap(crosswalk)
1269     plot_top_features_by_cluster(top_by_cluster)
1270     write_report(
1271         subject_table,
1272         pd_subjects,
1273         metrics,
1274         k_scan,
1275         assignments,
1276         feature_profiles,
1277         top_by_cluster,

```

```
1278         cluster_profiles,
1279         crosswalk,
1280         classifier_metrics,
1281         consensus,
1282     )
1283
1284
1285 if __name__ == "__main__":
1286     main()
```

A.6 代码六：全流程逻辑审计

```
1 from __future__ import annotations
2
3 import ast
4 import math
5 import re
6 from pathlib import Path
7 from typing import Any
8
9 import numpy as np
10 import pandas as pd
11
12
13 ROOT = Path(__file__).resolve().parents[1]
14 SRC_DIR = ROOT / "src"
15 DATA_DIR = ROOT / "data"
16 RESULTS_DIR = ROOT / "results"
17
18 OUTPUT_CSV_ENCODING = "utf-8-sig"
19
20 DATASET1_FILE = "pd_speech_features.csv"
21 DATASET2_FILE = "ReplicatedAcousticFeatures-ParkinsonDatabase.csv"
22
23 CHECK_COLUMNS = [
24     "stage",
25     "check_name",
26     "expected",
27     "observed",
28     "status",
29     "severity",
30     "evidence_file",
31     "notes",
32 ]
33
34 STATIC_KEYWORDS = [
35     "fit_transform",
36     "StandardScaler(",
37     "PCA(",
38     "SelectKBest",
39     "SMOTE",
40     "train_test_split",
41     "cross_val_score",
42     "GridSearchCV",
43     "LogisticRegressionCV",
44     "permutation_importance",
45     "GroupKFold",
46     "StratifiedGroupKFold",
47     ".groupby(",
48     "id",
49     "ID",
50     "Recording",
51     "sex_male",
52     "gender",
53     "Gender",
54     "class",
55     "Status",
56     "cluster",
57     "stage5_cluster",
58     "stage6_cluster",
59 ]
60
61 FORBIDDEN_FEATURE_COLUMNS = {
62     "id",
63     "ID",
64     "gender",
65     "Gender",
66     "class",
67     "Status",
68     "Recording",
69     "sex_male",
70     "record_count",
71     "cluster",
72     "stage5_cluster",
73     "stage6_cluster",
74 }
75
76 CLINICAL_SYMPTOM_TERMS = [
77     "resting tremor",
78     "bradykinesia",
79     "rigidity",
80     "pain",
81     "dementia",
82     "sleep disorder",
83     "静止性震颤",
84     "运动迟缓",
85     "肌肉僵直",
86     "疼痛",
87     "痴呆",
88     "睡眠障碍",
89 ]
90
```

```

91
92 class Audit:
93     def __init__(self) -> None:
94         self.rows: list[dict[str, Any]] = []
95         self.static_rows: list[dict[str, Any]] = []
96
97     def add_check(
98         self,
99         stage: str,
100         check_name: str,
101         expected: Any,
102         observed: Any,
103         status: str,
104         severity: str,
105         evidence_file: str,
106         notes: str = "",
107     ) -> None:
108         if status not in {"pass", "warning", "fail"}:
109             raise ValueError(f"Invalid check status: {status}")
110         self.rows.append(
111             {
112                 "stage": stage,
113                 "check_name": check_name,
114                 "expected": stringify(expected),
115                 "observed": stringify(observed),
116                 "status": status,
117                 "severity": severity,
118                 "evidence_file": evidence_file,
119                 "notes": notes,
120             }
121         )
122
123
124 def stringify(value: Any) -> str:
125     if isinstance(value, float):
126         if math.isnan(value):
127             return "nan"
128         return f"{value:.12g}"
129     if isinstance(value, (dict, list, tuple, set)):
130         return str(value)
131     return "" if value is None else str(value)
132
133
134 def rel(path: Path | str) -> str:
135     path_obj = Path(path)
136     try:
137         return path_obj.relative_to(ROOT).as_posix()
138     except ValueError:
139         return str(path)
140
141
142 def read_csv(path: Path, **kwargs: Any) -> pd.DataFrame:
143     return pd.read_csv(path, **kwargs)
144
145
146 def find_data_file(file_name: str) -> Path | None:
147     matches = sorted(DATA_DIR.rglob(file_name))
148     return matches[0] if matches else None
149
150
151 def value_counts_dict(series: pd.Series) -> dict[Any, int]:
152     out: dict[Any, int] = {}
153     for key, value in series.value_counts(dropna=False).sort_index().items():
154         if pd.isna(key):
155             out["nan"] = int(value)
156         elif isinstance(key, (np.integer, int)):
157             out[int(key)] = int(value)
158         elif isinstance(key, (np.floating, float)) and float(key).is_integer():
159             out[int(key)] = int(value)
160         else:
161             out[key] = int(value)
162     return out
163
164
165 def normalize_id_set(series: pd.Series) -> set[str]:
166     return set(series.astype(str).tolist())
167
168
169 def safe_read_text(path: Path) -> str:
170     try:
171         return path.read_text(encoding="utf-8")
172     except UnicodeDecodeError:
173         return path.read_text(encoding="utf-8", errors="replace")
174
175
176 def parse_dictish(value: Any) -> Any:
177     if pd.isna(value):
178         return value
179     if isinstance(value, str):
180         try:
181             return ast.literal_eval(value)
182         except (SyntaxError, ValueError):

```

```

183         return value
184     return value
185
186
187 def check_equal(
188     audit: Audit,
189     stage: str,
190     check_name: str,
191     expected: Any,
192     observed: Any,
193     severity: str,
194     evidence_file: str,
195     notes: str = "",
196 ) -> None:
197     audit.add_check(
198         stage,
199         check_name,
200         expected,
201         observed,
202         "pass" if observed == expected else "fail",
203         severity,
204         evidence_file,
205         notes,
206     )
207
208
209 def check_true(
210     audit: Audit,
211     stage: str,
212     check_name: str,
213     condition: bool,
214     expected: str,
215     observed: Any,
216     severity: str,
217     evidence_file: str,
218     fail_status: str = "fail",
219     notes: str = "",
220 ) -> None:
221     audit.add_check(
222         stage,
223         check_name,
224         expected,
225         observed,
226         "pass" if condition else fail_status,
227         severity,
228         evidence_file,
229         notes,
230     )
231
232
233 def numeric_feature_columns(df: pd.DataFrame, excluded: set[str]) -> list[str]:
234     return [
235         column
236         for column in df.select_dtypes(include="number").columns
237         if column not in excluded
238     ]
239
240
241 def audit_stage1_and_stage2(audit: Audit) -> dict[str, pd.DataFrame]:
242     out: dict[str, pd.DataFrame] = {}
243     dataset1_path = find_data_file(DATASET1_FILE)
244     dataset2_path = find_data_file(DATASET2_FILE)
245
246     check_true(
247         audit,
248         "Stage 1",
249         "Dataset 1 source file exists",
250         dataset1_path is not None,
251         DATASET1_FILE,
252         rel(dataset1_path) if dataset1_path else "missing",
253         "critical",
254         "data",
255     )
256     check_true(
257         audit,
258         "Stage 1",
259         "Dataset 2 source file exists",
260         dataset2_path is not None,
261         DATASET2_FILE,
262         rel(dataset2_path) if dataset2_path else "missing",
263         "critical",
264         "data",
265     )
266     if dataset1_path is None or dataset2_path is None:
267         return out
268
269     stage2_path = SRC_DIR / "stage2_qc_pipeline.py"
270     stage2_text = safe_read_text(stage2_path)
271     has_header_1 = bool(re.search(r"header\s*['\"]?\s*:\s*1|header\s*=\s*1", stage2_text))
272     check_true(
273         audit,
274         "Stage 1",

```

```

275         "Dataset 1 code uses header=1",
276         has_header_1,
277         "pd.read_csv(..., header=1)",
278         "header=1 found" if has_header_1 else "header=1 not found",
279         "critical",
280         rel(stage2_path),
281         notes="No standalone stage1_data_audit.py exists; stage2_qc_pipeline.py is the source of truth.",
282     )
283
284     raw1 = read_csv(dataset1_path, header=1)
285     raw2 = read_csv(dataset2_path)
286     out["raw1"] = raw1
287     out["raw2"] = raw2
288
289     check_equal(audit, "Stage 1", "Dataset 1 raw shape", (756, 755), raw1.shape, "critical", rel(dataset1_path))
290     check_equal(audit, "Stage 1", "Dataset 2 raw shape", (240, 48), raw2.shape, "critical", rel(dataset2_path))
291     check_equal(audit, "Stage 1", "Dataset 1 metadata columns", ["id", "gender", "class"], [c for c in ["id", "gender",
292     ↪ "class"] if c in raw1.columns], "high", rel(dataset1_path))
293     check_equal(audit, "Stage 1", "Dataset 2 metadata columns", ["ID", "Recording", "Status", "Gender"], [c for c in ["ID",
294     ↪ "Recording", "Status", "Gender"] if c in raw2.columns], "high", rel(dataset2_path))
295
296     d1_features = numeric_feature_columns(raw1, {"id", "gender", "class"})
297     d2_features = numeric_feature_columns(raw2, {"ID", "Recording", "Status", "Gender"})
298     check_equal(audit, "Stage 1", "Dataset 1 acoustic feature count", 752, len(d1_features), "critical", rel(dataset1_path))
299     check_equal(audit, "Stage 1", "Dataset 2 acoustic feature count", 44, len(d2_features), "critical", rel(dataset2_path))
300     check_equal(audit, "Stage 1", "Dataset 1 fully duplicated row count", 1, int(raw1.duplicated().sum()), "high",
301     ↪ rel(dataset1_path))
302     check_equal(audit, "Stage 1", "Dataset 1 missing values", 0, int(raw1.isna().sum().sum()), "high", rel(dataset1_path))
303     check_equal(audit, "Stage 1", "Dataset 2 missing values", 0, int(raw2.isna().sum().sum()), "high", rel(dataset2_path))
304     check_equal(audit, "Stage 1", "Dataset 1 subject label conflicts", 0,
305     ↪ int((raw1.groupby("id")["class"].nunique(dropna=False) > 1).sum()), "critical", rel(dataset1_path))
306     check_equal(audit, "Stage 1", "Dataset 2 subject label conflicts", 0,
307     ↪ int((raw2.groupby("ID")["Status"].nunique(dropna=False) > 1).sum()), "critical", rel(dataset2_path))
308
309     d1_ready_path = RESULTS_DIR / "model_ready_dataset1_deduplicated.csv"
310     d2_ready_path = RESULTS_DIR / "model_ready_dataset2_prepared.csv"
311     d1_ready = read_csv(d1_ready_path)
312     d2_ready = read_csv(d2_ready_path)
313     out["d1_ready"] = d1_ready
314     out["d2_ready"] = d2_ready
315
316     check_equal(audit, "Stage 2", "Dataset 1 ready row count after deduplication", 755, d1_ready.shape[0], "critical",
317     ↪ rel(d1_ready_path))
318     check_equal(audit, "Stage 2", "Dataset 1 ready column count", 756, d1_ready.shape[1], "high", rel(d1_ready_path))
319     check_equal(audit, "Stage 2", "Dataset 2 prepared shape", (240, 49), d2_ready.shape, "high", rel(d2_ready_path))
320     check_true(audit, "Stage 2", "Dataset 1 sex_male equals gender", bool((d1_ready["sex_male"] ==
321     ↪ d1_ready["gender"]).all()), "all rows sex_male == gender", "ok" if (d1_ready["sex_male"] ==
322     ↪ d1_ready["gender"]).all() else "mismatch", "high", rel(d1_ready_path))
323     check_true(audit, "Stage 2", "Dataset 2 sex_male equals 1 - Gender", bool((d2_ready["sex_male"] == 1 -
324     ↪ d2_ready["Gender"]).all()), "all rows sex_male == 1 - Gender", "ok" if (d2_ready["sex_male"] == 1 -
325     ↪ d2_ready["Gender"]).all() else "mismatch", "high", rel(d2_ready_path))
326     check_true(audit, "Stage 2", "Original gender columns retained", {"gender"}.issubset(d1_ready.columns) and
327     ↪ {"Gender"}.issubset(d2_ready.columns), "gender and Gender retained", f"dataset1 gender={ 'gender' in
328     ↪ d1_ready.columns }; dataset2 Gender={ 'Gender' in d2_ready.columns }", "medium", "results")
329     check_equal(audit, "Stage 2", "Dataset 1 subject record count distribution", {2: 1, 3: 251},
330     ↪ value_counts_dict(d1_ready.groupby("id").size()), "critical", rel(d1_ready_path))
331
332     raw2_features = numeric_feature_columns(raw2, {"ID", "Recording", "Status", "Gender"})
333     d2_ready_features = numeric_feature_columns(d2_ready, {"ID", "Recording", "Status", "Gender", "sex_male"})
334     same_feature_columns = raw2_features == d2_ready_features
335     same_feature_values = bool(np.allclose(raw2[raw2_features].to_numpy(dtype=float),
336     ↪ d2_ready[d2_ready_features].to_numpy(dtype=float))) if same_feature_columns else False
337     check_true(
338         audit,
339         "Stage 2",
340         "Dataset 2 prepared acoustic values unchanged",
341         same_feature_columns and same_feature_values,
342         "only sex_male added; acoustic columns unchanged",
343         f"same_columns={same_feature_columns}; same_values={same_feature_values}",
344         "critical",
345         f"{rel(dataset2_path)}; {rel(d2_ready_path)}",
346     )
347
348     for path in [
349         RESULTS_DIR / "dataset1_class_sex_male_crosstab_record.csv",
350         RESULTS_DIR / "dataset1_class_sex_male_crosstab_subject.csv",
351         RESULTS_DIR / "dataset2_class_sex_male_crosstab_record.csv",
352         RESULTS_DIR / "dataset2_class_sex_male_crosstab_subject.csv",
353     ]:
354         check_true(audit, "Stage 2", f"{path.name} exists", path.exists(), "file exists", rel(path) if path.exists() else
355         ↪ "missing", "medium", rel(path))
356
357     return out
358
359 def audit_stage3(audit: Audit) -> None:
360     code_path = SRC_DIR / "stage3_baseline_models.py"
361     code = safe_read_text(code_path)
362     fold_path = RESULTS_DIR / "stage3_fold_split_audit.csv"
363     metrics_path = RESULTS_DIR / "stage3_baseline_metrics_subject_level.csv"
364     folds = read_csv(fold_path)

```

```

351 metrics = read_csv(metrics_path)
352
353 check_true(audit, "Stage 3", "Uses StratifiedGroupKFold or GroupKFold", "StratifiedGroupKFold" in code and "GroupKFold"
↳ in code, "grouped CV code present", "present" if "GroupKFold" in code else "missing", "critical", rel(code_path))
354 check_true(audit, "Stage 3", "No executable train_test_split import/call", "from sklearn.model_selection import
↳ train_test_split" not in code and "train_test_split(" not in code, "no row-level train_test_split call", "no call"
↳ if "train_test_split(" not in code else "call found", "critical", rel(code_path))
355 check_equal(audit, "Stage 3", "Fold split audit overlap count", 0, int(folds["overlap_subject_count"].max()),
↳ "critical", rel(fold_path))
356 check_equal(audit, "Stage 3", "Fold splitter names", ["StratifiedGroupKFold"],
↳ sorted(folds["splitter"]).dropna().unique().tolist(), "high", rel(fold_path))
357 check_true(audit, "Stage 3", "Pipeline scaler for LR/SVM", "Pipeline" in code and "StandardScaler()" in code,
↳ "StandardScaler inside sklearn Pipeline", "present", "critical", rel(code_path))
358 check_true(audit, "Stage 3", "acoustic_only excludes sex_male", 'excluded = set(config.metadata_cols) | {"sex_male"}' in
↳ code, "metadata and sex_male excluded", "exclusion expression found" if 'excluded = set(config.metadata_cols) |
↳ {"sex_male"}' in code else "not found", "critical", rel(code_path))
359
360 required_metric_cols = {"balanced_accuracy", "roc_auc", "sensitivity", "specificity"}
361 check_true(audit, "Stage 3", "Subject-level metrics include required clinical metrics",
↳ required_metric_cols.issubset(metrics.columns), sorted(required_metric_cols), sorted(set(metrics.columns) &
↳ required_metric_cols), "medium", rel(metrics_path))
362 check_true(audit, "Stage 3", "Main subject-level acoustic_only results exist", ((metrics["feature_set"] ==
↳ "acoustic_only") & (metrics["summary"] == "mean")).any(), "summary rows for acoustic_only",
↳ int(((metrics["feature_set"] == "acoustic_only") & (metrics["summary"] == "mean")).sum()), "high",
↳ rel(metrics_path))
363 verify_metric_summary(audit, "Stage 3", metrics, ["dataset", "feature_set", "model"], "summary", "fold", ["accuracy",
↳ "balanced_accuracy", "roc_auc", "sensitivity", "specificity", "precision", "recall", "f1"], ddof=1,
↳ evidence_file=rel(metrics_path))
364
365
366 def audit_stage4(audit: Audit, tables: dict[str, pd.DataFrame]) -> None:
367 code_path = SRC_DIR / "stage4_biomarker_identification.py"
368 code = safe_read_text(code_path)
369 subject_path = RESULTS_DIR / "stage4_subject_level_feature_table_dataset1.csv"
370 top20_path = RESULTS_DIR / "stage4_top20_biomarkers_dataset1.csv"
371 top50_path = RESULTS_DIR / "stage4_top50_biomarkers_dataset1.csv"
372 subject = read_csv(subject_path)
373 top20 = read_csv(top20_path)
374 top50 = read_csv(top50_path)
375 tables["stage4_subject"] = subject
376 tables["stage4_top20"] = top20
377 tables["stage4_top50"] = top50
378
379 check_true(audit, "Stage 4", "Uses Dataset 1 deduplicated ready file", "model_ready_dataset1_deduplicated.csv" in code,
↳ "ready Dataset 1 input", "found" if "model_ready_dataset1_deduplicated.csv" in code else "missing", "critical",
↳ rel(code_path))
380 check_true(audit, "Stage 4", "Code aggregates by id before statistics", 'df.groupby("id" in code and
↳ "build_subject_table" in code, "subject-level groupby before feature tests", "found", "critical", rel(code_path))
381 check_equal(audit, "Stage 4", "Subject-level row count", 252, subject.shape[0], "critical", rel(subject_path))
382 check_equal(audit, "Stage 4", "Subject-level class distribution", {0: 64, 1: 188}, value_counts_dict(subject["class"]),
↳ "critical", rel(subject_path))
383 features = numeric_feature_columns(subject, {"id", "gender", "class", "sex_male", "record_count"})
384 check_equal(audit, "Stage 4", "Subject-level acoustic feature count", 752, len(features), "critical", rel(subject_path))
385 check_true(audit, "Stage 4", "Top20 row count", top20.shape[0] == 20, 20, top20.shape[0], "high", rel(top20_path))
386 check_true(audit, "Stage 4", "Top50 row count", top50.shape[0] == 50, 50, top50.shape[0], "high", rel(top50_path))
387 forbidden_top = sorted(set(top20["feature"]) | set(top50["feature"])) & FORBIDDEN_FEATURE_COLUMNS
388 check_equal(audit, "Stage 4", "Top20/Top50 forbidden metadata features", [], forbidden_top, "critical",
↳ f"{rel(top20_path)}; {rel(top50_path)}")
389 check_true(audit, "Stage 4", "L1 scaling inside Pipeline", "Pipeline" in code and "LogisticRegressionCV" in code and
↳ "StandardScaler()" in code, "Pipeline(StandardScaler, LogisticRegressionCV)", "found", "critical", rel(code_path))
390 check_true(audit, "Stage 4", "Permutation importance uses held-out fold function", "heldout_permutation_importance" in
↳ code and "x[test_idx]" in code, "held-out fold permutation", "found", "high", rel(code_path))
391 check_true(audit, "Stage 4", "Report guards candidate biomarkers from causal interpretation", "not clinical causal
↳ mechanisms" in code or "require independent validation" in code, "guardrail wording present", "present" if "clinical
↳ causal" in code or "independent validation" in code else "not found", "medium", rel(code_path))
392
393
394 def audit_stage5(audit: Audit, tables: dict[str, pd.DataFrame]) -> None:
395 code_path = SRC_DIR / "stage5_pd_two_cluster_phenotyping.py"
396 code = safe_read_text(code_path)
397 metrics_path = RESULTS_DIR / "stage5_clustering_metrics.csv"
398 assign_path = RESULTS_DIR / "stage5_cluster_assignments_dataset1_pd.csv"
399 input_audit_path = RESULTS_DIR / "stage5_pd_subject_clustering_input_audit.csv"
400 classifier_path = RESULTS_DIR / "stage5_cluster_label_classifier_metrics.csv"
401 report_path = RESULTS_DIR / "stage5_two_cluster_report.md"
402 metrics = read_csv(metrics_path)
403 assignments = read_csv(assign_path)
404 input_audit = read_csv(input_audit_path)
405 classifier = read_csv(classifier_path)
406 report = safe_read_text(report_path)
407 tables["stage5_assign"] = assignments
408
409 check_true(audit, "Stage 5", "Uses Stage 4 subject-level feature table",
↳ "stage4_subject_level_feature_table_dataset1.csv" in code, "Stage 4 subject table", "found", "critical",
↳ rel(code_path))
410 check_equal(audit, "Stage 5", "PD-only assignment row count", 188, assignments.shape[0], "critical", rel(assign_path))
411 check_equal(audit, "Stage 5", "Assignments class distribution", {1: 188}, value_counts_dict(assignments["class"]),
↳ "critical", rel(assign_path))
412 check_equal(audit, "Stage 5", "Assignment duplicated id count", 0, int(assignments["id"].duplicated().sum()),
↳ "critical", rel(assign_path))

```

```

413     check_equal(audit, "Stage 5", "Input audit acoustic feature count", 752, int(input_audit.loc[0,
    ↪ "acoustic_feature_count"]), "critical", rel(input_audit_path))
414     check_equal(audit, "Stage 5", "Top20 feature count", 20, int(input_audit.loc[0, "top20_feature_count"]), "high",
    ↪ rel(input_audit_path))
415     check_equal(audit, "Stage 5", "Top50 feature count", 50, int(input_audit.loc[0, "top50_feature_count"]), "high",
    ↪ rel(input_audit_path))
416     check_equal(audit, "Stage 5", "Recommended flag count", 1, int(metrics["recommended_flag"].sum()), "critical",
    ↪ rel(metrics_path))
417     recommended = metrics.loc[metrics["recommended_flag"] == True].iloc[0]
418     check_equal(audit, "Stage 5", "Recommended scheme", ("top20_biomarkers", "pca_5", "KMeans"),
    ↪ (recommended["feature_set"], recommended["representation"], recommended["clustering_method"]), "critical",
    ↪ rel(metrics_path))
419     check_equal(audit, "Stage 5", "Recommended cluster sizes", {0: 62, 1: 126}, value_counts_dict(assignments["cluster"]),
    ↪ "critical", rel(assign_path))
420     check_true(audit, "Stage 5", "Candidate count covers feature sets and methods", metrics.shape[0] == 21 and
    ↪ set(metrics["clustering_method"]) == {"KMeans", "GaussianMixture", "AgglomerativeClustering"}, "21 candidates with 3
    ↪ methods", f"rows={metrics.shape[0]}; methods={sorted(metrics['clustering_method'].unique())}", "medium",
    ↪ rel(metrics_path))
421     check_true(audit, "Stage 5", "Cluster input excludes forbidden columns by code", "is_forbidden_feature_column" in code
    ↪ and "METADATA_COLUMNS" in code, "metadata and label token exclusion", "found", "critical", rel(code_path))
422     check_true(audit, "Stage 5", "Cluster-label classifier uses selected acoustic features", "run_cluster_label_classifier"
    ↪ in code and "pd_subjects[features]" in code, "features only; labels are cluster labels", "found", "high",
    ↪ rel(code_path))
423     verify_metric_summary(audit, "Stage 5", classifier, ["model"], "fold", "fold", ["accuracy", "balanced_accuracy",
    ↪ "f1_macro", "f1_weighted", "roc_auc"], ddof=0, evidence_file=rel(classifier_path), summary_column_is_fold=True)
424     audit_report_boundary(audit, "Stage 5", report, rel(report_path), expected_classifier_phrase="cluster-label classifier")
425
426
427 def audit_stage6(audit: Audit, tables: dict[str, pd.DataFrame]) -> None:
428     code_path = SRC_DIR / "stage6_pd_six_cluster_phenotyping.py"
429     code = safe_read_text(code_path)
430     metrics_path = RESULTS_DIR / "stage6_six_cluster_metrics.csv"
431     assign_path = RESULTS_DIR / "stage6_six_cluster_assignments_dataset1_pd.csv"
432     input_audit_path = RESULTS_DIR / "stage6_six_cluster_input_audit.csv"
433     crosswalk_path = RESULTS_DIR / "stage6_stage5_crosswalk.csv"
434     classifier_path = RESULTS_DIR / "stage6_six_cluster_label_classifier_metrics.csv"
435     k_scan_path = RESULTS_DIR / "stage6_k_scan_metrics.csv"
436     report_path = RESULTS_DIR / "stage6_six_cluster_report.md"
437     metrics = read_csv(metrics_path)
438     assignments = read_csv(assign_path)
439     input_audit = read_csv(input_audit_path)
440     crosswalk = read_csv(crosswalk_path)
441     classifier = read_csv(classifier_path)
442     k_scan = read_csv(k_scan_path)
443     report = safe_read_text(report_path)
444     tables["stage6_assign"] = assignments
445
446     check_true(audit, "Stage 6", "Uses Stage 4 subject-level feature table",
    ↪ "stage4_subject_level_feature_table_dataset1.csv" in code, "Stage 4 subject table", "found", "critical",
    ↪ rel(code_path))
447     check_equal(audit, "Stage 6", "PD-only assignment row count", 188, assignments.shape[0], "critical", rel(assign_path))
448     check_equal(audit, "Stage 6", "Assignments class distribution", {1: 188}, value_counts_dict(assignments["class"]),
    ↪ "critical", rel(assign_path))
449     check_equal(audit, "Stage 6", "Assignment duplicated id count", 0, int(assignments["id"].duplicated().sum()),
    ↪ "critical", rel(assign_path))
450     check_equal(audit, "Stage 6", "Input audit acoustic feature count", 752, int(input_audit.loc[0,
    ↪ "acoustic_feature_count"]), "critical", rel(input_audit_path))
451     check_equal(audit, "Stage 6", "Stage 5 overlap subject count", 188, int(input_audit.loc[0,
    ↪ "stage5_label_overlap_subject_count"]), "high", rel(input_audit_path))
452     check_equal(audit, "Stage 6", "Recommended flag count", 1, int(metrics["recommended_flag"].sum()), "critical",
    ↪ rel(metrics_path))
453     recommended = metrics.loc[metrics["recommended_flag"] == True].iloc[0]
454     check_equal(audit, "Stage 6", "Recommended scheme", ("top20_biomarkers", "pca_5", "AgglomerativeClustering"),
    ↪ (recommended["feature_set"], recommended["representation"], recommended["clustering_method"]), "critical",
    ↪ rel(metrics_path))
455     check_equal(audit, "Stage 6", "Recommended cluster sizes", {0: 13, 1: 55, 2: 34, 3: 28, 4: 25, 5: 33},
    ↪ value_counts_dict(assignments["stage6_cluster"]), "critical", rel(assign_path))
456     check_equal(audit, "Stage 6", "Minimum cluster size", 13, int(assignments["stage6_cluster"].value_counts().min()),
    ↪ "high", rel(assign_path))
457     check_true(audit, "Stage 6", "k=2 to k=8 reference scan exists", sorted(k_scan["k"].unique().tolist()) == list(range(2,
    ↪ 9)), "k values 2..8", sorted(k_scan["k"].unique().tolist()), "medium", rel(k_scan_path))
458     check_true(audit, "Stage 6", "50-run stability analysis recorded", int(metrics["stability_n_runs"].min()) == 50 and
    ↪ int(k_scan["stability_n_runs"].min()) == 50, "stability_n_runs == 50",
    ↪ f"k6={metrics['stability_n_runs'].unique().tolist()}; scan={k_scan['stability_n_runs'].unique().tolist()}",
    ↪ "medium", f"{rel(metrics_path)}; {rel(k_scan_path)}")
459     check_true(audit, "Stage 6", "Cluster input excludes forbidden columns by code", "is_forbidden_feature_column" in code
    ↪ and "METADATA_COLUMNS" in code, "metadata and label token exclusion", "found", "critical", rel(code_path))
460     stage5_selection_note = "stage5_nmi" in code and "selection_score" in code
461     audit.add_check(
462         "Stage 6",
463         "Stage 5 cluster use limited to alignment/crosswalk/recommendation scoring",
464         "not used as clustering input",
465         "stage5_nmi appears in recommendation scoring" if stage5_selection_note else "no selection scoring use found",
466         "warning" if stage5_selection_note else "pass",
467         "medium",
468         rel(code_path),
469         "This is not clustering input leakage, but it should be disclosed because Stage 5 labels influence candidate
    ↪ recommendation.",
470     )
471     verify_crosswalk(audit, assignments, crosswalk, rel(crosswalk_path))

```

```

472     verify_metric_summary(audit, "Stage 6", classifier, ["model"], "fold", "fold", ["accuracy", "balanced_accuracy",
↪ "f1_macro", "f1_weighted", "precision_macro", "recall_macro", "multiclass_roc_auc_ovr"], ddof=0,
↪ evidence_file=rel(classifier_path), summary_column_is_fold=True)
473     audit_report_boundary(audit, "Stage 6", report, rel(report_path), expected_classifier_phrase="six-cluster-label
↪ classifier")
474
475
476 def audit_cross_stage(audit: Audit, tables: dict[str, pd.DataFrame]) -> None:
477     raw1 = tables["raw1"]
478     d1_ready = tables["d1_ready"]
479     subject = tables["stage4_subject"]
480     stage5 = tables["stage5_assign"]
481     stage6 = tables["stage6_assign"]
482     top20 = tables["stage4_top20"]
483     top50 = tables["stage4_top50"]
484
485     lineage = {
486         "raw_dataset1_rows": int(raw1.shape[0]),
487         "deduplicated_rows": int(d1_ready.shape[0]),
488         "subject_level_rows": int(subject.shape[0]),
489         "pd_subject_rows": int((subject["class"] == 1).sum()),
490         "stage5_assignment_rows": int(stage5.shape[0]),
491         "stage6_assignment_rows": int(stage6.shape[0]),
492     }
493     expected = {
494         "raw_dataset1_rows": 756,
495         "deduplicated_rows": 755,
496         "subject_level_rows": 252,
497         "pd_subject_rows": 188,
498         "stage5_assignment_rows": 188,
499         "stage6_assignment_rows": 188,
500     }
501     check_equal(audit, "Cross-stage", "Dataset 1 lineage", expected, lineage, "critical", "data/results")
502     check_equal(audit, "Cross-stage", "Stage 5 and Stage 6 id set equality", True, normalize_id_set(stage5["id"]) ==
↪ normalize_id_set(stage6["id"]), "critical", "results")
503
504     top20_features = top20["feature"].astype(str).tolist()
505     top50_features = top50["feature"].astype(str).tolist()
506     check_equal(audit, "Cross-stage", "Top20 equals first 20 of Top50", top20_features, top50_features[:20], "high",
↪ "results")
507     missing_top_features = sorted((set(top20_features) | set(top50_features)) - set(subject.columns))
508     check_equal(audit, "Cross-stage", "Top20/Top50 exist in subject table", [], missing_top_features, "critical", "results")
509     all_acoustic = numeric_feature_columns(subject, {"id", "gender", "class", "sex_male", "record_count"})
510     check_equal(audit, "Cross-stage", "All-acoustic feature count", 752, len(all_acoustic), "critical", rel(RESULTS_DIR /
↪ "stage4_subject_level_feature_table_dataset1.csv"))
511
512
513 def verify_metric_summary(
514     audit: Audit,
515     stage: str,
516     metrics: pd.DataFrame,
517     group_cols: list[str],
518     summary_col: str,
519     fold_marker_col: str,
520     metric_cols: list[str],
521     ddof: int,
522     evidence_file: str,
523     summary_column_is_fold: bool = False,
524 ) -> None:
525     mismatches: list[str] = []
526     for group_values, group in metrics.groupby(group_cols, dropna=False):
527         if not isinstance(group_values, tuple):
528             group_values = (group_values,)
529         if summary_column_is_fold:
530             fold_rows = group[~group[summary_col].astype(str).isin(["mean", "std"])]
531             mean_rows = group[group[summary_col].astype(str) == "mean"]
532             std_rows = group[group[summary_col].astype(str) == "std"]
533         else:
534             fold_rows = group[group[summary_col].astype(str) == fold_marker_col]
535             mean_rows = group[group[summary_col].astype(str) == "mean"]
536             std_rows = group[group[summary_col].astype(str) == "std"]
537         if mean_rows.empty or std_rows.empty:
538             mismatches.append(f"{group_values}: missing mean/std")
539             continue
540         for metric in metric_cols:
541             observed_mean = float(mean_rows.iloc[0][metric])
542             observed_std = float(std_rows.iloc[0][metric])
543             expected_mean = float(fold_rows[metric].mean(skipna=True))
544             expected_std = float(fold_rows[metric].std(ddof=ddof, skipna=True))
545             if not floats_close(observed_mean, expected_mean) or not floats_close(observed_std, expected_std):
546                 mismatches.append(
547                     f"{group_values}/{metric}: mean {observed_mean} vs {expected_mean}; std {observed_std} vs {expected_std}"
548                 )
549     check_equal(
550         audit,
551         stage,
552         "Metric summary rows equal fold recomputation",
553         [],
554         mismatches[:10],
555         "high",
556         evidence_file,

```

```

557         notes=f"Checked {len(metric_cols)} metrics per group; ddof={ddof}.",
558     )
559
560
561 def floats_close(left: float, right: float, tol: float = 1e-10) -> bool:
562     if math.isnan(left) and math.isnan(right):
563         return True
564     return abs(left - right) <= tol
565
566
567 def verify_crosswalk(audit: Audit, assignments: pd.DataFrame, crosswalk: pd.DataFrame, evidence_file: str) -> None:
568     observed = (
569         assignments.groupby(["stage5_cluster", "stage6_cluster"])
570         .size()
571         .reset_index(name="count")
572     )
573     merged = crosswalk[["stage5_cluster", "stage6_cluster", "count"]].merge(
574         observed,
575         on=["stage5_cluster", "stage6_cluster"],
576         how="outer",
577         suffixes=("_file", "_recomputed"),
578     ).fillna(0)
579     mismatches = merged.loc[merged["count_file"].astype(int) != merged["count_recomputed"].astype(int)]
580     check_equal(
581         audit,
582         "Stage 6",
583         "Stage 5 x Stage 6 crosswalk counts",
584         0,
585         int(mismatches.shape[0]),
586         "high",
587         evidence_file,
588     )
589
590
591 def audit_report_boundary(
592     audit: Audit,
593     stage: str,
594     report: str,
595     evidence_file: str,
596     expected_classifier_phrase: str,
597 ) -> None:
598     lower = report.lower()
599     has_classifier_phrase = expected_classifier_phrase.lower() in lower
600     check_true(
601         audit,
602         stage,
603         "Report names classifier as cluster-label classifier",
604         has_classifier_phrase,
605         expected_classifier_phrase,
606         "found" if has_classifier_phrase else "missing",
607         "high",
608         evidence_file,
609         fail_status="warning",
610     )
611     direct_symptom_terms = [term for term in CLINICAL_SYMPTOM_TERMS if term.lower() in lower]
612     guarded = any(
613         marker in report
614         for marker in [
615             "not a clinical",
616             "not clinical",
617             "not a clinical symptom diagnosis",
618             "cannot",
619             "不能",
620             "不是",
621             "不对应",
622             "不直接",
623             "而非",
624         ]
625     )
626     status = "pass" if not direct_symptom_terms or guarded else "fail"
627     audit.add_check(
628         stage,
629         "Report avoids treating clusters as true clinical labels",
630         "no unguarded clinical symptom labels",
631         direct_symptom_terms if direct_symptom_terms else "none found",
632         status,
633         "critical",
634         evidence_file,
635         "Clinical terms are acceptable only when explicitly negated or guarded.",
636     )
637     replacement_count = report.count("\ufffd")
638     audit.add_check(
639         stage,
640         "Report UTF-8 replacement character check",
641         "0 replacement characters",
642         replacement_count,
643         "warning" if replacement_count else "pass",
644         "low",
645         evidence_file,
646         "A warning here indicates display/encoding risk, not a modeling redline.",
647     )
648

```

```

649
650 def classify_static_match(file_path: Path, line_no: int, keyword: str, line: str) -> tuple[str, str, str]:
651     normalized = line.strip()
652     file_name = file_path.name
653     risk = "safe"
654     needs = "no"
655     explanation = "Pattern is used for configuration, reporting, grouping, or explicit exclusion."
656
657     if keyword in {"SMOTE", "SelectKBest", "cross_val_score", "GridSearchCV"}:
658         if keyword in normalized and not normalized.startswith("- No"):
659             risk = "warning"
660             explanation = f"{keyword} appears; verify it is not used outside grouped CV."
661     if keyword == "train_test_split":
662         if "No row-wise random" in normalized or normalized.startswith("- No"):
663             risk = "safe"
664             explanation = "Only appears in report guardrail text stating it was not used."
665         else:
666             risk = "fail"
667             needs = "yes"
668             explanation = "Row-level train_test_split would violate the Stage 3 subject grouping redline."
669     if keyword in {"StandardScaler(", "PCA(", "fit_transform)":
670         if file_name in {"stage3_baseline_models.py", "stage4_biomarker_identification.py"}:
671             if "Pipeline" in normalized or "scaler" in normalized or keyword == "PCA(":
672                 risk = "safe"
673                 explanation = "Supervised-model scaling is inside sklearn Pipeline or no PCA is used in Stage 3/4."
674             else:
675                 risk = "warning"
676                 explanation = "Verify fitting is restricted to training folds."
677         elif file_name in {"stage5_pd_two_cluster_phenotyping.py", "stage6_pd_six_cluster_phenotyping.py"}:
678             risk = "warning"
679             explanation = "Unsupervised PD-only clustering fits scaler/PCA on the PD subject subset; this is acceptable only
        ↳ because it is not supervised CV and forbidden columns are excluded."
680         else:
681             risk = "warning"
682             explanation = "Preprocessing fit pattern needs context review."
683     if keyword == "LogisticRegressionCV":
684         risk = "safe" if "Pipeline" in safe_nearby_text(file_path, line_no, radius=12) else "warning"
685         explanation = "Stage 4 L1 stability selection uses LogisticRegressionCV inside a fold-level Pipeline." if risk ==
        ↳ "safe" else "Verify LogisticRegressionCV is inside a fold-level Pipeline."
686     if keyword == "permutation_importance":
687         risk = "safe"
688         explanation = "Project implements held-out permutation importance rather than fitting on all data."
689     if keyword in {"GroupKFold", "StratifiedGroupKFold"}:
690         risk = "safe"
691         explanation = "Grouped splitters protect subject-level train/test separation."
692     if keyword in {"sex_male", "gender", "Gender", "class", "Status", "Recording", "id", "ID", "cluster", "stage5_cluster",
        ↳ "stage6_cluster"}:
693         nearby = safe_nearby_text(file_path, line_no, radius=3).lower()
694         if any(token in nearby for token in ["excluded", "not enter", "不进入", "metadata", "required", "post", "crosstab",
        ↳ "assignments", "audit"]):
695             risk = "safe"
696             explanation = "Metadata/label column appears in validation, exclusion, audit, assignment, or post-hoc
        ↳ description."
697         elif file_name == "stage6_pd_six_cluster_phenotyping.py" and keyword == "stage5_cluster":
698             risk = "warning"
699             explanation = "Stage 5 labels are aligned for crosswalk and recommendation scoring; audit verifies they are not
        ↳ clustering inputs."
700     elif keyword in {"class", "Status", "cluster", "stage5_cluster", "stage6_cluster"} and any(token in nearby for token
        ↳ in ["y =", "labels", "label"]):
701         risk = "warning"
702         explanation = "Label appears as a target or derived assignment; verify it is not part of feature matrices."
703     else:
704         risk = "safe"
705         explanation = "Metadata/label token occurrence is not evidence of feature leakage by itself."
706     return risk, explanation, needs
707
708
709 def safe_nearby_text(file_path: Path, line_no: int, radius: int) -> str:
710     lines = safe_read_text(file_path).splitlines()
711     start = max(0, line_no - radius - 1)
712     end = min(len(lines), line_no + radius)
713     return "\n".join(lines[start:end])
714
715
716 def audit_static_patterns(audit: Audit) -> None:
717     for file_path in sorted(SRC_DIR.glob("*.py")):
718         if file_path.name == Path(__file__).name:
719             continue
720         lines = safe_read_text(file_path).splitlines()
721         for line_no, line in enumerate(lines, start=1):
722             for keyword in STATIC_KEYWORDS:
723                 if keyword in line:
724                     risk, explanation, needs = classify_static_match(file_path, line_no, keyword, line)
725                     audit.static_rows.append(
726                         {
727                             "file": rel(file_path),
728                             "line": line_no,
729                             "keyword": keyword,
730                             "code_snippet": line.strip(),
731                             "risk_level": risk,
732                             "explanation": explanation,

```

```

733         "needs_modification": needs,
734     }
735 )
736 fail_count = sum(1 for row in audit.static_rows if row["risk_level"] == "fail")
737 warning_count = sum(1 for row in audit.static_rows if row["risk_level"] == "warning")
738 audit.add_check(
739     "Static code audit",
740     "High-risk pattern scan fail count",
741     0,
742     fail_count,
743     "pass" if fail_count == 0 else "fail",
744     "critical",
745     "results/audit_static_code_risk_patterns.csv",
746     f"warning_count={warning_count}; warnings require explanation but are not automatic redlines.",
747 )
748
749
750 def write_outputs(audit: Audit) -> None:
751     RESULTS_DIR.mkdir(exist_ok=True)
752     checks = pd.DataFrame(audit.rows, columns=CHECK_COLUMNS)
753     static = pd.DataFrame(audit.static_rows)
754
755     checks_path = RESULTS_DIR / "audit_all_stages_checks.csv"
756     static_path = RESULTS_DIR / "audit_static_code_risk_patterns.csv"
757     feature_path = RESULTS_DIR / "audit_feature_leakage_checks.csv"
758     lineage_path = RESULTS_DIR / "audit_subject_lineage_checks.csv"
759     metric_path = RESULTS_DIR / "audit_metric_consistency_checks.csv"
760     report_path = RESULTS_DIR / "audit_all_stages_logic_report.md"
761
762     checks.to_csv(checks_path, index=False, encoding=OUTPUT_CSV_ENCODING)
763     static.to_csv(static_path, index=False, encoding=OUTPUT_CSV_ENCODING)
764
765     feature_mask = checks["check_name"].str.contains(
766         "feature|forbidden|sex_male|metadata|classifier|cluster input|Report avoids",
767         case=False,
768         regex=True,
769     ) | checks["stage"].eq("Static code audit")
770     checks.loc[feature_mask].to_csv(feature_path, index=False, encoding=OUTPUT_CSV_ENCODING)
771
772     lineage_mask = checks["check_name"].str.contains(
773         "lineage|row count|subject|PD-only|id set|shape|class distribution|record count",
774         case=False,
775         regex=True,
776     )
777     checks.loc[lineage_mask].to_csv(lineage_path, index=False, encoding=OUTPUT_CSV_ENCODING)
778
779     metric_mask = checks["check_name"].str.contains(
780         "Metric|metrics|Recommended|cluster sizes|crosswalk|stability|k=2",
781         case=False,
782         regex=True,
783     )
784     checks.loc[metric_mask].to_csv(metric_path, index=False, encoding=OUTPUT_CSV_ENCODING)
785
786     report_path.write_text(build_report(checks, static), encoding="utf-8")
787
788
789 def build_report(checks: pd.DataFrame, static: pd.DataFrame) -> str:
790     critical_fails = checks[(checks["status"] == "fail") & (checks["severity"].isin(["critical", "high"]))]
791     all_fails = checks[checks["status"] == "fail"]
792     warnings = checks[checks["status"] == "warning"]
793     static_warnings = static[static["risk_level"] == "warning"] if not static.empty else pd.DataFrame()
794     static_fails = static[static["risk_level"] == "fail"] if not static.empty else pd.DataFrame()
795
796     if not all_fails.empty or not static_fails.empty:
797         conclusion = "FAIL"
798         rerun = "At least one redline or high-severity check failed; rerun the affected stage after the minimal fix."
799     elif not warnings.empty or not static_warnings.empty:
800         conclusion = "CONDITIONAL PASS"
801         rerun = "No core stage rerun is required based on this audit; address warnings as documentation or boundary  

802             ↳ clarifications unless new evidence changes them."
803     else:
804         conclusion = "PASS"
805         rerun = "No stage rerun is required."
806
807     by_status = checks["status"].value_counts().to_dict()
808     by_stage = checks.groupby(["stage", "status"]).size().unstack(fill_value=0).reset_index()
809
810     lines = [
811         "# Six-Stage Logic Audit Report",
812         "",
813         f"Final conclusion: `{conclusion}`",
814         "",
815         "## Rerun Decision",
816         "",
817         rerun,
818         "",
819         "## Check Summary",
820         "",
821         f"- Total checks: `{checks.shape[0]}`",
822         f"- Status counts: `{by_status}`",
823         f"- Static code findings: `{static.shape[0]} rows; warnings `{0 if static.empty else int((static['risk_level'] == 'warning').sum())}, fails `{0 if static.empty else int((static['risk_level'] == 'fail').sum())}`",

```

```

823     "",
824     markdown_table(by_stage),
825     "",
826     "## Redline Failures",
827     "",
828 ]
829 if critical_fails.empty and static_fails.empty:
830     lines.append("No critical/high redline failures were found.")
831 else:
832     fail_cols = ["stage", "check_name", "expected", "observed", "severity", "evidence_file", "notes"]
833     if not critical_fails.empty:
834         lines.append(markdown_table(critical_fails[fail_cols]))
835     if not static_fails.empty:
836         lines.append(markdown_table(static_fails[["file", "line", "keyword", "risk_level", "explanation",
837         ↪ "needs_modification"]].head(50)))
838
839 lines.extend(
840     [
841         "",
842         "## Warnings",
843         "",
844     ]
845 )
846 warning_cols = ["stage", "check_name", "observed", "severity", "evidence_file", "notes"]
847 if warnings.empty and static_warnings.empty:
848     lines.append("No warnings were recorded.")
849 else:
850     if not warnings.empty:
851         lines.append(markdown_table(warnings[warning_cols].head(80)))
852     if not static_warnings.empty:
853         lines.extend(
854             [
855                 "",
856                 "Static pattern warnings are explainability items, not automatic redlines:",
857                 "",
858                 markdown_table(static_warnings[["file", "line", "keyword", "risk_level", "explanation",
859                 ↪ "needs_modification"]].head(80)),
860             ]
861         )
862
863 lines.extend(
864     [
865         "",
866         "## Output Files",
867         "",
868         "- `results/audit_all_stages_logic_report.md`",
869         "- `results/audit_all_stages_checks.csv`",
870         "- `results/audit_feature_leakage_checks.csv`",
871         "- `results/audit_subject_lineage_checks.csv`",
872         "- `results/audit_metric_consistency_checks.csv`",
873         "- `results/audit_static_code_risk_patterns.csv`",
874         "",
875         "## Minimal Fix Guidance",
876         "",
877     ]
878 )
879 if conclusion == "FAIL":
880     lines.append("Fix only the failed redline rows above, then rerun the affected stage and this independent audit.")
881 elif conclusion == "CONDITIONAL PASS":
882     lines.append("Keep the sealed modeling outputs intact. Recommended follow-up is limited to clarifying warnings,
883     ↪ especially documentation around Stage 6 candidate recommendation scoring and any report encoding/display
884     ↪ issues.")
885 else:
886     lines.append("No fixes are required by this audit.")
887 return "\n".join(lines)
888
889 def markdown_table(df: pd.DataFrame) -> str:
890     if df.empty:
891         return ""
892     headers = list(df.columns)
893     rows = ["| " + " | ".join(headers) + " |", "| " + " | ".join(["---"] * len(headers)) + " |"]
894     for _, row in df.iterrows():
895         values = []
896         for column in headers:
897             value = row[column]
898             if isinstance(value, float):
899                 values.append(" " if math.isnan(value) else f"{value:.6g}")
900             else:
901                 text = str(value)
902                 text = text.replace("\n", " ").replace("|", "\\|")
903                 values.append(text)
904         rows.append("| " + " | ".join(values) + " |")
905     return "\n".join(rows)
906
907 def main() -> None:
908     audit = Audit()
909     tables = audit_stage1_and_stage2(audit)
910     audit_stage3(audit)
911     audit_stage4(audit, tables)
912     audit_stage5(audit, tables)

```

```
911     audit_stage6(audit, tables)
912     audit_cross_stage(audit, tables)
913     audit_static_patterns(audit)
914     write_outputs(audit)
915
916
917 if __name__ == "__main__":
918     main()
```